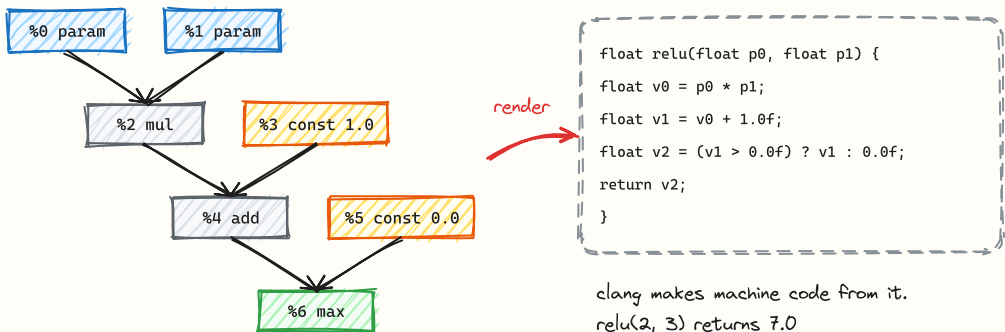


The first compiler

uops, rewrites and C, built from zero. Every program in this book was run, and every number was measured.



the graph for $\max(x*y + 1, 0)$. an arrow goes from an input to the node that uses it.

For anyone who wants to know what a compiler does

Contents

Start here: the short path

s1	See a program as text	8
s2	Run it	10
s3	Write C from it	11
s4	Compile it and run it	13
s5	The same node is one node	15
s6	Simplify it	16
s7	A rule that looks right and is wrong	18
s8	Integers wrap around	20
s9	Index math and ranges	21
s10	Memory: loads and stores	23
s11	Order matters	26
s12	Test it against itself	28
s13	Prove it	30
s14	What the real one writes	32

The deep book, part 1: the problem

1	What we are building	35
2	The numbers a machine really has	41
3	What every op means	51

Part 2: the graph

4	The uop	59
5	Hash consing	67
6	Types, shapes, address spaces and ranges	74

Part 3: rewriting

7	Patterns and rules	83
8	The rewrite engine	91
9	Running programs by folding	99
10	Simplifying floats	105
11	Simplifying integers	112

Part 4: from graph to machine

12	Rendering C	123
13	Compiling and running	132
14	Memory	139

Part 5: trust, and the real thing

15	How we know it works	150
16	The same ideas in tinygrad	160

The back: answers, references and measurements

A	Answers to the exercises	169
B	Op reference	178
C	Commands, counts and proofs	185
D	Measurements	190
E	Reading list and study plan	192
F	Glossary	196

How to read this book

This book teaches one thing: how to build a small compiler that takes a program written as a graph of arithmetic, simplifies it, turns it into C, compiles that C, and runs it. It is the first module of a course called compilers for machine learning, and it covers the first two weeks of that course.

You do not need to know what a compiler is. You need to be able to read Python and a little C. Every other idea is defined the first time it appears, in plain words, and then used the same way every time after.

Two ways in

The book has two layers, and you choose where to start.

The short path comes first. It is 14 steps, about two pages each. Each step is one small program in the folder `code/short/`, its real output, and a few sentences on what happened. The first step prints a program as text. The last step looks at the C that the real tinygrad writes. In between you run the program, write C from it, compile it, simplify it, break it with a wrong rule, make it wrap around, simplify index math, handle memory, test it against itself, and prove rules with a theorem prover. The short path is enough to understand what a compiler does and why the details are hard. You can read it in one sitting.

the short path: 14 steps



blue: build and run one program. gold: make it smaller. red: how it goes wrong.
grey: memory and the real compiler. green: find out whether it is right. ch: main chapters.

The deep book is the 16 chapters after it. Each short step points to the chapters that explain it in full: every op, every rule, every number, and the exercises. The deep book is where the guarantees are. Read it when you want to build the compiler yourself or to change it.

If you are not sure, read the short path. When a step makes you want to know why, follow its “go deeper” line.

What you get

A working compiler of about 1,500 lines of Python, and the reasons behind every line. You can read the code in the order it is explained. You can run it. You can break it on purpose and watch the tests catch you. The book also shows where each piece lives in tinygrad, the open source compiler that this course is modelled on, so you can read the real thing afterwards.

What "verified" means here

The front of a book like this is where authors promise things. Here are the promises, and the evidence for each one is in the appendices.

- Every program listing is code that ran. Most listings are cut out of the real source files by a script. The few short snippets inside the prose are copied by hand and were checked against the source.
- Every program output shown is the output of the command printed next to it. Blocks that show part of an output are cut by a script, and a few short snippets in the prose are copied from it by hand.
- The compiler is checked against two independent references: NumPy, and separate C code written for the tests. The test suite makes more than 2.1 million checks. Appendix C lists the counts.
- 59 claims about rewrite rules were settled with a theorem prover (z3), over 32 bit integers and 32 bit floats, except 8 integer proofs that are over mathematical integers with a no-wrap guard, which chapter 11 explains. The correct rules are proved for every input. The wrong ones come with an input where they fail.
- The test suite itself was tested. 19 bugs were put into the compiler on purpose, one at a time, and all 19 were caught. The first version of the suite was much weaker, and chapter 15 tells how that was fixed.
- Every performance number was measured on one machine, which is named in appendix d, with the benchmark program included. Your numbers will differ. The ratios will not change by much.
- Anything that could not be checked is marked where it appears, and listed in chapter 15.

How to use the code

Everything is in one folder. The compiler is the `uopc` package. The tests are in `tests`. The programs that produce the book's outputs are in `demos`, and their saved output is in `out`. Appendix C has the exact commands. You need Python 3.10 or newer, NumPy, z3, and clang. The book was built with Python 3.13 and clang 18.

Read a chapter, run the code it shows, then do the exercises. The exercises are small. Each has a checked solution in appendix A.

Conventions

A line starting with `%` followed by a number, like `%4 = add %2, %3`, is one node of a program in the text format. Code is in a monospace font. A name in a monospace font is a name in the code. A “uop” is one node of the graph. An “op” is the kind of node. The word “rewrite” always means replacing a node by an equal node.

Each chapter ends with a short list of where to learn more. Appendix e collects them into a reading list with a study plan.

Start here
The short path

Step 1

See a program as text

This book is the first module of a course called compilers for machine learning, designed by george hotz (geohot). The module covers the first two weeks of the course. It has two layers: this short path of 14 steps, and after it 16 long chapters. When a step says “chapter 4” it means one of those long chapters.

This is the first step. Each step is one small program, its real output, and a few sentences on what happened. Together they walk through everything a small compiler does. After step 14 you will have seen the main ideas of this book once, in their smallest form. The long chapters after that explain each idea in depth.

Every program here is in the folder `code/short/` and you can run it. Nothing is explained with a comparison to something else. The things are named and shown.

Start with the smallest program that has any content: add two floats.

step 1: build $x + y$ and print it in the text format

```
import _here
from uopc import F32, CallArg, Ops, ParamArg, UOp
from uopc.textfmt import dumps, loads

x = UOp(Ops.PARAM, (), ParamArg(0, F32))    # the first input, a float
y = UOp(Ops.PARAM, (), ParamArg(1, F32))    # the second input
total = x + y                               # builds an ADD node. nothing is
computed.
add = UOp(Ops.CALL, (total, UOp.const(0.0), UOp.const(0.0)), CallArg("add"))
print(dumps(add))
print("read back as the same graph:", loads(dumps(add)) is add)    # text in, the
very same node out
```

The first line, `import _here`, only makes the compiler package `uopc` findable from this folder. Ignore it in every step.

This builds a graph of nodes. `x` and `y` are inputs. `x + y` does not add anything. It makes one node, called `add`, that points at the two input nodes. The last line makes a `call` node. A call has a body, which is its first input, and then the values the function is called with. Here the body is the `add` node. The two arguments are the constant `0.0`, twice. They only

have to have the right type, `f32`, because the compiler checks that the arguments match the inputs the body uses. Their values are not used in the steps that follow. Every program that is compiled in this book is the body of a call.

Now print it:

the program as text · output of `short/s01-see.py`

```
; uop v1
%0 = param : dtype=f32 slot=0
%1 = param : dtype=f32 slot=1
%2 = add %0, %1
%3 = const : dtype=f32 value=0.0
%4 = call %2, %3, %3 : name=add

read back as the same graph: True
```

That is the whole program: a first line that names the version of the format, and five nodes.

Read one line. `%2 = add %0, %1` says: node number 2 is an `add`, and its two inputs are node 0 and node 1. A line starts with a node number, then `=`, then the name of the operation, then the numbers of the nodes it reads. Anything after a `:` is extra information that is not a node, for example `dtype=f32` (a 32 bit float) or `slot=0` (this is input number 0).

Things to notice:

- There are no variable names, and the graph does not say when anything runs. A node only says what it reads.
- The numbers are only names for nodes in the text. A node can only read nodes with a smaller number, so a line never refers to something that has not been written yet. This makes the text easy to check and easy to read from the top.
- The format is plain text. You can write it by hand, save it, and read it back. The last line printed above shows that reading the text back gives the very same node, not a copy. The code that writes and reads the text is `uopc/textfmt.py`, 130 lines.

The course uses a draft text format with the same shape, `%n = op %a, %b : key=value`. Our format is not copied from it. It was designed to the same idea and then checked, and where this book writes `%0 = param : dtype=f32 slot=0` the draft may write something slightly different. The draft is early and can change. Because all text handling sits in one file, changing it would not touch the rest of the compiler.

Go deeper

Chapter 4 defines the node, called a `UOp`, with its `op`, its inputs and its extra information. Chapter 6 defines the properties computed from those, such as its type. Chapter 3 says exactly what each operation means.

Try it

1. Change `x + y` to `x * y` and run the file. Which line changes? Answer: `%2 = mul %0, %1`, and nothing else.

Step 2

Run it

A graph is only a description. To find out what it computes, something has to walk it. The simplest walker is an interpreter written in Python. It visits each node after its inputs and computes the value.

step 2: run the graph with the python interpreter

```
import _here
from uopc import F32, Ops, ParamArg, UOp
from uopc.interp import evaluate

x = UOp(Ops.PARAM, (), ParamArg(0, F32))
y = UOp(Ops.PARAM, (), ParamArg(1, F32))
total = x + y
print(evaluate(total, {0: 1.5, 1: 2.25}))    # x = 1.5, y = 2.25
print(evaluate(total, {0: 1e8, 1: 1.0}))    # a float32 cannot hold 1000000001
```

output of `short/s02_run.py`

```
3.75
100000000.0
```

The second argument of `evaluate` says which value each input has: `{0: 1.5, 1: 2.25}` means input slot 0 is 1.5 and slot 1 is 2.25.

The first printed line is what you expect, $1.5 + 2.25 = 3.75$.

The second line is the first surprise. `1e8 + 1.0` should be 100000001, and the program prints `100000000.0`. A float32 has 24 bits for the digits of the number. Near one hundred million, the gap between two neighbouring float32 values is 8. 100000001 is not one of them, so the sum is rounded to the nearest one, which is 100000000. The interpreter is not wrong. It is doing what 32 bit float addition does, and chapter 2 shows why.

This matters for a compiler because the compiler must produce the same answers as this interpreter, including the rounding. An interpreter that does exact arithmetic would be easier to write and would be useless as a reference.

The interpreter is slow. Chapter 13 measures how slow: about 3,800 nanoseconds per element on the machine used for this book, against about 1 nanosecond per element for compiled C, when the loop over the elements is inside the C. That gap is the reason compilers exist.

Go deeper

Chapter 2 explains float32 and int32 bit by bit. Chapter 3 gives the exact meaning of every op, which the interpreter implements. Chapter 9 shows a second way to run a graph, by folding.

Try it

1. Use `16777216.0` and `1.0` in place of `1e8` and `1.0`. What is the result? Answer: `16777216.0`. Above 2^{24} the gap between float32 values is 2, so `16777217` is exactly halfway between two values, and a tie goes to the one with the even last digit in binary, which is `16777216`. Then try `16777218.0` and `1.0`: the answer is `16777220.0`, by the same rule.

Step 3

Write C from it

The next stage is the renderer. It walks the graph and writes one C statement per node.

step 3: render the graph as c

```
import _here
from uopc import F32, CallArg, Ops, ParamArg, UOp
```

```

from uopc.render import render

x = UOp(Ops.PARAM, (), ParamArg(0, F32))
y = UOp(Ops.PARAM, (), ParamArg(1, F32))
add = UOp(Ops.CALL, (x + y, UOp.const(0.0), UOp.const(0.0)), CallArg("add"))
source = render(add)
print(source[source.index("float add"):])    # skip the helper functions at the top

```

the c that the renderer wrote · output of short/s03_make_c.py

```

float add(float p0, float p1) {
    float v0 = p0 + p1;
    return v0;
}

```

The renderer also writes some helper functions above this one. The program prints only from the function name down.

The function takes the two inputs `p0` and `p1`. The `add` node became

`float v0 = p0 + p1;`. The call node became the function itself, and its body became the `return`. Input nodes and constant nodes have no statement. They are used directly where another node reads them.

This is the whole idea of the renderer. A graph where every node has a name and its inputs have smaller numbers can be written out in node order, and each line only uses names that were already defined. A bigger program just has more lines.

Not every op is a single C operator. Integer division is one example. C division does not match this compiler's contract for negative numbers or zero, so when the renderer cannot show that the dividend is not negative and the divisor is positive, it writes a call to a small helper function. Those helpers are the functions above the one printed here. Chapter 12 lists them.

Go deeper

Chapter 12 is the renderer in full: every op, the helper functions, and the rules that keep float results exact.

Try it

1. Change `x + y` to `x - y` and look at the C. There is no subtract node in this compiler. What does the renderer write instead? Answer: two statements, `float v0 = -p1;` and `float v1 = p0 + v0;`.

Step 4

Compile it and run it

C source is text. To run it, a C compiler turns it into machine code. The runtime does this in four steps: write the source to a file, run `clang` on it, load the result as a shared library, and call the function from Python.

step 4: compile the c with clang, call it, compare with the interpreter

```
import _here
from uopc import F32, CallArg, Ops, ParamArg, UOp
from uopc.interp import evaluate
from uopc.runtime import Program

x = UOp(Ops.PARAM, (), ParamArg(0, F32))
y = UOp(Ops.PARAM, (), ParamArg(1, F32))
add = UOp(Ops.CALL, (x + y, UOp.const(0.0), UOp.const(0.0)), CallArg("add"))
machine = Program(add) # writes the c, runs clang, loads the
                        result
for a, b in [(1.5, 2.25), (1e8, 1.0), (-0.0, 0.0), (float("inf"), float("-inf"))]:
    print(a, b, "clang:", machine(a, b), " interpreter:", evaluate(x + y, {0: a, 1:
    b}))
```

`Program(add)` does the first three steps. Calling `machine(a, b)` does the fourth. The compiler is told to follow strict rules for numbers: do not rearrange, fuse or approximate float operations, and wrap integers around when they overflow. The exact flags are `-O2 -fwrapv -ffp-contract=off -fno-fast-math -std=c11`. Chapter 12 explains `-fwrapv`, `-ffp-contract=off` and `-fno-fast-math`. The other two are the optimisation level and the C language version.

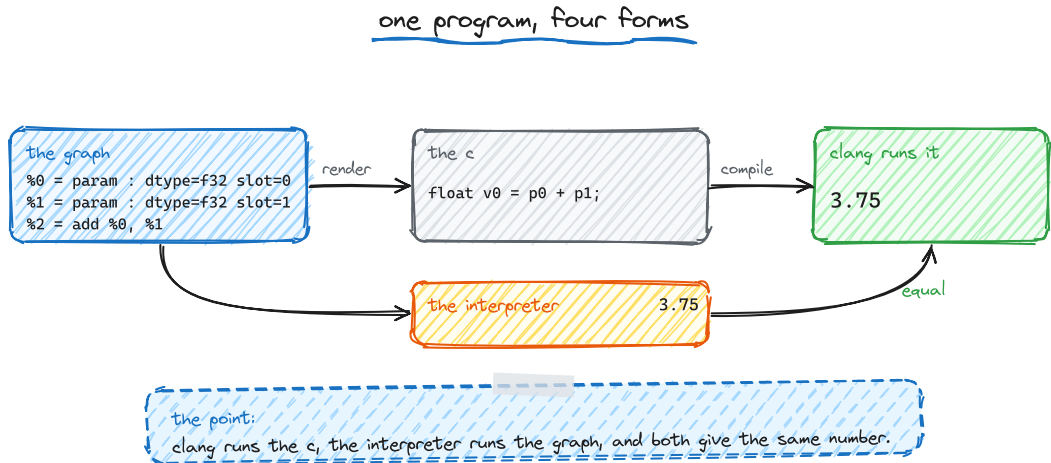
output of `short/s04_run_c.py`

```
1.5 2.25 clang: 3.75 interpreter: 3.75
100000000.0 1.0 clang: 100000000.0 interpreter: 100000000.0
-0.0 0.0 clang: 0.0 interpreter: 0.0
inf -inf clang: nan interpreter: nan
```

Each line shows the two inputs, the answer from clang, and the answer from the interpreter. All four lines agree.

The four input pairs were chosen on purpose. The first is an ordinary sum. The second is the rounding case from step 2. The third adds `-0.0` and `0.0`. float32 has two zeros, and step 7 explains them. The IEEE 754 rule, with its default rounding, gives `0.0`, and both programs agree. The fourth adds infinity (`inf`) and minus infinity, which has no answer, so the result is `nan`, which stands for “not a number”. Both programs return `nan`.

This is the main testing method of the book. There are two independent ways to run a program, the interpreter and the compiled C. Whenever the compiler changes, they must still agree. When they disagree, one of them has a bug. Chapter 15 does this, and more, in more than two million checks.



Go deeper

Chapter 13 covers compiling and caching: the compiled library is cached in a directory under a name that includes the sha256 hash of the C source, the compiler and the flags, so an identical request is not compiled again while that directory exists. It also measures compile time and run time.

Try it

1. Add the pair (3.4e38, 3.4e38) to the list. What do both programs print? Answer: `inf`. The largest float32 is about 3.4028e38, and the sum is larger.

Step 5

The same node is one node

So far each program was tiny. A real program repeats parts of itself, and how the graph handles repeats decides whether the compiler is fast or unusable.

step 5: build the same node twice, then build a repeated sum

```
import _here
from uopc import F32, Ops, ParamArg, UOp

x = UOp(Ops.PARAM, (), ParamArg(0, F32))
y = UOp(Ops.PARAM, (), ParamArg(1, F32))
a = x + y
b = x + y
print("the same object:", a is b)

p = x
for _ in range(18):
    p = p + p # p + p, eighteen times
print("nodes in the graph:", len(p.toposort()))
print("nodes it would need as a tree:", 2 ** 19 - 1)
```

output of short/s05_sharing.py

```
the same object: True
nodes in the graph: 19
nodes it would need as a tree: 524287
```

The first line says that writing `x + y` twice gives the same object in memory, not two equal copies. The node constructor keeps a table of every node it has made, keyed by its operation, its inputs and its extra information. When you ask for a node that is already in the table, you get the existing one. This is called **hash consing**.

The second part builds `p = p + p` eighteen times. Written as a tree, each `p + p` has two copies of everything below it, and the tree has 524,287 nodes. As a graph, each `p + p` is one new node that reads the previous node twice, so the graph has 19 nodes: the input and 18 adds.

Two things follow from this.

1. Size. A repeated part of a program that is built the same way is stored once. The tree already has half a million nodes after 18 repeats, and every further repeat doubles it.
2. Equality. Two nodes built the same way are one object, so asking whether two nodes were built the same way costs one pointer comparison. A rewrite rule that finds `x + y`

in two places finds one node, and rewriting it once fixes both. This does not know that `x + y` and `y + x` are the same computation. They are built differently, so they are two nodes, and a rewrite rule has to relate them.

There is a cost. Each node is a Python object, and chapter 5 measured about 423 bytes per node. The table does not keep nodes alive that nothing else uses.

Go deeper

Chapter 5 is hash consing in full, with the table and its cost in memory.

Try it

1. Change 18 to 30. How many nodes does the graph have, and how many would the tree have? Answer: 31 and 2,147,483,647. The graph is still instant.

Step 6

Simplify it

The compiler can change a graph as long as the new graph computes the same thing. A change is written as a **rule**: when the graph has this shape, replace it with that shape.

step 6: `simplify ((x * 1) + 0) * (2 + 3)`

```
import _here
from uopc import I32, Ops, ParamArg, UOp
from uopc.symbolic import simplify
from uopc.textfmt import dumps

x = UOp(Ops.PARAM, (), ParamArg(0, I32))
before = ((x * 1) + 0) * (UOp.const(2) + 3)
after = simplify(before)
print(len(before.toposort()), "nodes before,", len(after.toposort()), "nodes after")
print(dumps(after))
```

output of `short/s06_simplify.py`

```
9 nodes before, 3 nodes after
; uop v1
%0 = param : dtype=i32 slot=0
```

```
%1 = const : dtype=i32 value=5
%2 = mul %0, %1
```

The program starts with 9 nodes and ends with 3. Three rules were applied, each of them on a part of the graph:

- $x * 1$ becomes x
- $x + 0$ becomes x
- $2 + 3$ becomes 5 , because both sides are constants

The result is $x * 5$, which is the same function with fewer operations.

A rule is a pattern and a replacement. The engine applies rules to every node, from the inputs upward, until nothing changes. Chapter 7 shows how a pattern is written. Chapter 8 shows the engine, what it means for the engine to be finished, and what happens when two rules undo each other and it never finishes.

The important question about any rule is whether it is correct for every input. For 32 bit integers, all three rules above are correct for every input: $x * 1$ and $x + 0$ return x for every x , and $2 + 3$ is just evaluated. Step 7 shows a rule that looks just as simple and is wrong.

Go deeper

Chapter 7 is patterns and rules. Chapter 8 is the engine. Chapter 9 is constant folding, the $2 + 3 = 5$ rule. Chapter 11 is the integer rules.

Try it

1. Simplify $i * 0$ for an i32 input i and print it. Answer: a single constant node, `%0 = const : dtype=i32 value=0.`

Step 7

A rule that looks right and is wrong

For floats, `x + 0.0` looks like it should be replaced with `x`. Adding zero changes nothing. This step shows that it does change something.

step 7: the rule `x + 0.0` becomes `x`, tried under strict and fast-math rules

```
import math
import _here
from uopc import F32, Ops, ParamArg, UOp
from uopc.interp import evaluate
from uopc.symbolic import simplify

x = UOp(Ops.PARAM, (), ParamArg(0, F32))
print("strict rules,    x + 0.0 becomes x:    ", simplify(x + 0.0) is x)
print("strict rules,    x + (-0.0) becomes x:", simplify(x + UOp.const(-0.0)) is x)
print("fast math rules, x + 0.0 becomes x:    ", simplify(x + 0.0, fast_math=True)
      is x)

v = evaluate(x + 0.0, {0: -0.0})          # what the program really does at x =
-0.0
print("x = -0.0, the program gives", v, "with sign", math.copysign(1.0, v))
print("the unsafe rule would give ", -0.0, "with sign", math.copysign(1.0, -0.0))

import numpy as np
with np.errstate(divide="ignore"):
    print("1 / +0.0 =", np.float32(1.0) / np.float32(0.0), "  1 / -0.0 =",
          np.float32(1.0) / np.float32(-0.0))
```

output of `short/s07_wrong_rule.py`

```
strict rules,    x + 0.0 becomes x:    False
strict rules,    x + (-0.0) becomes x: True
fast math rules, x + 0.0 becomes x:    True
x = -0.0, the program gives 0.0 with sign 1.0
the unsafe rule would give  -0.0 with sign -1.0
1 / +0.0 = inf   1 / -0.0 = -inf
```

The first three lines say that the strict rule set does not rewrite `x + 0.0`, does rewrite `x + (-0.0)`, and that the separate fast-math rule set does rewrite `x + 0.0`.

float32 has two zeros, `+0.0` and `-0.0`. They compare equal, but they are different bit patterns. Adding them follows the IEEE 754 rule: under round-to-nearest, `-0.0 + 0.0` is `+0.0`. So for `x = -0.0`, the program `x + 0.0` returns `+0.0`, and the rewritten program `x` returns `-0.0`. The output shows both: sign `1.0` and sign `-1.0`.

Does the sign matter? The last line shows that it can. `1 / +0.0` is `inf` and `1 / -0.0` is `-inf`. A program that divides by the result gets a different answer.

two zeros, two different programs

`x = -0.0`

program: `x + 0.0`
result `+0.0`

→ `1 / result = inf`

rewrite: `x`
result `-0.0`

→ `1 / result = -inf`

the point:

`x + 0.0` is not `x` when `x` is `-0.0`. the two results look equal but divide differently.

`x + (-0.0)` is safe. Adding `-0.0` returns `x` unchanged for every number `x`, including both zeros.

So this compiler keeps two rule lists. The **strict** list is correct for every float32 input and is the default. The **fast-math** list contains rules that are correct only if you accept that there are no `nan` and no infinities, that the sign of zero does not matter, and that the last bit of a rounded result may change. You choose it explicitly with `simplify(root, fast_math=True)`.

The “correct for every input” claims in this step use the contract of chapter 3, in which every `nan` counts as equal to every other `nan`. Real processors may differ in the bits of a `nan`.

This is why the book spends so long on exact rules. A rule that looks correct and is not is a silent bug: the program still runs and returns plausible numbers. The next steps show how to find such bugs by testing and how to rule them out by proof.

Go deeper

Chapter 10 goes through every float rule and says why each is in the strict list or the fast-math list. Chapter 3 defines the float operations.

Try it

1. Is `x * 1.0` rewritten to `x` by the strict rules? Answer: yes. Multiplying by 1.0 returns the same float for every input, including `nan` and both zeros.

Step 8

Integers wrap around

An `i32` holds whole numbers from -2,147,483,648 to 2,147,483,647. A program that adds 1 to the largest one needs a defined answer.

step 8: add 1 to the largest `i32`, in python, in the contract and in clang

```
import _here
from uopc import I32, INT_MAX, CallArg, Ops, ParamArg, UOp
from uopc.interp import evaluate
from uopc.runtime import Program

x = UOp(Ops.PARAM, (), ParamArg(0, I32))
print("python says:", INT_MAX + 1)
print("the contract says:", evaluate(x + 1, {0: INT_MAX}))
inc = UOp(Ops.CALL, (x + 1, UOp.const(0)), CallArg("inc"))
print("clang says:", Program(inc)(INT_MAX))
```

output of `short/s08_wrap.py`

```
python says: 2147483648
the contract says: -2147483648
clang says: -2147483648
```

Python says 2,147,483,648, because Python integers have no size limit. That number does not fit in 32 bits.

The contract of this compiler says what the 32 bit result is: the result is taken modulo 2^{32} and read as a signed number, so it wraps to -2,147,483,648. This is called two's complement wrap-around. The interpreter implements it, and the clang result agrees.

In C, the language rules say signed integer overflow is **undefined**. A C compiler is allowed to assume it never happens, and to produce any result if it does. That is why the build flags include `-fwrapv`, which tells clang to define it as wrap-around. Without that flag, a rule such as `(x + 1) > x` could be turned into `true` by clang, and the program would no longer match the interpreter.

The point to take from this step is that every operation needs a total definition: an answer for every input, with no cases left open. Chapter 3 builds that contract for each op, including cases such as dividing by zero, where this compiler defines `x // 0` as `0`.

Go deeper

Chapter 2 is the bits. Chapter 3 is the contract. Chapter 12 shows the C traps in detail: this one, and the places where the renderer writes a helper function because C does not match the contract.

Try it

1. Subtract 1 from the smallest `i32`, -2,147,483,648, in the interpreter. Answer: `2147483647`. It wraps the other way.

Step 9

Index math and ranges

In a program that reads arrays, most integer nodes compute positions: row times width plus column. These expressions are often simplifiable, and they are also where a wrong rule does the most damage, because a wrong index reads the wrong memory.

step 9: ranges of integer expressions and a rule that needs them

```
import _here
from uopc import I32, Ops, ParamArg, UOp
from uopc.symbolic import simplify

i = UOp(Ops.PARAM, (), ParamArg(0, I32, 0, 15))    # i is promised to be in [0, 15]
j = UOp(Ops.PARAM, (), ParamArg(1, I32, 0, 3))     # j is in [0, 3]
e = (i * 4 + j) // 4
print("range of the whole expression:", e.vmin, e.vmax)
print("simplify gives i:", simplify(e) is i)
```

```
big = UOp(Ops.PARAM, (), ParamArg(2, I32, 0, 2 ** 30))
risky = (big * 4) // 4
print("big*4 may wrap, safe =", (big * 4).safe, "; simplify gives big:",
      simplify(risky) is big)
```

output of short/s09_index.py

```
range of the whole expression: 0 15
simplify gives i: True
big*4 may wrap, safe = False ; simplify gives big: False
```

An input can carry a declared range. Here `i` is declared to be between 0 and 15 and `j` between 0 and 3. Every node computes its own range from the ranges of its inputs. `vmin` and `vmax` are bounds on a node's value that the compiler can prove. They can be wider than the values that really occur. Nothing checks at run time that an input stays inside its declared range, so a caller that breaks the declaration can get wrong results from rewrites that relied on it. `ParamArg(slot, dtype, low, high)` gives an input its slot, its type and its declared range. `//` is floor division: it rounds the quotient toward minus infinity (chapter 3). The expression `(i*4 + j) // 4` has range 0 to 15: the smallest is $(0*4+0)//4 = 0$ and the largest is $(15*4+3)//4 = 15$.

The rule that simplifies `(i*4 + j) // 4` to `i` needs two facts. First, `j` is between 0 and 3, so adding it never reaches the next multiple of 4. Second, none of the arithmetic wraps. The range tells the compiler both, so the rewrite is applied, and the output says `simplify gives i: True`.

The second half is the case where the rule must not be applied. `big` goes up to 2^{30} . `big * 4` can reach 2^{32} , which does not fit in an `i32`, so it wraps. The node's `safe` flag, which is true when none of the arithmetic inside the node can wrap, is `False`, and `simplify` leaves `(big*4)//4` alone. This is correct: for `big =  $2^{30}$` , `big*4` wraps to 0, so `(big*4)//4` is 0 and not `big`.

This is how the compiler can simplify index math aggressively without being wrong. The integer rules that are only true when nothing wraps are guarded by a range check like this one.

Go deeper

Chapter 6 defines ranges and how each op computes them. Chapter 11 is the integer rules, including the guarded rule used here.

Try it

1. Change the bound of `j` to `0..4`. What are the range of the expression and the answer of `simplify(e) is i`? Answer: the range is 0 to 16, and the answer is `False`. With `j = 4` the expression equals `i + 1`.

Step 10

Memory: loads and stores

Until now a program was a function from a few numbers to one number. Real programs read arrays and write arrays. This needs four more operations, and two more kinds of node, `buffer` and `sink`, which appear below.

- `index` computes the address of one element of a buffer.
- `load` reads the element at an address.
- `store` writes a value to an address.
- `after` says that one thing must happen before another.

This program adds two arrays of 3 elements into a third, `out[i] = a[i] + b[i]`. A `Builder` helper writes the nodes.

step 10: `out[i] = a[i] + b[i]`, rendered and run

```
import _here
from uopc import F32, Ops, ParamArg, Ptr, UOp
from uopc.builder import Builder, make_call
from uopc.interp import run_call
from uopc.render import render
from uopc.runtime import Buffer, Program
from uopc.textfmt import dumps

b = Builder()
a, bb, out = b.buffer(0, F32, 3), b.buffer(1, F32, 3), b.buffer(2, F32, 3)
for i in range(3):
    out.store(i, a.load(i) + bb.load(i))    # out[i] = a[i] + b[i]
args = [UOp(Ops.BUFFER, (), ParamArg(s, Ptr(F32, 3))) for s in (0, 1, 2)]
add3 = make_call(b.body(), args, "add3")

text = dumps(add3).splitlines()
print("\n".join(text[:15]))
print("...", len(text) - 15, "more lines")
src = render(add3)
```

```

print(src[src.index("void add3"):])
A, B, 0 = Buffer(F32, 3).copyin([1, 2, 3]), Buffer(F32, 3).copyin([10, 20, 30]),
Buffer(F32, 3)
Program(add3)(A, B, 0)
print("clang:      ", 0.tolist())
check = [0.0] * 3
run_call(add3, [[1.0, 2.0, 3.0], [10.0, 20.0, 30.0], check])
print("interpreter:", check)

```

output of short/s10_buffers.py

```

; uop v1
%0 = param : dtype=f32 slot=2 size=3
%1 = const : dtype=i32 value=0
%2 = index %0, %1
%3 = param : dtype=f32 slot=0 size=3
%4 = index %3, %1
%5 = load %4
%6 = param : dtype=f32 slot=1 size=3
%7 = index %6, %1
%8 = load %7
%9 = add %5, %8
%10 = store %2, %9
%11 = after %0, %10
%12 = const : dtype=i32 value=1
%13 = index %11, %12
... 20 more lines
void add3(float *p0, float *p1, float *p2) {
    float *v0 = p2 + 0;
    float *v1 = p0 + 0;
    float v2 = *v1;
    float *v3 = p1 + 0;
    float v4 = *v3;
    float v5 = v2 + v4;
    *v0 = v5;
    float *v6 = p2 + 1;
    float *v7 = p0 + 1;
    float v8 = *v7;
    float *v9 = p1 + 1;
    float v10 = *v9;
    float v11 = v8 + v10;
    *v6 = v11;
    float *v12 = p2 + 2;
    float *v13 = p0 + 2;
    float v14 = *v13;
    float *v15 = p1 + 2;
    float v16 = *v15;
    float v17 = v14 + v16;
    *v12 = v17;
}

```

```
clang:      [11.0, 22.0, 33.0]
interpreter: [11.0, 22.0, 33.0]
```

The whole program is wrapped in a `call`, as in step 1. The arrays are `buffer` nodes that are passed as arguments of the call. Inside the body, the `param ... size=3` lines are the body's names for those arrays. A `sink` node near the end is the root of the body. It holds the last store to each buffer, and the earlier stores are reached through the `after` nodes. The caller must pass three different buffers. Nothing checks this (chapter 14). The text shows the first 15 of 35 lines. `%2 = index %0`, `%1` is the address of element 0 of the output buffer. `%5 = load %4` reads input `a`. `%9 = add %5`, `%8` adds the two loaded values. `%10 = store %2`, `%9` writes the sum. `%11 = after %0`, `%10` is the output buffer, but only once the store has happened. The next element's address is made from that `after`, which is how element 1 is forced to come after element 0.

The C is a straight list of statements, one per node, except that `param`, `const`, `after` and `sink` write no statement of their own. It reads from `p0` and `p1` and writes to `p2`.

The last two lines show clang and the interpreter agree: `[11.0, 22.0, 33.0]`.

A real compiler would write this as a loop over the elements. This module stops before loops and writes the three copies out one after the other. The memory ideas are all here, though.

Go deeper

Chapter 14 is memory in full: what a buffer is, what each memory op does, and how the `Builder` adds the ordering nodes.

Try it

1. Change `+` to `*` in the loop body. What does the program print? Answer: `[10.0, 40.0, 90.0]`.

Step 11

Order matters

A graph has no order of execution, only dependencies. If two nodes do not depend on each other, the compiler may run them in either order. When both touch the same memory, that freedom changes the answer.

This program has a store `buf[0] = 5` and a copy `buf[1] = buf[0]`. Nothing in the graph says which comes first. The program is run twice, with two different orders. A `sink` groups the two stores into one program. The `"id"` order visits nodes in the order they were created. The `"dfs"` order visits them in the order the renderer writes the C, which is depth first from the root. Both put every node after its inputs, so both are valid. At the end the same program is built again with the `Builder`, which adds the missing ordering.

step 11: two valid orders of the same program, then the fixed program

```
import _here
from uopc import F32, Ops, ParamArg, Ptr, UOp
from uopc.builder import make_call
from uopc.interp import run_call

buf = UOp(Ops.PARAM, (), ParamArg(0, Ptr(F32, 3)))
first = UOp(Ops.INDEX, (buf, UOp.const(0)))
write = UOp(Ops.STORE, (first, UOp.const(5.0)))           # buf[0] = 5
read = UOp(Ops.LOAD, (first,))                          # read buf[0]
copy = UOp(Ops.STORE, (UOp(Ops.INDEX, (buf, UOp.const(1))), read))
# buf[1] = what we read
call = make_call(UOp.sink(copy, write), [UOp(Ops.BUFFER, (), ParamArg(0, Ptr(F32, 3)
)], "racy")
for order in ("id", "dfs"):                             # two orders that both respect "inputs
first"
    data = [1.0, 0.0, 0.0]
    run_call(call, [data], order)
    print("order", order, "->", data)

# the same program built with Builder, which adds the ordering for us
from uopc import Ops
from uopc.builder import Builder
b = Builder()
buf2 = b.buffer(0, F32, 3)
buf2.store(0, 5.0)
buf2.store(1, buf2.load(0))                             # buf[0] = 5, then buf[1] = buf[0]
fixed = make_call(b.body(), [UOp(Ops.BUFFER, (), ParamArg(0, Ptr(F32, 3)))], "ordered")
for order in ("id", "dfs"):
    data = [1.0, 0.0, 0.0]
```

```
run_call(fixed, [data], order)
print("with AFTER, order", order, "->", data)
```

output of short/s11_order.py

```
order id -> [5.0, 5.0, 0.0]
order dfs -> [5.0, 1.0, 0.0]
with AFTER, order id -> [5.0, 5.0, 0.0]
with AFTER, order dfs -> [5.0, 5.0, 0.0]
```

With no ordering the answers differ: `[5.0, 5.0, 0.0]` in one order and `[5.0, 1.0, 0.0]` in the other. In the second order the copy ran first and read the old value 1.0.

Neither answer is a bug in the interpreter. The program did not say what it meant. In a compiler, a change to the program that is legal for the graph but picks a different order would silently change the result.

The fix is the `after` node from step 10. The last two lines show the same program built with the `Builder`, which makes the read of `buf[0]` depend on the store to `buf[0]`. Both orders now give `[5.0, 5.0, 0.0]`.

The `Builder` follows three rules:

- A read waits for the last write to the same buffer.
- A write waits for the reads made since the last write.
- A write waits for the last write.

All three rules are per buffer. The `Builder` does not look at which element is touched.

Go deeper

Chapter 14 explains the three cases, read after write, write after read and write after write, and shows the interpreter run in both orders on a correctly sequenced program.

Try it

1. Change `5.0` to `7.0` in the unordered program. What do the two orders give?
Answer: `[7.0, 7.0, 0.0]` and `[7.0, 1.0, 0.0]`.

Step 12

Test it against itself

How do you find out that a rewrite rule is wrong without already knowing which one? You make many random programs, run each before and after simplification, and compare.

step 12: random programs, simplified and not, compared on random inputs

```
import random
import _here
from common import gen_program, rand_value
from uopc.interp import evaluate
from uopc.semantics import same
from uopc.symbolic import simplify
from uopc.textfmt import dumps

def first_difference(fast_math, count=2000):
    for seed in range(count):
        r = random.Random(seed)
        root, params = gen_program(r)
        small = simplify(root, fast_math=fast_math)
        for _ in range(20):
            env = {p.arg.slot: rand_value(r, p.dtype) for p in params}
            if not same(evaluate(root, env), evaluate(small, env)):
                return seed, env
    return None

print("strict rules: ", first_difference(False) or "2000 programs, no difference")
found = first_difference(True)
print("fast math rules: first difference at seed", found[0], "with inputs", found[1])
)
root, params = gen_program(random.Random(found[0]))
print("the program:")
print(dumps(root))
print("fast math simplifies it to:")
print(dumps(simplify(root, fast_math=True)))
print("program gives: ", evaluate(root, found[1]))
print("simplified gives:", evaluate(simplify(root, fast_math=True), found[1]))
```

`gen_program` makes a random program. Every random program has three inputs: an int in slot 0, a float in slot 1 and a bool in slot 2. It may use any of them or none. `rand_value` makes the input values, and these include the dangerous values: zero, `-0.0`, `nan`, infinities, and the largest and smallest integers. Each program is run on 20 inputs. `same` compares floats by their bits, so `0.0` does not equal `-0.0`, and every `nan` counts as equal to every other `nan`.

output of short/s12_test_it.py

```
strict rules:    2000 programs, no difference
fast math rules: first difference at seed 217 with inputs {0: 7, 1: nan, 2: False}
the program:
; uop v1
%0 = const : dtype=f32 value=-0.0
%1 = param : dtype=f32 slot=1
%2 = mul %0, %1
%3 = cast %2 : dtype=bool

fast math simplifies it to:
; uop v1
%0 = const : dtype=bool value=false

program gives:    True
simplified gives: False
```

With the strict rules, 2000 programs show no difference.

With the fast-math rules, the very first difference is at seed 217, the 218th program. It is `bool(-0.0 * y)`. The `cast` node in the text converts the float to a bool: zero gives `False`, and any other value, `nan` included, gives `True`. The fast-math list has a rule that rewrites `x * 0` to `0`. It assumes `y` is never `nan` and never infinite. When `y` is `nan`, `-0.0 * nan` is `nan`, and converting `nan` to a bool gives `True`. The rewritten program is the constant `False`. The program and the rewrite give different answers on the input `y = nan`. Slots 0 and 2 are not used by this program.

This is the same story as step 7: a rule with a hidden assumption. The test found it without being told where to look.

Random testing cannot prove a rule right. 2000 programs with no difference is evidence, not a guarantee. The book's full test suite runs more than two million checks, and it was itself tested by planting 19 bugs in the compiler and checking that each was caught. Chapter 15 has the results. Step 13 shows how to get a guarantee.

Go deeper

Chapter 15 is the whole testing method: three independent references, four kinds of input, the planted bugs and which test caught which.

Try it

1. Name one input value for `y` that breaks the rule `y * 0 = 0` for floats, other than `nan`. Answer: `infinity`. `inf * 0.0` is `nan`.

Step 13

Prove it

A theorem prover can settle a claim about every input at once. The tool here is `z3`. It is given a claim written as two expressions, and it searches for an input where the two sides differ. If it proves that no such input exists, the claim is true for all inputs. If it finds one, it prints it.

step 13: five claims about rewrite rules, proved or refuted by `z3`

```
import z3

def check(name, left, right):
    s = z3.Solver()
    s.add(left != right)
    # look for an input where the two sides differ
    if s.check() == z3.unsat:
        print(f"{name:22s} true for all 2^32 inputs")
    else:
        print(f"{name:22s} false, for example {s.model()}")

x = z3.BitVec("x", 32)
check("x * 1 == x", x * 1, x)
check("x + x == 2 * x", x + x, 2 * x)
check("(x / 2) * 2 == x", (x / 2) * 2, x)    # / is c style division here

f = z3.FP("f", z3.Float32())
def bits(v):
    return z3.fpToIEEBV(v)
plus0 = z3.fpAdd(z3.RNE(), f, z3.FPVal(0.0, z3.Float32()))
minus0 = z3.fpAdd(z3.RNE(), f, z3.FPVal(-0.0, z3.Float32()))
for name, e in [("f + 0.0 == f", plus0), ("f + (-0.0) == f", minus0)]:
    s = z3.Solver()
    s.add(z3.Not(z3.fpIsNaN(f)), bits(e) != bits(f))
    print(f"{name:22s}", "true for every float32 except nan" if s.check() ==
z3.unsat else f"false, for f = {s.model()[f]}")
```

output of short/s13_prove.py

```
x * 1 == x           true for all 2^32 inputs
x + x == 2 * x       true for all 2^32 inputs
(x / 2) * 2 == x     false, for example [x = 3889799679]
f + 0.0 == f         false, for f = -0.0
f + (-0.0) == f      true for every float32 except nan
```

The first three claims are about 32 bit integers. `x * 1 == x` and `x + x == 2 * x` hold for all 2^{32} values. z3 does not try them one by one. It treats the 32 bits of `x` as unknowns and works out what must be true of them. The answer `unsat` in the code means that no input makes the two sides differ. The third claim, `(x / 2) * 2 == x`, is false, and z3 gives one counterexample, `x = 3889799679`, which z3 prints as an unsigned number. The solver may choose another value on another run. Any odd number works, because the division drops the 1. In this call, z3's `/` on 32 bit values divides as signed numbers and rounds toward zero, which differs from the `//` of step 9 for negative numbers. The claim is false with either.

The last two are about float32. The claim `f + 0.0 == f` is false, and z3 finds the same value as step 7: `f = -0.0`. The claim `f + (-0.0) == f` holds for every float32 that is not `nan`. `nan` is left out because the claim is about numbers. The comparison is on bit patterns, so `-0.0` and `+0.0` count as different.

The book checks the rules this way. 59 claims were put to z3: 51 hold and 8 are refuted. Each refutation tells you a condition the rule needs, and chapter 15 lists them.

What a proof does and does not cover: it proves the claim about the arithmetic as z3 models it, that is, 32 bit integers and IEEE 754 float32 in z3's own definition of them, with rounding to nearest, ties to even. (8 of the 59 integer claims in the book are over mathematical integers with a guard against wrap-around, which chapter 11 explains.) it does not prove that the Python code of the rule matches the claim. That is why both are used. The proofs show the rules are right. The tests show the code does what the rules say.

Go deeper

Chapter 15 explains them. Appendix C lists the claims and has the exact commands.

Try it

1. Prove `x - x == 0` and `x * 2 == x + x` for 32 bit integers, with the same `check` function. Answer: both hold.

2. Try `check("x + 1 > x", x + 1 > x, z3.BoolVal(True))`. Is it true? Answer: false. For `x = 2147483647` the sum wraps to the smallest number. This is the same wrap as step 8.

Step 14

What the real one writes

You have now seen the whole path: a graph, the text format, an interpreter, C, clang, sharing, simplification, wrong rules, wrap-around, index ranges, memory, order, testing and proof.

The real tinygrad does the same path on much bigger graphs. This is the C it writes for `relu(x*y + 1)` on four values, with its default optimisations. `relu(v)` means `max(v, 0)`, a function used in machine learning. The command is `DEV=CPU DEBUG=4`, which prints the source it compiles.

the c that tinygrad writes for `relu(x*y+1)` on four values, default optimisations · output of `demos/tinygrad_demo.py`

```
typedef float float4 __attribute__((aligned(16),ext_vector_type(4)));
void E_4(float* restrict data0_4, float* restrict data1_4, float* restrict data2_4)
{
    float4 val0 = (*((float4*)((data1_4+0))));
    float4 val1 = (*((float4*)((data2_4+0))));
    float alu0 = ((val0[0]*val1[0])+1.0f);
    float alu1 = ((val0[1]*val1[1])+1.0f);
    float alu2 = ((val0[2]*val1[2])+1.0f);
    float alu3 = ((val0[3]*val1[3])+1.0f);
    float alu4 = ((alu0<0.0f)?0.0f:alu0);
    float alu5 = ((alu1<0.0f)?0.0f:alu1);
    float alu6 = ((alu2<0.0f)?0.0f:alu2);
    float alu7 = ((alu3<0.0f)?0.0f:alu3);
    *((float4*)((data0_4+0))) = (float4){alu4,alu5,alu6,alu7};
}
result: [11.0, 41.0, 91.0, 161.0]
```

Compare it with step 3.

- It is still straight-line code with named values. The names are `val0` and `alu0` where ours are `v0`. One statement can hold several operations: `alu0` is a multiply and an add in one expression, where ours would use one statement for each.

- The pointers carry `restrict`, which promises the buffers do not overlap. This book's renderer leaves it out, and chapter 12 says why. The book's compiler also needs the caller to pass buffers that do not overlap, and does not check it (chapter 14).
- The four values of each input are read with one 16 byte vector load, which reads four floats together, and the four results are written by one vector store. The module does not do this.
- The `max(., 0)` is written `(alu0<0.0f)?0.0f:alu0`. Ours writes the larger of two values as `a>b?a:b`. For a `nan` input they differ, and for `-0.0` too. Try the exercise below.

What is missing from this module is everything above the graph: tensors with shapes, the choice of how to split loops, and the choice of which operations to run together. Those are the later modules of the course. What this module gives you is what those modules build on: a graph, an exact meaning for every op, rules you can trust, and a way to compile and test them.

Where to go next

- Read chapter 1 to see the same ideas with the reasons.
- Read chapters 2 and 3 first if the numbers in steps 2, 7 and 8 surprised you.
- Read chapters 10 and 11 if you want to write rewrite rules.
- Read chapter 15 if you want to see how the compiler is tested.
- Read chapter 16 to compare with `tinygrad`'s source.

Try it

1. `relu` of `nan`: what does our `max(v, 0)` return, and what does `tinygrad`'s line return? Answer: ours returns `0.0` and `tinygrad`'s returns `nan`. This is one of the choices in the book that you may want to reverse. Chapter 16 discusses it.

The deep book, part 1
The problem

Chapter 1

What we are building

A compiler is a program that reads a description of a computation and writes a program that does the same computation faster or on different hardware. That is all the word means. The interesting part is the word “same”. A compiler is only correct if the program it writes gives exactly the answers the description asks for, on every input. Most of this book is about how to make that sentence true and how to check that it is.

This module builds the smallest compiler that is still a real one. It has an input format, a way to simplify programs, a way to write C, a way to run that C, and a way to handle memory. Later modules add loops and tensors. None of them throw this module away.

The whole path, once

Here is one tiny program followed all the way through. The program takes two floats `x` and `y` and computes `max(x*y + 1, 0)`. In machine learning this shape of computation is common: a multiply, an add, and a clamp at zero.

You write it in Python using the compiler’s own types:

```
x, y = UOp(Ops.PARAM, (), ParamArg(0, F32)), UOp(Ops.PARAM, (), ParamArg(1, F32))
body = (x * y + 1.0).max(UOp.const(0.0))
```

Nothing is computed here. Each operator call builds one node of a graph and returns it. When the graph is written in the compiler’s text format it looks like this. This is real output from `demos/walkthrough.py`:

the relu program in the text format · printed by `demos/walkthrough.py`

```
; uop v1
%0 = param : dtype=f32 slot=0
%1 = param : dtype=f32 slot=1
%2 = mul %0, %1
%3 = const : dtype=f32 value=1.0
%4 = add %2, %3
%5 = const : dtype=f32 value=0.0
%6 = max %4, %5
%7 = call %6, %5, %5 : name=relu
```

Each line is one node. `%2 = mul %0, %1` says: node 2 is a multiplication whose inputs are nodes 0 and 1. There are no variables and no order of execution. The graph says what depends on what, and nothing else.

The compiler then writes C, one statement per node:

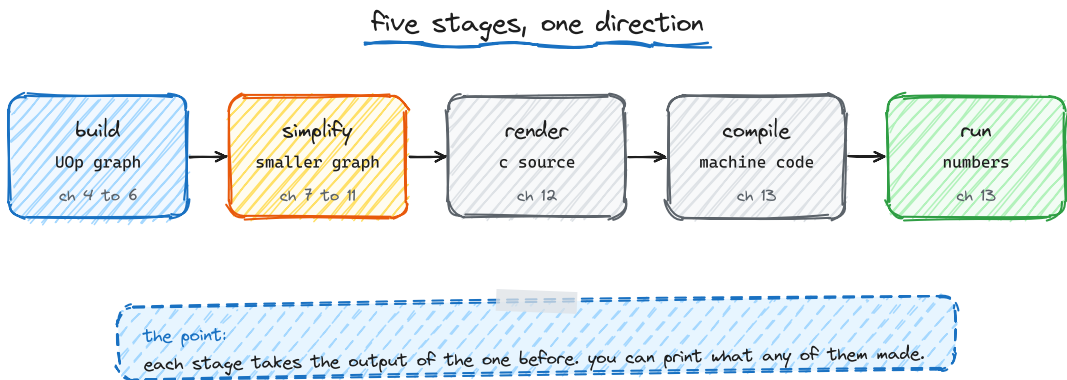
the c the compiler wrote for it · printed by demos/walkthrough.py

```
float relu(float p0, float p1) {
    float v0 = p0 * p1;
    float v1 = v0 + 1.0f;
    float v2 = (v1 > 0.0f) ? v1 : 0.0f;
    return v2;
}
```

clang turns that C into machine code. The runtime loads the machine code and calls it.

`relu(2, 3)` returns 7.0 and `relu(-2, 3)` returns 0.0. You can see both in `out/demos.txt`.

That is the entire pipeline.



Five stages, each a few hundred lines. None of them is hard alone. The work is in making each stage exact, so that when you chain them nothing slips.

Why a graph

A program could be kept as text, or as a tree like the ones a parser makes. A compiler for machine learning keeps it as a graph, and the reasons are mechanical.

In a tree, a value used twice appears twice. The expression `(p+q)*(p+q) + (p+q)` has 11 nodes as a tree and 5 as a graph, because the three copies of `p+q` are the same node. On large programs this is the difference between a compile that finishes and one that runs out of memory. Chapter 5 measures it: a chain of 18 doublings is 19 nodes as a graph and 524,287 as a tree.

In a graph, “is this the same value as that one” is a pointer compare. The compiler builds nodes in a way that guarantees two nodes with the same contents are one object. Chapter 5 explains how. This single decision removes a whole family of bugs and a whole pass, the one that finds repeated work.

A graph also has no hidden order. The compiler chooses the order when it writes C. That freedom is what lets later modules reorder and fuse loops.

Why rewriting

Simplifying a program means replacing part of it with something equal that is cheaper. `x*1` becomes `x`. `(i*4+j)//4` becomes `i` when `j` is always between 0 and 3. Each such change is a rule. A rule has a pattern to look for and a replacement.

The compiler in this module does all its optimising with rules and one driver that applies them until none applies. There is no separate pass for constant folding, none for algebra, none for index math. It is one mechanism. This is also how `tinygrad` works, and it is why the course says the work “aggressively builds on the previous week”: every later feature is more rules.

The price of this design is that every rule must be right. A wrong rule silently changes answers on some inputs. Much of this book is about the ways rules go wrong, in particular with floating point numbers and with integer overflow, and about how to be certain they do not.

Why C

The compiler writes C source and asks `clang` to turn it into machine code. This avoids writing a register allocator, an instruction selector and an object file writer, which are large and well understood and not what this course is about. `tinygrad` does the same for CPUs: it has a renderer that writes C, in `tinygrad/renderer/cstyle.py`, and a runtime that compiles it.

C is a hard target in one way. C has rules about what programs mean that are not the rules you might expect. Adding two ints that overflow is not defined. Dividing the smallest int by -1 crashes. Converting a huge float to an int is not defined. A compiler that writes C has to write C that avoids all of these, or has to tell `clang` to treat them in a specific way. Chapter 12 shows each trap with a real demonstration.

What is in this module and what is not

The syllabus lists two weeks of material. Here is where each item is covered.

Syllabus item	Where in this book
The UOp and its graph structure	Chapter 4
Hash consed	Chapter 5
Recursive properties: dtype, shape, addrspc	Chapter 6
Ops: PARAM, CALL, CONST, casts and math	Chapters 3 and 4
Ops: ADD, MUL, CMPLT, CMPNE, MAX and bool meanings	Chapter 3
FLOORDIV, FLOORMOD, WHERE	Chapter 3
The rewrite engine and running programs by const folding	Chapters 7, 8, 9
Symbolic simplification	Chapters 10 and 11
Renderer	Chapter 12
Runtime	Chapter 13
BUFFER, ALLOC, INDEX, LOAD, STORE, AFTER	Chapter 14
The interchange text format	Chapter 5

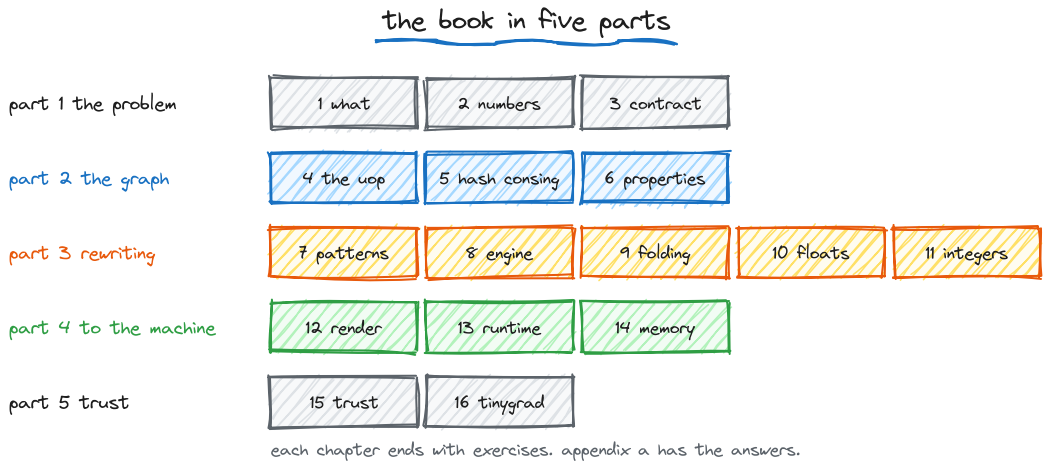
This implementation makes some choices the syllabus leaves open, and you should know them before you start.

- **Language.** Python. The course lets the student choose. Python is slow, which chapter 13 measures, and it is easy to read, which is the point of a teaching compiler.
- **Scalars only.** Every value computed by an arithmetic op is a single number. The `shape` property exists and is checked, but the only shapes that occur are the empty shape for a number and a one dimensional size for a buffer. Arrays of values arrive with the loops and movement ops of week 3. Chapter 6 says exactly what the shape property does here.
- **One extra op.** `SINK` collects several stores into one root so a function can write more than one buffer. It is not in the syllabus. Chapter 14 explains why it is needed.
- **Total arithmetic.** Every op gives an answer for every input. `x//0` is 0 and `x%0` is `x`. Integers wrap. Converting a float to an int saturates. This is a choice, and chapter 3 gives the reasons. `tinygrad` treats a division by a constant zero as an error instead.
- **Strict floating point.** The default rule set never changes a floating point result by even one bit. A second rule set, kept apart, is allowed to. Chapter 10 covers this.

How the rest of the book goes

Each chapter introduces one idea, shows the code, then measures or proves something about it. You will see three kinds of evidence over and over: a program run with its real output, a test that compares the compiler against an independent reference, and a proof from a theorem prover. When none of those is possible the book says so at that spot.

The exercises at the end of each chapter are the shortest path to understanding. Do them.



Where to learn more

- The course syllabus that this module follows is in the notes by george hotz (geohot) for compilers for machine learning. The module covers its weeks 1 and 2.
- tinygrad, at github.com/tinygrad/tinygrad. In this book's reading of it, `tinygrad/uop/ops.py` holds the `UOp`, `UPat`, `PatternMatcher` and `graph_rewrite`; `tinygrad/uop/symbolic.py` holds most simplification rules; `tinygrad/renderer/cstyle.py` writes C. The files named in this book were checked against the repository at commit `06e57e3` on the day of writing. The layout changes often, so look for the names, not the line numbers.
- *Crafting interpreters* by robert nystrom, free at craftinginterpreters.com. It builds two complete interpreters and is the clearest introduction to the ideas of a language implementation. Read its first chapters if “parse”, “tree” or “bytecode” are new words.
- Cornell's CS 6120 self guided course, at cs.cornell.edu/courses/cs6120/2020fa/self-guided. It covers intermediate representations and optimisation with video lessons, in a different style from this module.

Exercises

1. In the expression $(p+q)*(p+q) + (p+q)$, name the nodes of the graph and count them. Compare with the tree.
2. Run `python3 demos/run_demos.py` and find the line for demo 8. What is the C function's name and how many statements does its body have?
3. The `relu` program computes `max(x*y + 1, 0)`. What does it return when `x*y + 1` is `nan`? Read chapter 3 first, then come back and answer.
4. Write a program in the text format that computes `x + x` for an `i32 x`. How many lines does it need, without counting the `call` line?

Chapter 2

The numbers a machine really has

A compiler changes programs. To change a program without changing its answers, you must know exactly what its answers are. In Python the answer to `2**100` is a huge integer and the answer to `0.1 + 0.2` is a decimal-looking number. On a machine, numbers are finite strings of bits, and every operation has a precise rule for what happens when the true result does not fit. A compiler that does not know those rules will be wrong on some inputs and right on all the ones you tried.

This chapter lists the rules for the three kinds of value in this module: `bool`, `int32` and `float32`. Every one is demonstrated by a program in `demons/number_facts.py`, and chapter 3 turns the rules into the definition of each op.

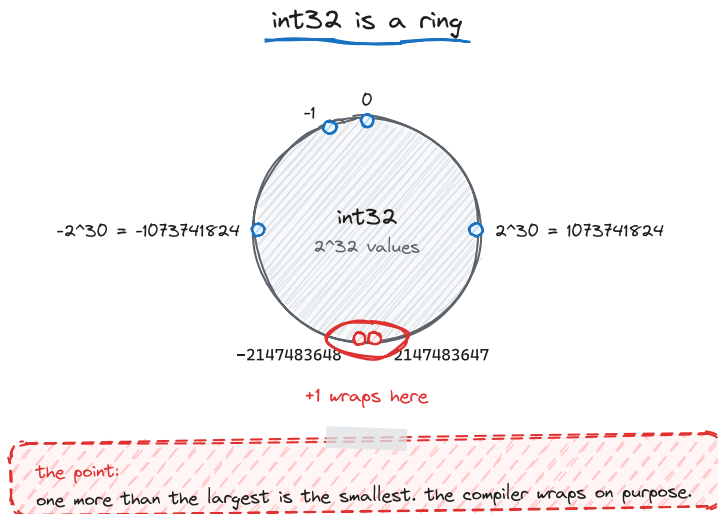
Bits, and integers read from bits

A bit is a 0 or a 1. A string of 32 bits has $2^{32} = 4,294,967,296$ possible values. What number a string means depends on how you read it.

Read as an unsigned number, the bits `b31 ... b0` mean the sum of `b_i * 2^i`. The smallest is 0 and the largest is 4,294,967,295.

Read as a signed number in two's complement, the same bit string means the unsigned value, minus 2^{32} if the top bit is 1. So the top bit has weight -2^{31} instead of $+2^{31}$. The smallest value is -2,147,483,648 and the largest is 2,147,483,647. This is `int32`. There is one more negative number than positive, and this fact causes real bugs.

Two's complement is used because addition needs no special case. The hardware adds two 32 bit strings the same way whether they are signed or not. Only the reading differs.



Adding 1 to the largest `int32` gives a string of 32 bits that reads as the smallest. The values form a cycle: after the largest comes the smallest. The compiler's contract says `int32` addition, multiplication and negation work this way: compute the true result, then reduce it modulo 2^{32} into the range. That reduction is the function `wrap32`:

`wrap32` and the integer helpers in `uopc/semantics.py` · every function here is tested against `numpy` and `c`

```
def wrap32(x: int) -> int:
    """reduce any python int to the int32 it is congruent to (mod 2**32)."""
    return ((x + 2 ** 31) & 0xFFFFFFFF) - 2 ** 31

def f32(x: float) -> float:
    """round a python float (a double) to the nearest float32, ties to even."""
    try:
        return _F32.unpack(_F32.pack(x))[0]
    except OverflowError:
        # too big for float32: rounds to
        infinity
        return math.copysign(math.inf, x)

def floordiv(a: int, b: int) -> int:
    if b == 0:
        return 0
    return wrap32(a // b)
    # python's // already rounds toward minus
    infinity

def floormod(a: int, b: int) -> int:
    if b == 0:
        return a
```

```

return a % b                                     # python's % takes the sign of the divisor

def cast(v, dtype):
    if dtype is BOOL:
        return v != 0
    if dtype is I32:
        if type(v) is float:
            if v != v:
                return 0
            if v >= 2147483648.0:
                return INT_MAX
            if v <= -2147483648.0:
                return INT_MIN
            return math.trunc(v)
        return int(v)
    if dtype is F32:
        return f32(float(v))
    raise ValueError(dtype)

```

The first lines of the output of `demons/number_facts.py` show the wrap:

int32 values wrapping · output of `demons/number_facts.py`

```

2147483647 -> wrap32 gives 2147483647 bits
01111111111111111111111111111111
2147483648 -> wrap32 gives -2147483648 bits
10000000000000000000000000000000
-2147483649 -> wrap32 gives 2147483647 bits
01111111111111111111111111111111
4294967301 -> wrap32 gives 5 bits
0000000000000000000000000000000101
2147488281 -> wrap32 gives -2147479015 bits
1000000000000000000000001001000011001
numpy int32 agrees: True

```

2147483648 is 2^{31} , one past the top, and `wrap32` gives -2147483648. -2147483649 wraps to 2147483647. 4294967301 is $2^{32} + 5$ and wraps to 5. $46341 * 46341$ is 2,147,488,281, which is just above 2^{31} and wraps to a large negative number. The program checks the same wrap in NumPy's `int32` and gets the same answer.

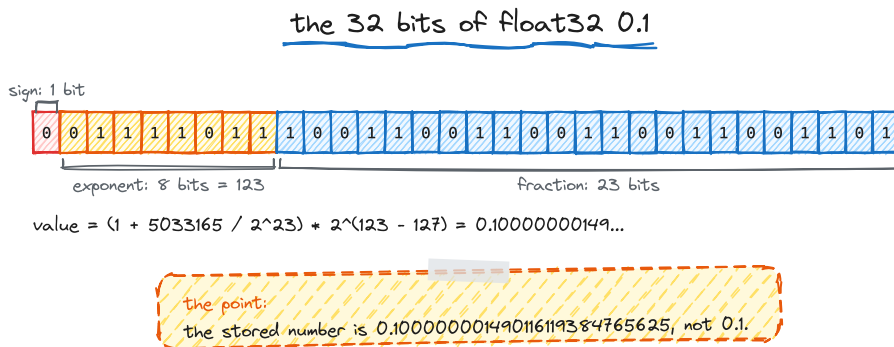
The C trap

C does not say that signed overflow wraps. The C standard says it is undefined: the program has no meaning, and the compiler may assume it never happens. This is not a small technicality. Chapter 12 shows clang compiling `x + 1 > x` to 1 at -O2, because if overflow never happens then `x + 1` is always greater than `x`. So the same source file gives

0 or 1 for the input `INT_MAX` depending on the optimisation level. The compiler in this book avoids this with a flag, `-fwrapv`, which tells clang that signed overflow wraps. Without that flag the contract above would be false for the C that we write.

Floating point

An `int32` has one reading. A `float32` has a more complicated one, defined by the IEEE 754 standard. A float32 is 32 bits split into three fields:



- 1 sign bit `s`.
- 8 exponent bits `e`, read as an unsigned number from 0 to 255.
- 23 fraction bits `f`, read as the binary fraction `f / 223`.

If `e` is between 1 and 254, the value is $(-1)^s * (1 + f/2^{23}) * 2^{(e - 127)}$. The `1 +` is not stored. It is implied. For `e = 0`, the value is $(-1)^s * (f/2^{23}) * 2^{-126}$, which are the subnormal numbers, and they include the two zeros. For `e = 255`, the value is infinity if `f = 0` and “not a number” (NaN) if `f` is not 0.

Here is the bit layout of nine values, printed by the program:

sign, exponent, fraction of nine values · output of `demos/number_facts.py`

```

1.0  0 01111111 000000000000000000000000
0.15625 0 01111100 010000000000000000000000
-2.5  1 10000000 010000000000000000000000
0.1   0 01111011 10011001100110011001101
0.0   0 00000000 000000000000000000000000
-0.0  1 00000000 000000000000000000000000
inf   0 11111111 000000000000000000000000
nan   0 11111111 100000000000000000000000
1e-45 0 00000000 000000000000000000000001
0.1 as float32 is exactly 0.10000000149011612  0.1 as a python float is exactly

```

```
0.100000000000000005551115123126
exact decimal value of float32 0.1: 0.100000001490116119384765625000
```

Check `0.15625` by hand. The exponent bits `01111100` are 124, so the power is $2^{(124 - 127)} = 2^{-3} = 0.125$. The fraction bits `0100...` are 0.25, so the value is $1.25 * 0.125 = 0.15625$. Correct.

Now `0.1`. Its exponent is `01111011` = 123, so 2^{-4} , and its fraction is `10011001100110011001101`. A repeating pattern, rounded at the end. The stored number is not 0.1. Its exact decimal value is `0.100000001490116119384765625`, which the program prints, next to the exact value of Python's own `0.1` (a 64 bit double): `0.1000000000000000055511151231`. Neither is 0.1, because 0.1 is not a sum of powers of two.

Rounding

The real result of an operation is usually not one of the 2^{32} representable values. The standard says: pick the nearest representable value, and if two are equally near, pick the one whose last fraction bit is 0. This is “round to nearest, ties to even”, and it is what the hardware does for `+`, `-`, `*`, `/` and `sqrt`.

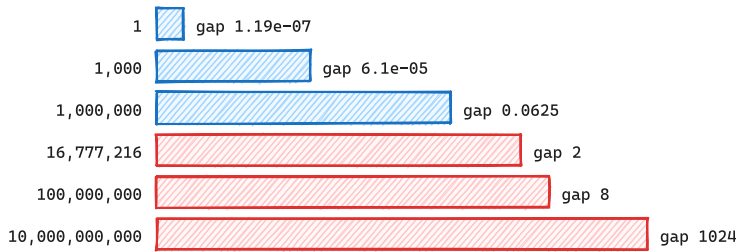
The gap between neighbouring floats grows with the size of the number. Between 1.0 and the next float is 2^{-23} , about $1.19\text{e-}7$. Near $1\text{e}6$ the gap is 0.0625. Near $1\text{e}10$ it is 1024. Past $2^{24} = 16,777,216$ the gap is 2, and you cannot represent every integer any more:

rounding in float32 · output of demos/number_facts.py

```
python 0.1 + 0.2 = 0.30000000000000004    float32: 0.30000001192092896 (printed with
%.9g: 0.300000012)
2**24 + 1 in float32 = 16777216    2**24 + 2 = 16777218
gap between neighbours at 1.0: 1.1920928955078125e-07    at 1e6: 0.0625    at 1e10:
1024.0
(1e8 + 1) - 1e8 in float32 = 0.0 (the 1 is lost, the gap at 1e8 is 8)
(a+b)+c = 1.0    a+(b+c) = 0.0    with a=1e8, b=-1e8, c=1
```

$2^{24} + 1$ rounds back down to 16,777,216, a tie that goes to the even neighbour. $2^{24} + 2$ is representable. $(1e8 + 1) - 1e8$ is 0.0, not 1.0: the gap at $1e8$ is 8, so adding 1 changes nothing.

The last line is the most important fact in this chapter for a compiler. With `a = 1e8`, `b = -1e8`, `c = 1`, the value `(a + b) + c` is `1.0` and `a + (b + c)` is `0.0`. Floating point addition is **not associative**. The two sums differ by the entire answer. Any rule that rewrites one grouping into the other changes results, and chapter 10 is built on that.

the gap to the next float growslog scale. gaps from `numpy.spacing` on `float32`.

the point:

past 2^{24} the gap is 2, so not every integer exists. $1e8 + 1$ is $1e8$.

Why one rounding is enough

The compiler is written in Python, whose floats are 64 bit doubles. To compute a float32 `a + b`, the reference implementation adds the two numbers as doubles, then rounds the double to float32 with `struct.pack`. That rounds twice: once into a double, once into a float32. Double rounding can give a different answer from rounding once. It does not, for `+`, `-`, `*`, `/` and `sqrt`, when the wider format has at least $2p + 2$ bits of precision and the narrower has p . A double has 53 bits and a float32 has 24, and 53 is at least $2 * 24 + 2 = 50$. The result is proved, with a machine checked proof, by pierre roux in *innocuous double rounding of basic arithmetic operations* (journal of formalized reasoning, 2014). This book does not rely on the proof alone: the semantic functions are compared with NumPy's real float32 arithmetic and with C's, with 400,000 checks each, bit for bit. Every check passes.

Special values

Floats have values that integers do not, and each one needs a decision.

the special values · output of `demons/number_facts.py`

```
nan == nan: False   nan != nan: True   nan < 1: False   nan > 1: False
0.0 == -0.0: True   but 1/0.0 and 1/-0.0 are inf and -inf (copysign shows the sign:
-1.0 )
max(nan, 1) in python: nan   max(1, nan): 1   (a > b ? a : b) rule: nan>1 is false
```

```
so gives 1; 1>nan false so gives nan
inf - inf = nan    0 * inf = nan
```

- **NaN** means “not a number”. It is the result of `inf - inf`, `0 * inf` and `log2(-1)`. Every comparison with NaN is false, except `!=`, which is true. So `nan == nan` is false and `nan != nan` is true. That is the one case where a value is not equal to itself.
- **Infinity** has both signs and is the result of overflow (a finite result too large to represent) and of `1 / 0.0`.
- **Negative zero** is a zero with the sign bit set. `0.0 == -0.0` is true, but the bit patterns differ, and `1 / -0.0` is `-inf` where `1 / 0.0` is `+inf`. So the two zeros are different values that compare equal. A rule that treats them as the same can be wrong.

The output also shows what a naive `max` does with NaN. Python’s own `max(nan, 1)` returns `nan` and `max(1, nan)` returns `1`. The order of the arguments changes the answer. The definition used by this compiler is `MAX(a, b) = a > b ? a : b`, which is what C’s ternary operator does, and it also depends on the order of arguments with a NaN. This matters for the rules in chapter 10: swapping the arguments of a float `MAX` is not allowed, because it can change the answer.

Division

Dividing integers has three conventions in use. For `-7` and `2`:

division conventions · output of `demos/number_facts.py`

```
truncate toward zero (C):          -3 remainder -1.0
floor (python //, our FLOORDIV):    -4 remainder 1
our floordiv/floormod on (-7, 2):  -4 1
python 7 // -2 = -4    7 % -2 = -1 (the remainder takes the sign of the divisor)
identity a == b*(a//b) + a%b over a in [-20,20], b in [-5,5], b!=0: True
x//0 and x%0 under our contract: 0 7 INT_MIN // -1: -2147483648
```

C truncates toward zero, so `-7 / 2` is `-3` with remainder `-1`. Python and this compiler **floor**, which means round toward minus infinity, so `-7 // 2` is `-4` with remainder `1`. In both conventions `a == b * (a / b) + a % b`. The program checks that for floor division over 410 pairs.

$x \% 4$ for floor and for c

x	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7	8
floor	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3	0
c	0	-3	-2	-1	0	-3	-2	-1	0	1	2	3	0	1	2	3	0

the point:
floor keeps the remainder in 0 to 3. c can give negatives, so a rule built for one breaks on the other.

Floor division is chosen because it keeps one important fact: $x \% c$ for a positive c is always between 0 and $c - 1$, even for negative x . Index arithmetic, which is most of what the symbolic rules in chapter 11 simplify, depends on that. With truncation, $-1 \% 4$ is -1 , and a rule that assumes remainders are non-negative would be wrong.

Division by zero and the one overflowing case need definitions too, because the contract has no undefined behaviour:

- $x // 0$ is 0 and $x \% 0$ is x . This is arbitrary, and it is a choice, not a fact of nature. It is chosen so that $x == 0 * (x // 0) + x \% 0$ still holds, and so that no input can crash the program.
- $\text{INT_MIN} // -1$ would be 2^{31} , which does not fit. It wraps to INT_MIN . In C this input is undefined and, on x86, it crashes with SIGFPE. Chapter 12 shows the crash.

Casts

Casts between the three types need a rule each.

float to int casts · output of demos/number_facts.py

```
cast f32->i32 of      3.99 =      3
cast f32->i32 of     -3.99 =     -3
cast f32->i32 of 10000000000.0 = 2147483647
cast f32->i32 of -10000000000.0 = -2147483648
cast f32->i32 of      nan =      0
cast f32->i32 of      inf = 2147483647
cast f32->i32 of 2147483520.0 = 2147483520
cast f32->i32 of 2147483648.0 = 2147483647
numpy float32(2**31).astype(int32), the raw x86 result that C leaves undefined:
```

```
-2147483648
i32 -> f32 of 16777217: 16777216 (not exactly representable)
```

- Float to int truncates toward zero, so `3.99` becomes `3` and `-3.99` becomes `-3`. A value past the end of the int range **saturates** to the nearest end, `1e10` becomes `2147483647`. NaN becomes `0`. This is the rule that Rust uses for its `as` operator and that its reference states. C leaves out-of-range conversions undefined. The raw x86 instruction returns `-2147483648` for them, which NumPy shows in the last lines. The compiler's C has a helper that adds the two tests.
- Int to float rounds to nearest. `16777217` is not representable and becomes `16777216`.
- Anything to bool is `value != 0`. So NaN converts to true.
- Bool to int is 0 or 1.

Bool

There are two values. The syllabus fixes the meaning of three ops on them: `MUL` is AND, `MAX` is OR, and `CMPE` is XOR. Each is the same function as on numbers, restricted to 0 and 1. `1 * 1 = 1`, `1 * 0 = 0`. `max(0, 1) = 1`. `0 != 1` is true. The benefit is that one op set covers masks and arithmetic. There is no `ADD` for bool: `1 + 1` has no value in a type with two values, and `CMPE` already does XOR.

What Python gives you for free, and what it does not

Python's `int` never wraps. Python's `float` is a 64 bit double. Python's `//` floors, and `7 // 0` raises an error. So the reference implementation has to add wrapping by hand, round to float32 by hand, and handle the zero divisor by hand. That is what `semantics.py` does. It is 139 lines and is the most heavily tested file in the module.

Where to learn more

- David goldberg, *what every computer scientist should know about floating-point arithmetic*, at docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html. The standard introduction. Read sections 1 and 2 for rounding and the representation, and the section on cancellation.
- Randal Bryant and david o'hallaron, *computer systems: a programmer's perspective* (CS:APP). Chapter 2 covers two's complement, overflow and floating point with exercises. It is the best treatment of integers on a real machine.
- Henry warren, *hacker's delight*. A catalogue of integer tricks, including division by constants, which chapter 11 uses indirectly.

- John regehr, *a guide to undefined behavior in C and C++*, at blog.regehr.org/archives/213. Three parts. Read part 1 for the signed overflow case this chapter described.
- Pierre roux, *innocuous double rounding of basic arithmetic operations*, journal of formalized reasoning 7(1), 2014, for the proof used above.
- Jean-michel muller and others, *handbook of floating-point arithmetic*. The long reference. You do not need it for this module.

Exercises

1. Compute `wrap32(2**31 + 7)` and `wrap32(-2**31 - 7)` by hand, then check with the function.
2. Write the 32 bit pattern of `-1.0` as sign, exponent and fraction. Check it with `struct`.
3. Find the smallest positive integer `n` for which `float32(n + 1) == float32(n)`. What is the gap between floats there?
4. Show that `(-7) // 2 == -4` and `(-7) % 2 == 1` give `a == b*(a//b) + a%b`. Then show that for truncating division the same identity holds with a different quotient and remainder.
5. With `a = 1e8`, `b = -1e8`, `c = 1.0` in `float32`, compute `(a+b)+c` and `a+(b+c)`. Then find three `float32` values of your own where the two groupings differ.

Chapter 3

What every op means

An op is a kind of node: `ADD`, `MUL`, `WHERE` and so on. Before any graph exists, each op needs a definition that says what value it gives for every possible input. This book calls that definition the **contract**, and it lives in one file, `uopc/semantics.py`.

The contract is the most important design decision in the module, for one reason. A rewrite is correct if the new graph gives the same value as the old one, and “the same value” has no meaning until each op has an exact meaning. Everything else in the compiler is judged against this file.

One place that knows

Four parts of the compiler need to know what ops mean: the constant folder (chapter 9), the reference interpreter, the symbolic rules (chapters 10 and 11, which assume the meaning), and the C renderer (chapter 12, which must produce C with the same meaning). The contract is the only place the meaning is written as code. The folder calls it. The interpreter calls it. The renderer is tested against it. The rules are tested against it, and the important ones are proved.

Here is the function:

`eval_op` in `uopc/semantics.py` · compared with numpy and c on 400,000 checks each

```
def eval_op(op: Ops, dtype, vals, arg=None):
    """apply `op` to python values. `dtype` is the dtype of the result."""
    if op is Ops.CAST:
        return cast(vals[0], dtype)
    if op is Ops.WHERE:
        return vals[1] if vals[0] else vals[2]
    k = dtype.kind
    if len(vals) == 1:
        a = vals[0]
        if op is Ops.NEG:
            return wrap32(-a) if k == "int" else -a
        if op is Ops.RECIP:
            return f32(_recip(a))
        if op is Ops.EXP2:
            return f32(_exp2(a))
        if op is Ops.LOG2:
            return f32(_log2(a))
    else:
        a, b = vals
        if op is Ops.ADD:
            return wrap32(a + b) if k == "int" else f32(a + b)
```

```

if op is Ops.MUL:
    if k == "bool":
        return a and b
    return wrap32(a * b) if k == "int" else f32(a * b)
if op is Ops.MAX:
    return a if a > b else b
if op is Ops.CMPLT:
    return a < b
if op is Ops.CMPNE:
    return a != b
if op is Ops.FLOORDIV:
    return floordiv(a, b)
if op is Ops.FLOORMOD:
    return floormod(a, b)
raise ValueError(f"no semantics for {op}")

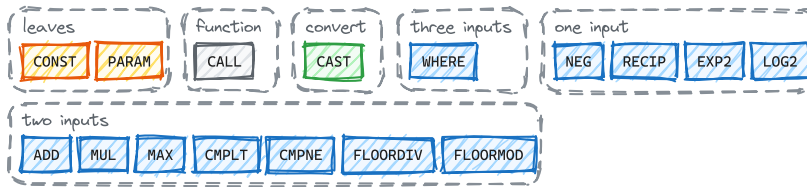
```

It takes the op, the dtype of the result, and Python values, and returns the value. Every branch is total: no input makes it raise an error or leaves the result undefined. The helpers `floordiv`, `floormod` and `cast` were shown in chapter 2.

The table

The full op set of this module has 23 ops. The 16 ops of week 1 are in this table. The memory ops come in chapter 14.

Op	Srcs	Dtypes	Value
CONST	None	Any	The number in <code>arg</code>
PARAM	None	Any	Input number <code>slot</code> of the call
CALL	Body, args	Any	The body with each <code>PARAM</code> replaced by its argument
CAST	1	Any to any	Convert to <code>arg</code> , rules in chapter 2
NEG	1	Int, float	<code>-x</code> ; for int <code>wrap32(-x)</code>
RECIP	1	Float	<code>1.0 / x</code> , with <code>1/0 = inf</code> of the right sign
EXP2	1	Float	2 to the power <code>x</code>
LOG2	1	Float	The base 2 logarithm
ADD	2	Int, float	The sum; ints wrap, floats round
MUL	2	Bool, int, float	The product; bool is AND
CMPLT	2	Any	Bool, <code>a < b</code>
CMPE	2	Any	Bool, <code>a != b</code> ; on bool it is XOR
MAX	2	Bool, int, float	<code>a</code> if <code>a > b</code> else <code>b</code> ; on bool it is OR
FLOORDIV	2	Int	Floor quotient, <code>x // 0 = 0</code>
FLOORMOD	2	Int	Floor remainder, <code>x % 0 = x</code>
WHERE	3	Any	<code>a</code> if <code>c</code> else <code>b</code> , for <code>WHERE(c, a, b)</code>

the 16 ops

the point:

every op has one rule for its dtypes and one rule for its value. nothing else.

The dtype rule for every op is checked when a node is built, and chapter 4 shows how. An `ADD` of an `int` and a `float` does not exist. A program that tries to build one fails at once, with a message that names the op.

Why so few ops

There is no subtraction, no division, no `CMPLE`, no `CMPGT`, no `AND`, no `OR`, no `NOT`, no `SQRT`. Each of these is a short combination of ops that exist:

- `a - b` is `a + (-b)`. In floats this is exact: subtraction is addition of the negation in IEEE arithmetic.
- `a / b` is `a * recip(b)`. This is **not** the same as correctly rounded division, because the reciprocal rounds and then the product rounds again. It is the definition used here, so the compiler's result for a division is the result of a reciprocal followed by a multiply, and it can differ from a correctly rounded division in the last bit. Exercise 4 finds such an input.
- `a > b` is `b < a`. `a >= b` for ints is `not (a < b)`.
- `not x` for bool is `x != true`, which is a `CMPNE` with a constant.
- `sqrt(x)` is `exp2(0.5 * log2(x))`, up to the accuracy of those two, which is not exact. This module does not need it.

Every op removed from the set is one fewer case for every rule, every renderer and every test, so the set is small on purpose.

Careful with comparisons on floats. For ints, `a <= b` is exactly `not (b < a)`. For floats it is not: with `a = nan`, `b < a` is false so `not (b < a)` is true, but `a <= b` is false. A front end that builds `a <= b` from `CMPLT` must handle NaN explicitly, or must say its comparison is not the IEEE one. The contract does not hide this.

The one rule that makes the contract possible: totality

Every op is defined for every input. That is why `x // 0` is 0 and why float to int saturates. The reason is not tidiness. It is `WHERE`.

`WHERE(c, a, b)` is a select: it gives `a` if `c` is true and `b` otherwise. In a graph, `a` and `b` are nodes like any others, and the C renderer writes one statement for each node. So both `a` and `b` are computed, always. The program

```
y = WHERE(x != 0, 100 // x, 0)
```

Computes `100 // x` even when `x` is 0, and throws the result away. If division by zero crashed, this program would crash exactly when the programmer meant to protect against it. A language that evaluates both branches of a select must make every op safe to run on every input. `x // 0 = 0` is the cheapest safe definition.

The same argument covers `INT_MIN // -1` and the float to int cast. In C each of these is undefined or crashes. The contract gives each a value, and the C renderer writes helper functions to match.

What is exact and what is not

Every op in the table is **bit exact** between the Python definition and the C output, except two. `EXP2` and `LOG2` call the math library, and the math library of Python and the math library of C do not return the same last bit on every input. The tests measure the gap:

the semantics checked against numpy and c · output of tests/test_semantics.py

```
semantics vs numpy: 400000 checks passed; max ulp difference for exp2/log2: {'EXP2': 1, 'LOG2': 1}
semantics vs C:      400000 checks passed; max ulp difference for exp2f/log2f: {'exp2': 1, 'log2': 1}
```

The largest difference found, over 20,000 random and special inputs each, was 1 ulp, where an ulp is the gap between neighbouring floats at that value. So the contract for these two ops is “within 1 ulp of the correctly rounded result, as the C library computes it”, and nothing stronger. This has a consequence the book repeats where it matters: folding `EXP2(3.0)` at compile time and computing it at run time are not guaranteed to give the same last bit for an arbitrary argument. The bit exact tests in this module leave `EXP2` and `LOG2` out of the random programs for this reason, and the text says so wherever a test depends on that. For the other 14 ops, the tests demand identical bits.

How the contract was checked

There are three independent implementations of the contract, so a mistake in one shows up as a disagreement:

1. `semantics.py`, in Python, using wrapping and `struct`.
2. NumPy, which has its own `int32` and `float32` arithmetic. The test compares every op, for ints, floats and casts.
3. `tests/ref_ops.c`, a separate C implementation written to use different algorithms. For floor division it uses 64 bit arithmetic and no helper from the renderer.

Random inputs are only part of it. The generator draws 25% of integers from a list of values that break things (0, 1, -1, 2^{30} , the extremes, one off the extremes) and 40% of floats from a list of special values (the two zeros, the infinities, NaN, the largest and smallest normal and subnormal floats, 16777216, 16777217). And every pair from those two lists is included once. A bug that only appears with `INT_MIN` or `-0.0` cannot hide in random data, so it is put in by hand.

The result is 400,000 checks against NumPy and 400,000 against C, and every one passes. This is evidence, not proof. Chapter 15 says what a proof would add.

Where to learn more

- *The IEEE 754-2019 standard* defines the operations and rounding. The standard itself is behind a paywall. Goldberg’s paper in chapter 2’s reading list is the free substitute.
- The Python language reference, section on binary arithmetic operations, defines floor division and modulo. Python’s definition is the one this contract copies for non-zero divisors.
- The C standard, section on integer conversions and undefined behaviour, is the other side. N3096, a late draft of c23, is free and readable. Search for “6.3.1.4 real floating and integer” for the float to int rule.
- Chris lattner’s blog series *what every C programmer should know about undefined behavior* (LLVM project blog, 2011, three parts) explains why compilers exploit it.
- The tinygrad source: `tinygrad/uop/ops.py` has a function called `python_alu` that plays the role of this chapter’s `eval_op`.

Exercises

1. Evaluate `eval_op(Ops.FLOORDIV, I32, [-8, 3])` and `eval_op(Ops.FLOORMOD, I32, [-8, 3])` by hand, then with the code. Then compute $3 * (-8 // 3) + (-8 \% 3)$.
2. What does the C ternary `(a > b) ? a : b` return for `a = 0.0` and `b = -0.0`? And for the arguments swapped? What does this say about reordering the arguments of `float MAX`?
3. Build `WHERE(x != 0, 100 // x, 0)` for an int param `x` and evaluate it at `x = 0`, `x = 7` and `x = -3`.
4. Write `a / b` in terms of the ops in the table. Compute `7.0 / 3.0` that way in `float32` and compare with NumPy's `np.float32(7) / np.float32(3)`. Are they the same? Find an `a, b` pair where they differ.
5. Explain why `a <= b` for floats cannot be written as `not (b < a)`. Give the input.

Part 2

The graph

Chapter 4

The uop

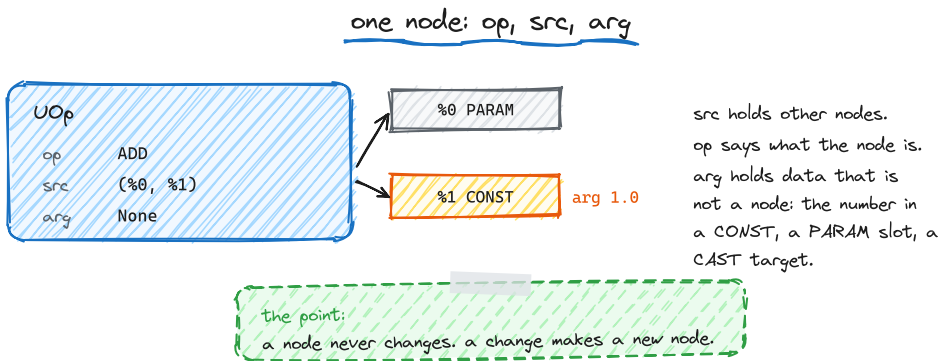
The compiler has one data structure for everything: the `UOp`. A program, an expression, a constant, a loop (in a later module), and the function that wraps them are all `UOp`s. One type means one set of tools for walking, printing, comparing and rewriting, and nothing else to learn.

The name is short for “micro operation”. A `UOp` is a single node in a graph. It has three fields.

```
UOp(op, src, arg)
```

- `op` is the kind of node. It is a member of the `Ops` enum: `ADD`, `CONST`, `PARAM`, and the rest from chapter 3.
- `src` is a tuple of other `UOp`s: the inputs to this node. `ADD` has two. `NEG` has one. `CONST` has none.
- `arg` is extra data that the op needs and that is not itself a node: the number in a `CONST`, the slot of a `PARAM`, the target dtype of a `CAST`.

A `UOp` is **immutable**. Once built, none of its fields can change. Every change to a program makes new nodes. This is the property that makes sharing safe, and chapter 5 depends on it.



Reading a graph

The relu program from chapter 1 is a graph of eight nodes. Here it is again in the text format, which is how this book prints graphs:

the relu program · text format

```

; uop v1
%0 = param : dtype=f32 slot=0
%1 = param : dtype=f32 slot=1
%2 = mul %0, %1
%3 = const : dtype=f32 value=1.0
%4 = add %2, %3
%5 = const : dtype=f32 value=0.0
%6 = max %4, %5
%7 = call %6, %5, %5 : name=relu

```

Read from the top. `%0` and `%1` are the two inputs, parameters number 0 and 1, both `f32`. `%2` multiplies them. `%3` is the constant 1.0. `%4` adds. `%5` is the constant 0.0 and `%6` takes the larger of the sum and zero. `%7` is the `CALL`, which says: the function body is `%6`, and it is called with `%5` and `%5` as the arguments for parameters 0 and 1. Here both are the constant 0.0, which only supplies the types of the parameters. The renderer turns a `CALL` into a C function and ignores the argument values.

Two things to see in this listing. A node refers to its inputs by number, and an input always has a smaller number than the node using it. And `%5` is used three times: once by `%6` and twice by `%7`. A number appearing in more than one place is not a copy. It is the same node. This format is called **SSA**, for static single assignment: each name is assigned exactly once. A graph in SSA form has no variables that change, which is why rewriting it is safe.

The syllabus fixes this text format as the interchange format of the course. Chapter 5 covers writing and reading it.

How nodes are built

You do not call `UOp(...)` directly for arithmetic. Python operators build nodes:

```

x * y           # UOp(Ops.MUL, (x, y))
x * y + 1.0     # UOp(Ops.ADD, (that, UOp.const(1.0)))
-x             # UOp(Ops.NEG, (x,))
x - y          # x + (-y), because there is no SUB op
x // 3         # UOp(Ops.FLOORDIV, (x, UOp.const(3)))
a.lt(b)        # CMPLT
a.max(b)       # MAX
c.where(a, b)  # WHERE

```

A plain Python number on the right is turned into a `CONST` of the right type: `x * 2` for an `int` `x` and `x * 2.0` for a float. There are no `==` or `<` operators on `UOp`. In Python, `a == b` would be expected to return a bool, and the compiler needs `a == b` to mean “are these two the same node”. The `==` that Python gives every object does exactly that, and it is a pointer compare. The compiler uses it everywhere.

The graph checks itself

The most useful property of the `UOp` is that **a graph with a type error cannot be built**. Each time a node is created, a function called `infer` looks at the `op`, the `src` and the `arg`, and either computes the node's type or raises an error:

`infer` in `uopc/uop.py` · the type checker, run on every node as it is built

```
def infer(op: Ops, src, arg):
    def need(cond, msg):
        if not cond:
            raise UOpError(f"{op.name}: {msg}")

    n = len(src)
    if op is Ops.CONST:
        need(n == 0, "takes no srcs")
        dt = dtype_of_value(arg)
        need(dt is not I32 or INT_MIN <= arg <= INT_MAX, f"{arg} does not fit in int32")
        return dt, (), ALU_SPACE
    if op in (Ops.PARAM, Ops.BUFFER, Ops.ALLOC):
        need(n == 0, "takes no srcs")
        need(isinstance(arg, ParamArg), "arg must be a ParamArg")
        dt = arg.dtype
        if op is Ops.PARAM and isinstance(dt, DType):
            need(dt in VALUE_DTYPES, "a scalar param must be bool, i32 or f32")
            return dt, (), ALU_SPACE
        need(isinstance(dt, Ptr), "needs a pointer dtype")
        need(dt.size > 0, "size must be positive")
        return dt, (dt.size,), MEM_SPACE
    if op is Ops.CALL:
        need(n >= 1 and isinstance(arg, CallArg), "needs a body and a CallArg")
        body = src[0]
        inside = body.toposort()
        need(not any(m.op in (Ops.CALL, Ops.BUFFER) for m in inside),
            "a body may not contain a CALL or a BUFFER (global buffers come in as params)")
        for p in (m for m in inside if m.op is Ops.PARAM):
            s = p.arg.slot
            need(0 <= s < n - 1, f"PARAM slot {s} has no argument")
            need(src[1 + s].dtype == p.dtype, f"argument {s} has dtype {src[1 + s].dtype}, "
                f"the body wants {p.dtype}")
            need(src[1 + s].shape == p.shape, f"argument {s} has shape {src[1 + s].shape}, "
                f"the body wants {p.shape}")
        for a in src[1:]:
            need(a.dtype != VOID, "an argument must have a value")
        return body.dtype, body.shape, body.space
    if op in ALU:
        need(n >= 1, "needs at least one src")
        for s in src:
            need(s.space == ALU_SPACE and isinstance(s.dtype, DType) and s.dtype !=
```

```

VOID,
    f"every src must be a value in ALU space, got {s.op.name} in {s.space}")
    shape = src[0].shape
    need(all(s.shape == shape for s in src), "all srcs must have the same
shape")
    a = src[0].dtype
    if op is Ops.CAST:
        need(n == 1 and arg in VALUE_DTYPES,
"needs one src and a value dtype as arg")
        return arg, shape, ALU_SPACE
    if op is Ops.WHERE:
        need(n == 3 and a is BOOL, "needs a bool condition and two values")
        need(src[1].dtype == src[2].dtype, "both branches must have the same
dtype")
        return src[1].dtype, shape, ALU_SPACE
    if op in (Ops.NEG,):
        need(n == 1 and a in (I32, F32), "needs one int or float")
        return a, shape, ALU_SPACE
    if op in (Ops.RECIP, Ops.EXP2, Ops.LOG2):
        need(n == 1 and a is F32, "needs one float")
        return a, shape, ALU_SPACE
    need(n == 2 and src[1].dtype == a, "needs two srcs of the same dtype")
    if op is Ops.ADD:
        need(a in (I32, F32), "ADD is not defined on bool (CMPNE is xor)")
        return a, shape, ALU_SPACE
    if op in (Ops.MUL, Ops.MAX):
        return a, shape, ALU_SPACE
    if op in (Ops.CMPLT, Ops.CMPNE):
        return BOOL, shape, ALU_SPACE
    need(a is I32, "needs ints") # FLOORDIV, FLOORMOD
    return a, shape, ALU_SPACE
    if op is Ops.INDEX:
        need(n == 2, "needs a pointer and an index")
        p, i = src
        need(p.space == MEM_SPACE and isinstance(p.dtype, Ptr) and p.op is not
Ops.INDEX,
            "src[0] must be a buffer-like pointer")
        need(i.dtype is I32 and i.space == ALU_SPACE and i.shape == (), "the index
must be a scalar i32")
        return p.dtype, p.shape, MEM_SPACE
    if op is Ops.LOAD:
        need(n == 1 and src[0].op is Ops.INDEX, "needs one INDEX")
        return src[0].dtype.base, (), ALU_SPACE
    if op is Ops.STORE:
        need(n == 2 and src[0].op is Ops.INDEX, "needs an INDEX and a value")
        need(src[1].dtype == src[0].dtype.base and src[1].space == ALU_SPACE and
src[1].shape == (),
            "the stored value must be a scalar of the element dtype")
        return VOID, (), None
    if op is Ops.AFTER:
        need(n >= 1 and src[0].space == MEM_SPACE and isinstance(src[0].dtype,
Ptr), "src[0] must be a pointer")
        need(all(s.op in (Ops.STORE, Ops.LOAD) for s in src[1:]), "the other srcs

```

```

must be STOREs or LOADs")
    return src[0].dtype, src[0].shape, MEM_SPACE
if op is Ops.SINK:
    need(n >= 1 and all(s.op is Ops.STORE for s in src), "needs one or more
STOREs")
    return VOID, (), None
    raise UOpError(f"unknown op {op}")

```

`infer` returns three things for each node: its dtype, its shape (chapter 6) and its address space (chapter 6). The type of a node is never stored by the person who wrote the program. It is computed from the node's own contents. So it cannot disagree with them.

Here is what the checks look like in practice, from `demos/errors.py`:

eleven ways to build a bad graph, and one good one · output of `demos/errors.py`

int + float	UOpError: ADD: needs two srcs of the
same dtype	
ADD of two bools	UOpError: ADD: ADD is not defined on
bool (CMPNE is xor)	
WHERE with an int condition	UOpError: WHERE: needs a bool
condition and two values	
WHERE with branches of different dtype	UOpError: WHERE: both branches must
have the same dtype	
RECIP of an int	UOpError: RECIP: needs one float
FLOORDIV of floats	UOpError: FLOORDIV: needs ints
CONST 2**31 as int32	UOpError: CONST: 2147483648 does not
fit in int32	
CAST to a pointer type	UOpError: CAST: needs one src and a
value dtype as arg	
CALL with a missing argument	UOpError: CALL: PARAM slot 1 has no
argument	
CALL with an argument of the wrong dtype	UOpError: CALL: argument 1 has dtype
i32, the body wants f32	
a CALL inside a CALL	UOpError: CALL: a body may not
contain a CALL or a BUFFER (global buffers come in as params)	
a well typed CALL	built

Every message names the op and says what was wrong. No bad node is created, so no later stage ever sees one. A rewrite rule that returns a badly typed replacement fails at the line that built the replacement, not three passes later in the renderer. This kind of early failure is the main way the compiler stays debuggable.

The checks do not catch every wrong program. `x // y` for ints is well typed even if `y` is always 0. Checks of that kind are the work of the tests.

CALL and PARAM

A `CALL` has one `src[0]`, the **body**, and then zero or more arguments `src[1:]`. The body is a graph, and wherever it needs an input it contains a `PARAM`. A `PARAM` has a `slot`, a number that says which argument it stands for. `PARAM` slot 0 stands for `src[1]`, slot 1 stands for `src[2]`, and so on. The meaning of a `CALL` is the meaning of its body with each `PARAM` replaced by the matching argument. That is a function call written as a graph.

`infer` checks it all. Every `PARAM` slot used in the body must have an argument. The argument's dtype and shape must be what the `PARAM` expects. A `CALL` may not appear inside a body, nor may a `BUFFER`, so a body is always a flat expression over its parameters. Chapter 9 shows how the compiler runs a `CALL` by substitution. Chapter 12 shows how it writes one as a C function whose parameters are the `PARAM`s.

A `PARAM` for an int may carry bounds: `ParamArg(slot, dtype, vmin, vmax)`. These are a promise by the caller that the value will stay within them. Chapter 6 uses them for range analysis, and chapter 11 for index simplification.

Walking a graph

The one operation every pass needs is to visit each node once, with every node after its inputs. This order is called a **topological order**, and `toposort` makes it:

`toposort` in `uopc/uop.py`

```
def toposort(self, skip=None) -> list["UOp"]:
```

"""every node reachable from self, each after all of its srcs. iterative: graphs can be deeper than python's recursion limit."""

```
    order, seen = [], set()
    stack = [(self, False)]
    while stack:
        n, done = stack.pop()
        if done:
            order.append(n)
            continue
        if n in seen or (skip is not None and skip(n)):
            continue
        seen.add(n)
        stack.append((n, True))
        for s in reversed(n.src):
            stack.append((s, False))
    return order
```

It uses an explicit stack, not recursion. A recursive version fails on a deep graph because Python limits recursion to 1000 calls. `demos/errors.py` builds a chain of 200,000 additions and walks, evaluates and simplifies it:

a graph 200,000 nodes deep · output of `demos/errors.py`

```
chain of 200,000 adds: 400,001 nodes; build 4.09s, toposort 0.62s, evaluate 0.64s,
simplify 14.19s
evaluate gives -1,474,736,480, the sum 1..200,000 is 20,000,100,000 wrapped to
int32: -1,474,736,480
simplify merged all 200,000 constants into one: the result has 3 nodes, an ADD of x
and the constant -1,474,736,480
```

The build took 4.1 seconds, the sort 0.6, the evaluation 0.6. Simplify took 14.2 seconds and merged the 200,000 added constants into a single one. The three nodes left are `x`, the constant and the `ADD`. That sum is `20,000,100,000`, which does not fit in an `int32`, so it is wrapped, and the program prints the wrapped value both ways to show that evaluating the long chain and evaluating the simplified graph agree.

Every pass in this book is written with an explicit stack for this reason. A compiler for neural networks sees graphs with hundreds of thousands of nodes.

The same idea in tinygrad

The class in tinygrad is also called `UOp`, in `tinygrad/uop/ops.py`. It also has `op`, `src` and `arg`, plus a `tag` field used by some passes, which this module does not need. It is hash consed in the same way, through a table called `ucache`. Its dtype is also computed from the other fields, by a function called `dtype_from_uop`, and cached on first use. So the design in this chapter is the design of the real compiler.

One difference is in when checking happens. Here `infer` runs on every node, always. In tinygrad the type rules of legal graphs live in `tinygrad/uop/spec.py` and are checked when the environment variable `SPEC` is set above 1, so that normal runs do not pay for them. tinygrad's ops use the names `FLOORDIV` and `FLOORMOD` in the version read for this book, the same as the syllabus.

Where to learn more

- *Engineering a compiler*, by keith cooper and linda torczon. Chapter 5 covers intermediate representations, including graphs and SSA form.
- The LLVM language reference, at LLVM.org/docs/LangRef.html, is the best real example of a typed SSA IR.

- The free draft of the SSA book (*SSA-based compiler design*, many authors), at pfalcon.github.io/ssabook/latest. Start with the introduction.
- The tinygrad source: `tinygrad/uop/ops.py`, the `UOp` class, and `tinygrad/uop/spec.py`, which holds the rules for what a legal graph is.
- Chris lattner and vikram adve, *LLVM: a compilation framework for lifelong program analysis and transformation*, CGO 2004. The original LLVM paper.

Exercises

1. Build the graph for `(x + 1) * (x + 1)` with an `int x`. How many nodes does it have? Which node is used twice?
2. Try to build `x + y` where `x` is `i32` and `y` is `f32`. Read the message. Now build `x.cast(F32) + y`.
3. Write the text format for `x * 2 + 3` by hand, for an `int x` with slot 0. Check it against `dumps` of the real graph.
4. A `PARAM` with slot 2 is used in a body, and the `CALL` passes only two arguments. What error do you get, and where is it raised?
5. What is `UOp.const(1)` is `UOp.const(1)`? What is `UOp.const(1)` is `UOp.const(True)`? Explain.

Chapter 5

Hash consing

The syllabus says five words about the `UOp` that matter more than the rest: “the `UOp` is hash consed”. This chapter explains what that means, how to build it, which mistakes are easy to make, and why almost every later chapter gets simpler because of it.

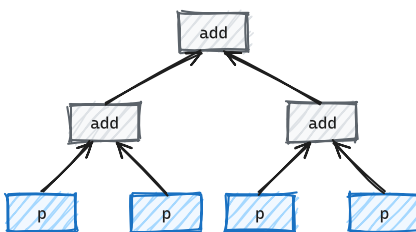
The idea

Hash consing means: when you build a node, first check whether a node with the same contents already exists. If it does, return that node and build nothing. If not, build it and remember it.

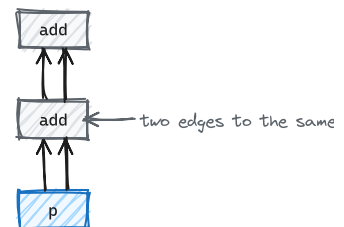
The effect is a guarantee. At any moment, two live nodes with the same `op`, the same `src` and the same `arg` are the same object. Two nodes that are different objects have different contents. So to ask “do these two graphs compute the same thing structurally” you compare two pointers. In Python that is `a is b`. It takes constant time, however large the graphs are.

The name comes from lisp, where building a list cell is called “consing”. The table that remembers existing nodes is a hash table, so the check is fast.

$p = p + p$, twice: a tree and a shared graph



as a tree: 7 nodes



hash consed: 3 nodes

the point:
with sharing, 18 doublings make 19 nodes. as a tree they would make 524,287.

The code

The whole mechanism is in `UOp.__new__`. In Python, `__new__` runs before `__init__` and decides which object is returned:

`UOp.__new__` in `uopc/uop.py`: look up, else build

```
def __new__(cls, op: Ops, src=(), arg=None):
    src = tuple(src)
    if op is Ops.CONST:
        arg = _norm_const(arg)
    key = (op, src, _key_arg(arg))
    u = cls._cache.get(key)
    if u is not None:
        return u
    dtype, shape, space = infer(op, src, arg)
    u = object.__new__(cls)
    u.op, u.src, u.arg = op, src, arg
    u.dtype, u.shape, u.space = dtype, shape, space
    u._rng = None
    u.id = cls._next_id
    cls._next_id += 1
    cls._cache[key] = u
    return u
```

The key is the tuple `(op, src, _key_arg(arg))`. The lookup is a single dictionary access. If it finds a node, the function returns it and nothing else runs, in particular not `infer`, the type checker from chapter 4. A node that is found in the table was type checked when it was first built. Checking it again would be waste.

There are two properties that make this work.

The srcs are already unique. The key contains `src`, a tuple of nodes. Python hashes a tuple by hashing its items, and hashes a node by its address, since `UOp` defines neither `__eq__` nor `__hash__`. Two keys are equal only if the srcs are the same objects. This is correct because every src was itself built through the table. By induction, the table is a correct index of all nodes ever built.

Nodes never change. If a node could be edited after it entered the table, its key would no longer match its contents. So all fields are set once, in `__new__`, and never again.

The table is a `weakref.WeakValueDictionary`. A weak reference does not keep an object alive. When the last ordinary reference to a node disappears, the node is deleted and its table entry disappears too. Without this, the table would grow forever as the compiler built and discarded graphs.

Each node gets an integer `id` from a counter when it is built. Ids increase with creation time. A node that is built again after it was deleted gets a new id. The ids matter for two things, the ordering of operands by the rule `canonical order` (chapter 8) and one of the visiting orders of the memory interpreter (chapter 14). They are never used to compare nodes.

Three traps in the key

The key is easy to get subtly wrong, because Python's idea of equality is not the idea that a compiler needs. The key for the `arg` of a constant has to handle three cases, and each is a real bug in a first draft:

the key of a constant · `uopc/uop.py`

```
def _norm_const(v):
    if type(v) is float:
        v = f32(v)
        return math.nan if v != v else v
    return v

def _key_arg(arg):
    """the hash-cons key for an arg. python says True == 1 == 1.0 and 0.0 == -0.0
    and nan != nan,
    and all three would corrupt the table, so constants are keyed by (type,
    bits)."""
    t = type(arg)
    if t is float:
        return ("f32", f32_bits(arg))
    if t is bool or t is int:
        return (t.__name__, arg)
    return arg
```

True, 1 and 1.0 are equal in Python. `True == 1` is true, and as keys of a dictionary they collide. So `UOp.const(True)` and `UOp.const(1)` would return the same node, and a boolean would turn into an integer without any error. The key therefore includes the type of the value: `("bool", True)` and `("int", 1)` are different keys.

0.0 == -0.0 in Python. They are equal as numbers, and different values for a compiler: the two zeros behave differently in chapter 2 (`1 / 0.0` and `1 / -0.0`). The key for a float is the bit pattern of its float32 form. `0.0` has bits `0x00000000` and `-0.0` has `0x80000000`, so they are different nodes.

`nan != nan` in Python. A NaN used as a dictionary key is never found again by value, so every `UOp.const(nan)` would build a new node and the table would fill with copies. The key uses a single canonical NaN bit pattern, so all NaNs are one node. This loses the distinction between different NaN payloads, which the contract does not have.

Each trap has a test. The unit test for the zero key is one of the 19 bugs planted in chapter 15: with the key changed to `float(arg)`, the suite fails at once. The output of the demo:

hash consing and the three traps · output of `demos/run_demos.py`

```
(p+q) is (p+q): True
nodes in (p+q)*(p+q)+(p+q): 5 (a tree would have 11)
-0.0 and 0.0 are different nodes: True
1 and True are different nodes: True
python says 1 == True: True and 0.0 == -0.0: True
```

What you get for free

Common subexpression elimination. If a program writes `p + q` three times, there is one node. There is no pass to find the repeats. `(p+q)*(p+q) + (p+q)` has 5 nodes, as the output shows, where a tree has 11. A pass that does this in a compiler without hash consing needs a table that does what the constructor does here.

Equality is a pointer compare. The pattern matcher of chapter 7 matches a pattern that uses the same name twice, like `x + x`, by checking that both positions are the same node: `a` is `b`. It is one instruction.

Memoised rewriting. The rewrite engine of chapter 8 keeps a dictionary from each node to what it rewrote to. A node reached from two places is rewritten once. Sharing costs nothing and is preserved.

Small graphs. The largest saving is not constants but sharing in repeated structure. The program `p = p + p` applied 18 times is a graph of 19 nodes. As a tree it would have 524,287. The benchmark program (`bench.py`, appendix d) builds the graph and counts its 19 nodes. The tree size is computed from the formula $2^{19} - 1$.

Memory use is not small. For 300 random index expressions, Python allocated 870 KiB, which is 423 bytes per distinct node, measured with `tracemalloc` while building them. A Python object with eight fields, a tuple, a weak reference and a dictionary entry is that large. A compiled implementation would use a small fraction of it. This module does not care. A node does not need to be small when the graphs are small, and the syllabus lets the student choose any language.

The text format

A program in memory is a graph of objects. To store one, print one, or send one to another program, it needs a text form. The syllabus fixes a format for the whole course: a line per node, in the order of a topological sort, naming each node by its position. This is the format used for the listings in this book:

```
; uop v1
%0 = param : dtype=f32 slot=0
%1 = const : dtype=f32 value=1.0
%2 = add %0, %1
```

The first line is a comment that names the version. Every other line is `%N = op srcs : key=value ...`. The `srcs` are earlier line numbers. The keys depend on the op: `dtype` and `value` for a constant, `dtype` and `slot` and optionally `size`, `min` and `max` for a param or buffer, `dtype` for a cast, `name` for a call. Every key a node needs is printed, so the text alone is the whole program. Nothing is left implicit.

`dumps` writes it. It sorts the graph with `toposort` and numbers the nodes by their position. `loads` reads it. It builds each node with `UOp(...)`, in order, which is exactly what makes the result hash consed. The property that the two are inverses is stated as a test:

```
loads(dumps(u)) is u          # a pointer compare, not an equality of fields
```

It holds, as a pointer compare, for 300 random programs and 300 randomly simplified programs (and the bad inputs below). The `is` is stronger than “the same structure”. It says that reading the text built no new nodes. Every node was found in the table, so the text described exactly the graph that existed.

A draft of the course’s own text format was shown in public by the course author, george hotz. This book saw it only as screenshots, not as a specification. The draft has the same shape as the format here: one line per node, `%n = op %a, %b : key=value`. The draft is early and may change, and the two formats were not checked against each other beyond that shape. Every piece of code that reads or writes this book’s format is in one file, `uopc/textfmt.py`, so if the course format changes, that is the file to edit.

This has a use beyond saving. When two compilers disagree, the text of the graph before and after each rewrite is a record you can read and diff.

`loads` also rejects bad text, with a line number. The first line must name the version. Every source must be a number that appears above. An op or a key must exist. And because `loads` builds nodes through `UOp`, a line that would build an ill-typed node is rejected with the type checker’s message:

five bad files and a good one · output of demos/errors.py

```

missing header                                ParseError: the first line must be ';'
uop v1' (a title may follow), got '%0 = param : dtype=f32 slot=0'
use before definition                          ParseError: line 2: '%1' is not
defined above this line
unknown op                                    ParseError: line 2: unknown op
'frobnicate'
a bad number in a source list                  ParseError: line 3: '%x' is not
defined above this line
wrong dtype in a line                         ParseError: line 4: ADD: needs two
srcs of the same dtype
a good file                                    built

```

Ten further bad inputs are in the test suite. Each must raise `ParseError`, and none may build a node.

Where the cost is

Hash consing makes building a node slower. Every build is a hash of the key and a dictionary lookup. This cost is paid on every node of every rewrite. It is a good trade, and tinygrad makes it too. In `tinygrad/uop/ops.py`, the class `UOpMetaClass` holds a dictionary called `ucache` and, in `__call__`, looks the key up first. The key there is `(op, src, arg, tag, type(arg))`, and a comment above it says the key must separate nodes of different dtype, because `True == 1` as dictionary keys, the same trap as the first one above.

Where to learn more

- Eiichi goto, *monocopy and associative algorithms in an extended lisp*, 1974, an early description of the idea in lisp. It is hard to find. The idea is covered in any text on lisp implementation.
- Jean-christophe filliâtre and sylvain conchon, *type-safe modular hash-consing*, ML workshop 2006. A short paper on doing this correctly in a typed language.
- Donald knuth, *the art of computer programming*, volume 4A, the section on binary decision diagrams, where a table of unique nodes keeps diagrams reduced by the same method.
- The Python documentation for `weakref.WeakValueDictionary`, to see exactly when an entry disappears.
- The tinygrad source: `UOpMetaClass` in `tinygrad/uop/ops.py`.

Exercises

1. Build $(p+q)*(p+q)$ and $(p+q)*(q+p)$. Are the two products the same node? Explain, then run `simplify` on both and check again. (Read chapter 8 first if you are stuck.)
2. Write a loop that builds a graph by `p = p + p` 30 times. How many nodes does the graph have? How many would a tree have? Do not walk the tree.
3. Delete every reference to a graph and run `gc.collect()`. Build the same graph again. Is the `id` of its root the same as before? Why?
4. Take dumps of a program with a float constant. Change `value=1.0` to `value=1` and call `loads`. Is it accepted, and is the node the same as before? Then change an int constant's `value=1` to `value=1.5`. What happens?
5. Make the mistake `float(arg)` in `_key_arg` in a copy of the code. Run `tests/test_rules_unit.py` in the copy. What is the first failure message, and what does it tell you?

Chapter 6

Types, shapes, address spaces and ranges

Some facts about a node are not stored anywhere. They are computed from the node's `srcs`, every time one is asked for, and remembered after the first time. The syllabus calls these **recursive properties**: the property of a node is a function of the properties of its `srcs`. There are four in this module.

Property	What it says	Introduced
<code>dtype</code>	<code>bool</code> , <code>i32</code> , <code>f32</code> , <code>void</code> , or a pointer to one	Week 1
<code>shape</code>	A tuple of sizes, <code>()</code> for a single value	Week 2
<code>addrspace</code>	<code>ALU</code> for a value, <code>MEM</code> for a place in memory	Week 2
Range	The smallest and largest value an int can take, and whether it can wrap	This book

The first three are computed by `infer` when a node is built, as chapter 4 showed. The fourth is computed on demand by a different function. All four have the same shape of definition: look at the node's `op`, look at the properties of its `srcs`, return the answer.

Dtype

A node's `dtype` is the type of the value it computes. A `CONST` has the type of its Python value: `bool` for `True`, `i32` for `5`, `f32` for `5.0`. The order of the test matters, because in Python `True` is also an `int`, and the first line of `dtype_of_value` checks for `bool` before it checks for `int`. A `PARAM` has the `dtype` in its `ParamArg`. An `ADD` has the `dtype` of its `srcs`. A `CMPLT` has `bool` whatever its `srcs` are. A `WHERE` has the `dtype` of its branches. A `STORE` has `void`, because it does not produce a value, it changes memory.

A `BUFFER` or an `ALLOC` has a **pointer** `dtype`, `Ptr(base, size)`: a pointer to `size` consecutive elements of type `base`. Its `repr` reads `ptr<f32,4>`. You cannot add to a pointer. The type checker refuses, as the next section shows.

Shape

The shape of a node is a tuple of sizes. A single number has shape `()`. A buffer of 4 floats has shape `(4,)`. In a later module a reshape, a permute, an expand will change shapes, and the check “every src of an ALU op has the same shape” is what will catch a mismatched broadcast.

In this module the shape property is complete and almost idle. Every arithmetic op requires its srcs to have equal shapes. `CALL` takes the shape of its body, and requires each argument to have the shape of the `PARAM` that it replaces. But the only shapes that appear are `()` and `(n,)`, and the `(n,)` shapes belong to memory, which has no arithmetic. The arithmetic that a student of this course will write later, over whole arrays, will use the check without changing it. This is the reason the property is here at all: it is cheap now and impossible to add cleanly later.

Address space

Every node lives in one of two spaces.

- `ALU` is the space of values. A number, a boolean, the result of arithmetic. It lives in a register, in the generated code.
- `MEM` is the space of places. A buffer, a scratch array, a pointer to one element of either. It lives in memory and has to be loaded from or stored to.

The rules are short. Every arithmetic op needs all srcs in `ALU`. `INDEX` takes a `MEM` pointer and an `ALU` integer and gives a `MEM` pointer. `LOAD` takes a `MEM` pointer and gives an `ALU` value. `STORE` takes a `MEM` pointer and an `ALU` value and gives nothing. Memory is only touched through `LOAD` and `STORE`, and arithmetic only sees values. These are the errors the checker gives when you break them:

six memory graphs that cannot exist, and the properties of a store · output of demos/errors.py

ADD of a pointer and a number value in ALU space, got PARAM in MEM	UOpError: ADD: every src must be a
INDEX with a float index scalar i32	UOpError: INDEX: the index must be a
LOAD straight from a buffer (no INDEX)	UOpError: LOAD: needs one INDEX
STORE of an int into a float buffer must be a scalar of the element dtype	UOpError: STORE: the stored value
a CALL that passes a size 8 buffer for size 4 ptr<f32,8>, the body wants ptr<f32,4>	UOpError: CALL: argument 0 has dtype
SINK of a value that is not a STORE STOREs	UOpError: SINK: needs one or more

```
a node: [('INDEX', 'ptr<f32,4>', (4,), 'MEM'), ('CONST', 'f32', (), 'ALU'),
('STORE', 'void', (), None)]
```

The last line prints (op, dtype, shape, space) of the three nodes of a store. The INDEX is in MEM and has the pointer dtype and the shape of its buffer. The constant is in ALU. The STORE has void, an empty shape, and no space at all.

Chapter 14 covers memory ops. The property matters here because it is the thing that keeps the two kinds of node apart, so that no rewrite rule written for arithmetic can ever be applied to a pointer by mistake.

Ranges

The fourth property is the most useful one in the module. For every i32 or bool node, the compiler can compute a lower bound vmin and an upper bound vmax on its value, and a flag safe.

Where do the bounds come from? From the leaves and from interval arithmetic. A CONST has the range [c, c]. A PARAM has the range of its declared bounds, or the whole of int32 if none were declared. A bool has [0, 1]. Each op then computes the range of its result from the ranges of its inputs, with the rule that is exact for that op:

- ADD: [lo1 + lo2, hi1 + hi2].
- NEG: [-hi, -lo].
- MUL: the minimum and the maximum of the four products lo1*lo2, lo1*hi2, hi1*lo2, hi1*hi2. Four products are needed because signs can flip the order.
- MAX: [max(lo1, lo2), max(hi1, hi2)].
- WHERE: the smallest and largest of the two branches' bounds, or one branch's range if the condition is known.
- CMPLT, CMPNE: [1, 1] if the ranges prove the answer, [0, 0] if they prove the opposite, else [0, 1].
- FLOORDIV by a constant c: divide both bounds by c and sort them. FLOORMOD by a constant c > 0 is [0, c-1], unless the whole input range lies inside one multiple of c, in which case it is exact. For example x % 4 with x in [9, 10] is [1, 2], exactly, and the rule says so.

Here are the ranges of the six nodes of (i*4 + j) // 4 with i in [0, 15] and j in [0, 3]. This is real output of demos/walkthrough.py:

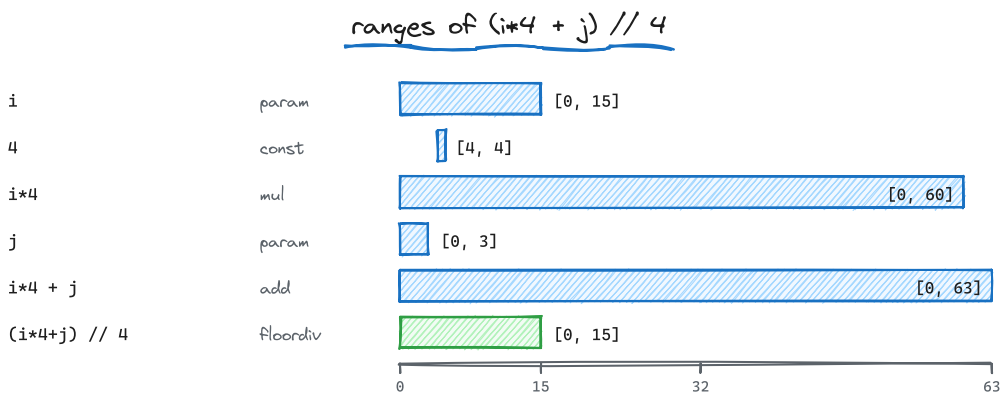
the range of every node · output of demos/walkthrough.py

```
%0 param      0          range [0, 15]  safe=True
%1 const      4          range [4, 4]   safe=True
```

```
%2 mul    %0, %1    range [0, 60]  safe=True
%3 param   1        range [0, 3]  safe=True
%4 add     %2, %3    range [0, 63]  safe=True
%5 floordiv %4, %1    range [0, 15]  safe=True
```

big in $[0, 2^{30}]$, big*4: range $[-2147483648, 2147483647]$ safe=False ($4 \cdot 2^{30} = 2^{32}$ does not fit, so it wraps)
 (big*4)//4 simplified: FLOORDIV (unchanged, because the guard says the split could be wrong)

The bound on `%2`, `i*4`, is $[0, 60]$. Adding `j` gives $[0, 63]$. Dividing by 4 gives $[0, 15]$, which is the range of `i`. The ranges already say something that the simplifier of chapter 11 will turn into a rewrite: the quotient is always the same as `i`.



the point:
 the range of the quotient equals the range of `i`. that is the fact the rewrite in chapter 11 uses.

The safe flag

The intervals above are correct for the true integers. An `int32` is not the true integers: it wraps. If the true result of an `ADD` is 2^{31} , the machine value is -2^{31} , which is far outside the interval that the rule computed. So an interval rule is only correct if the result fits in `int32`.

The compiler therefore checks, in a helper named `fit`, whether the computed interval fits. If it does, the range is returned and the flag stays as it was. If it does not, the range becomes the full range of `int32` and the flag `safe` becomes **False**. An unsafe node is one that might wrap. The flag then spreads upward: a node is safe only if all its `srcs` are safe and the node's own interval fits.

This is the demonstration, also from `demos/walkthrough.py`:

```
big in [0, 2^30], big*4: range [-2147483648, 2147483647] safe=False
```

`big*4` has an interval of `[0, 2^32]`, which does not fit. The flag is `False` and the range is the whole of `int32`. A rule that wants to use “`big*4` is always a multiple of 4 and non-negative” has to check `safe` first, because on the wrapped values it is neither.

Here is the code of the whole range computation:

`_range` and `_compute_range` in `uopc/uop.py`

```
def _range(self):
    if self._rng is None:
        for n in self.toposort(skip=lambda n: n._rng is not None):
            if n._rng is None:
                n._rng = _compute_range(n)
    return self._rng

def _compute_range(u: UOp):
    """(vmin, vmax, safe) for int and bool values. for everything else: the widest
    possible range.
    when an int op could wrap, the range becomes the full int32 range and safe
    becomes False."""
    op, s = u.op, u.src
    if u.dtype is F32 or u.dtype is VOID or not isinstance(u.dtype, DType) or
    u.space != ALU_SPACE:
        return (-math.inf, math.inf, True)
    if op is Ops.CONST:
        return (int(u.arg), int(u.arg), True)
    if op is Ops.PARAM:
        if u.dtype is BOOL:
            return (0, 1, True)
        a = u.arg
        return (INT_MIN if a.vmin is None else a.vmin, INT_MAX if a.vmax is None else
        a.vmax, True)
    if op is Ops.LOAD:
        return (0, 1, True) if u.dtype is BOOL else (*FULL, True)
    rs = [x._range() for x in s]
    safe = all(r[2] for r in rs)

    def fit(lo, hi):
        if INT_MIN <= lo and hi <= INT_MAX:
            return (lo, hi, safe)
        return (*FULL, False)

    if op is Ops.CAST:
        if u.dtype is BOOL:
            if s[0].dtype is F32:
                return (0, 1, True)
            lo, hi, _ = rs[0]
            return (0, 0, True) if lo == hi == 0 else (0, 1, True) if lo <= 0 <= hi
            else (1, 1, True)
```

```

    if s[0].dtype is BOOL:
        return (rs[0][0], rs[0][1], True)
    if s[0].dtype is F32:
        return (*FULL, True)
    return rs[0]
if op is Ops.NEG:
    lo, hi, _ = rs[0]
    return fit(-hi, -lo)
if op is Ops.ADD:
    return fit(rs[0][0] + rs[1][0], rs[0][1] + rs[1][1])
if op is Ops.MUL:
    c = [x * y for x in rs[0][:2] for y in rs[1][:2]]
    return fit(min(c), max(c))
if op is Ops.MAX:
    return (max(rs[0][0], rs[1][0]), max(rs[0][1], rs[1][1]), safe)
if op is Ops.CMPLT:
    (al, ah, _), (bl, bh, _) = rs
    if s[0].dtype is F32:
        return (0, 1, True)
    return (1, 1, True) if ah < bl else (0, 0, True) if al >= bh else (0, 1, True)
e)
if op is Ops.CMPNE:
    (al, ah, _), (bl, bh, _) = rs
    if s[0].dtype is F32:
        return (0, 1, True)
    if ah < bl or bh < al:
        return (1, 1, True)
    return (0, 0, True) if al == ah == bl == bh else (0, 1, True)
if op is Ops.WHERE:
    cl, ch, _ = rs[0]
    if cl == ch == 1:
        return rs[1]
    if cl == ch == 0:
        return rs[2]
    return (min(rs[1][0], rs[2][0]), max(rs[1][1], rs[2][1]), safe)
if op in (Ops.FLOORDIV, Ops.FLOORMOD):
    (al, ah, _), (bl, bh, _) = rs
    if bl == bh and bl != 0:
        # a constant divisor
        c = bl
        if op is Ops.FLOORDIV:
            if c == -1:
                return fit(-ah, -al)
            lo, hi = sorted((al // c, ah // c))
            return (lo, hi, safe)
        if al // c == ah // c:
            # the whole range sits in
one bucket
            k = al // c
            return (al - k * c, ah - k * c, safe)
            return (0, c - 1, safe) if c > 0 else (c + 1, 0, safe)
        if bl > 0 or bh < 0:
            # divisor range excludes
zero
            if op is Ops.FLOORMOD:
                return (0, bh - 1, safe) if bl > 0 else (bl + 1, 0, safe)
            m = max(abs(al), abs(ah))

```

```

    return fit(-m, m)
    return (*FULL, safe)
    raise AssertionError(op)

```

The first function is the recursive property: it walks the graph with `toposort`, skipping nodes whose range is already cached, and fills in the missing ones in order. There is no recursion, so no depth limit.

Why safe matters

The flag exists because of one fact: **a rewrite that is correct in true mathematics may be wrong in wrapped arithmetic**. The rule $(a * 4 + b) // 4 \rightarrow a + b // 4$ is true for all true integers. With $a = 2^{30}$ and $b = 0$, the true value of $a * 4$ is 2^{32} , which wraps to 0 in `int32`. The left side computes $(0 + 0) // 4 = 0$. The right side computes $2^{30} + 0 = 2^{30}$. The rule changed the answer. So chapter 11's rule only fires if `safe` is true for the parts it moves. z3 finds the case on its own: for the guard-less version of that rule it reports $a = 1073741824$, $b = -2147483648$ as an input where the two sides differ (`out/proofs.txt`).

How the ranges were tested

Two tests check the range property directly, apart from the rules that use it.

The first, `test_ranges` in `tests/test_pipeline.py`, builds 500 random programs, runs each on 40 random inputs inside the declared bounds, and evaluates every node in two ways at once: with the contract's wraparound, and with exact integer arithmetic that never wraps. For every int and bool node it checks $\text{vmin} \leq \text{value} \leq \text{vmax}$. And for every node marked `safe`, it checks that the wrapped value and the exact value are equal. 125,600 checks, and 61,840 of them were on nodes marked safe. None failed.

The second is the exhaustive test of chapter 11, which runs every rewritten program on every input of a small domain. It catches a wrong range through a wrong rewrite.

And the planted bugs test the tests. A range rule made one too small, a `FLOORMOD` range made one too small, and a `MUL` that forgets to check overflow are three of the bugs of chapter 15, and each is caught.

The same property in tinygrad

tinygrad's `UOp` has `vmin` and `vmax`, defined in `tinygrad/uop/ops.py`, and its simplifier uses them the way chapter 11 will. Its index math is the main user. Its `tinygrad/uop/validate.py` goes further: it uses z3 to prove that a memory access stays in bounds. This book uses z3 for something different, to prove the rewrite rules themselves (chapter 15).

Where to learn more

- Patrick Cousot and Radhia Cousot, *abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints*, POPL 1977. The paper that founded the area. Interval arithmetic is its simplest instance.
- Anders Møller and Michael Schwartzbach, *static program analysis*, free at cs.au.dk/~amoeller/spa. The chapters on lattice theory and on interval analysis with widening are the clearest short treatment.
- *Engineering a compiler*, chapter 9 on data flow analysis.
- The LLVM source file `llvm/lib/IR/ConstantRange.cpp` implements the same idea as this chapter, with wrapped ranges, for real.
- The tinygrad source: `_min_max` and the `vmin` and `vmax` properties in `tinygrad/uop/ops.py`.

Exercises

1. Give the range of `i*4 + j` and of `(i*4 + j) % 8` for `i` in `[0, 15]` and `j` in `[0, 3]`. Then check by running all 64 inputs.
2. What is the range of `i - j` for `i` in `[0, 15]` and `j` in `[0, 3]`? Build the graph and read `vmin`, `vmax`.
3. For `x` in `[-8, 8]`, give the ranges of `x // 5` and `x % 5` by hand, and check them against all 17 inputs.
4. Build `x * x` for `x` in `[-50000, 50000]`. Is it safe? What is `50000 * 50000`? Repeat for `x` in `[-46340, 46340]`.
5. The range of `x % 1` is a single point. What point? What does the “range is a point” rule of chapter 11 do with it?

Part 3

Rewriting

Chapter 7

Patterns and rules

A **rewrite** replaces part of a program with something that has the same value. It is the one way the compiler changes programs. The next five chapters are about doing it correctly. This chapter is about the first half of the mechanism: how to say where a rewrite applies. Chapter 8 is about the second half: how to apply many rewrites to a whole graph until none applies.

A rule is a pattern and a function

Simplifying `x * 1` to `x` takes two decisions. Does this node look like a multiplication by one? And if so, what should replace it? A **rule** is those two things together: a **pattern**, which describes the nodes the rule applies to, and a **function**, which takes the parts the pattern found and returns the replacement.

Here is the rule `x*1` from `uopc/symbolic.py`:

```
@rule("x*1", UPat(Ops.MUL, dtype=(I32, F32), src=(var("x"), cvar("c"))), SYM)
def _mul_one(x, c):
    if c.arg == 1:
        return x
```

The pattern says: a node whose op is `MUL`, whose dtype is `i32` or `f32`, and which has two srcs, the first anything, which the pattern names `x`, and the second a constant, which it names `c`. The function receives the two nodes the pattern found. If the constant is 1 it returns `x`. If not, it returns nothing, and the rule does not apply.

The split of work is deliberate. The pattern does the cheap structural test, which the matcher can run on any node. The function does the part that needs thought: here a value test, in later rules a range test or arithmetic on constants. A function can return `None` after seeing the arguments, and it does so often.

The pattern

A pattern is an object of the class `UPat` (“uop pattern”). It has five optional fields, and each one that is given must hold for a node to match:

Field	Meaning
<code>op</code>	The node's <code>op</code> is this one, or one of these
<code>dtype</code>	The node's <code>dtype</code> is this one, or one of these
<code>src</code>	The node has exactly this many <code>srcs</code> , and each matches the pattern in its position
<code>arg</code>	The node's <code>arg</code> equals this
<code>name</code>	On a match, give the matched node this name

A pattern with no fields given matches any node. So `UPat.var("x")` is “any node, call it `x`”, and `UPat.cvar("c")` is “a constant node, call it `c`”. A pattern is a tree, and it matches a node by comparing the tree to the top of the graph below that node.

This is the whole of the matching code:

UPat, Rule, rule and PatternMatcher in `uopc/rewrite.py`

```
class UPat:
    """a pattern. it matches one UOp, and every `name` it contains is bound to the
    node it matched.
    the same name used twice must match the same node, which is a pointer compare
    because of
    hash consing."""
    __slots__ = ("op", "dtype", "src", "arg", "name")

    def __init__(self, op=None, dtype=None, src=None, arg=ANY, name=None):
        self.op = None if op is None else (op,) if isinstance(op, Ops) else tuple(op
        )
        self.dtype = None if dtype is None else tuple(dtype) if isinstance(dtype, (t
        uple, list, set)) else (dtype,)
        self.src = None if src is None else tuple(src)
        self.arg, self.name = arg, name

    @staticmethod
    def var(name=None, dtype=None): return UPat(dtype=dtype, name=name)
    @staticmethod
    def cvar(name=None, dtype=None): return UPat(Ops.CONST, dtype=dtype, name=name)

    def match(self, u: UOp, store: dict) -> bool:
        if self.op is not None and u.op not in self.op:
            return False
        if self.dtype is not None and u.dtype not in self.dtype:
            return False
        if self.arg is not ANY and _key_arg(u.arg) != _key_arg(self.arg):
            return False
        if self.src is not None:
            if len(u.src) != len(self.src):
                return False
```

```

        for p, s in zip(self.src, u.src):
            if not p.match(s, store):
                return False
    if self.name is not None:
        if self.name in store:
            return store[self.name] is u
        store[self.name] = u
    return True

@dataclass
class Rule:
    name: str
    pat: UPat
    fn: Callable
    wants_ctx: bool

def rule(name: str, pat: UPat, registry: list):
    """decorator: `@rule("x+0", UPat(...), RULES)` registers the function as a
    rule."""
    def deco(fn):
        registry.append(Rule(name, pat, fn, "ctx" in
            inspect.signature(fn).parameters))
        return fn
    return deco

class PatternMatcher:
    def __init__(self, rules: list[Rule]):
        self.rules = list(rules)
        self.by_op = {op: [r for r in self.rules if r.pat.op is None or op in
            r.pat.op] for op in Ops}
        self.fired: Counter = Counter()

    def __add__(self, other: "PatternMatcher") -> "PatternMatcher":
        return PatternMatcher(self.rules + other.rules)

    def rewrite(self, u: UOp, ctx=None):
        """the first rule that matches and returns a new node wins. None means no
        rule applies."""
        for r in self.by_op[u.op]:
            store: dict = {}
            if r.pat.match(u, store):
                new = r.fn(**store, ctx=ctx) if r.wants_ctx else r.fn(**store)
                if new is not None and new is not u:
                    self.fired[r.name] += 1
                return new
        return None

```

Read `match` from the top. Each given field is checked, and a failed check returns `False` at once. The `srcs` are matched by recursion, in order. At the end, if the pattern has a name, the node is stored under it.

The same name twice

The last lines hold the most important idea in the matcher. If a name is already stored, the node must be **the same node**:

```
if self.name in store:
    return store[self.name] is u
```

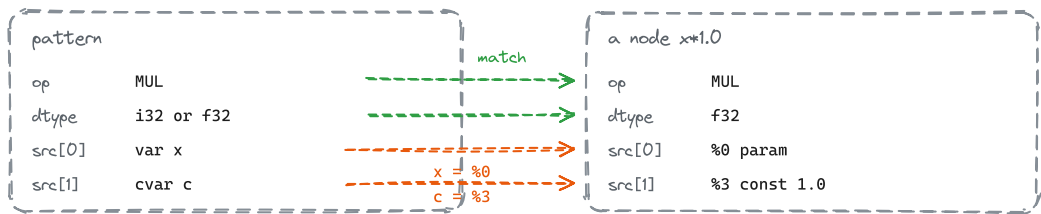
This is how a pattern says “the same value twice”. `ADD(x, x)` matches `a + a` and does not match `a + b`. The check is `is`, a pointer compare, and it is correct because of hash consing: structurally equal nodes are the same node (chapter 5). Here is the matcher at work, from `demos/matcher.py`:

patterns against nodes · output of `demos/matcher.py`

<code>ADD(x, x) against a+a</code>	<code>match</code>	<code>{'x':</code>
<code>'param#0'}</code>		
<code>ADD(x, x) against a+b</code>	<code>no match</code>	
<code>ADD(x, cvar c) against a+5</code>	<code>match</code>	<code>{'x':</code>
<code>'param#0', 'c': 'const'}</code>		
<code>ADD(x, cvar c) against a+b</code>	<code>no match</code>	
<code>ADD(x, cvar c) against a+5 with dtype f32</code>	<code>no match</code>	
<code>ADD(x, cvar c) against f+5.0 with dtype f32</code>	<code>match</code>	<code>{'x':</code>
<code>'param#2', 'c': 'const'}</code>		
<code>MUL(ADD(x, c1), c2) against (a+3)*4</code>	<code>match</code>	<code>{'x':</code>
<code>'param#0', 'c1': 'const', 'c2': 'const'}</code>		
<code>ADD(x, x) against (a+b)+(a+b)</code>	<code>match</code>	<code>{'x': 'add'}</code>
<code>ADD(x, x) against (a+b)+(b+a) (no canonical order yet)</code>	<code>no match</code>	
<code>CONST(arg=7) against CONST 7</code>	<code>match</code>	<code>{'k': 'const'}</code>
<code>CONST(arg=7) against CONST 8</code>	<code>no match</code>	

Look at the three `ADD(x, x)` lines. Against `a+a` it matches and `x` is the parameter. Against `a+b` it does not. Against `(a+b) + (a+b)` it matches, and `x` is the `ADD` node: the two copies of `a+b` in the program are one node, so the name binds. The fourth, against `(a+b) + (b+a)`, does not match: `a+b` and `b+a` are two nodes. The compiler will put them into one form in chapter 10, with a rule called “canonical order” (chapter 8), and then this pattern will match.

In a language without hash consing, matching `x + x` has to compare two subtrees by value, which costs time in proportion to their size. Here it is one comparison.

a pattern against a node

the point:
the pattern tests shape. the function then tests values: here `c.arg == 1`, so it returns `x`.

The matcher

A `PatternMatcher` is a list of rules, plus an index. Its method `rewrite(node)` finds the first rule that applies to a node and returns the replacement, or `None`.

```
def rewrite(self, u, ctx=None):
    for r in self.by_op[u.op]:
        store = {}
        if r.pat.match(u, store):
            new = r.fn(**store, ctx=ctx) if r.wants_ctx else r.fn(**store)
            if new is not None and new is not u:
                self.fired[r.name] += 1
                return new
    return None
```

Three things to see.

The index. `by_op` maps each op to the rules whose top pattern can match it. A rule whose top pattern says `MUL` is never tried on an `ADD`. In the default matcher of this module, 30 rules are loaded, and a node is tried against at most 8 of them. A `CONST` is tried against none:

how many rules each op is tried against · output of `demos/matcher.py`

```
rules in the default matcher: 30 (fold 1, inline 1, symbolic 28); fast math adds 5
a node with op ADD          is tried against 7 rules
a node with op MUL          is tried against 8 rules
a node with op NEG          is tried against 4 rules
a node with op CMPLT        is tried against 3 rules
a node with op FLOORDIV     is tried against 5 rules
a node with op FLOORMOD     is tried against 6 rules
a node with op WHERE        is tried against 6 rules
a node with op CAST         is tried against 3 rules
```

```

a node with op CONST      is tried against 0 rules
a node with op LOAD       is tried against 0 rules
a node with op CALL       is tried against 1 rules
rule names in order: ['const fold', 'inline call', 'canonical order', 'range is a
point', 'max by range', 'x*1', 'x*-1', 'x+(-0.0) (float)'] ...

```

The index cuts the work per node by about a factor of four here, and the factor grows with the number of rules. A compiler with 300 rules benefits more.

The first match wins. The loop returns the first replacement. Rule order is part of the program. In this module the order is: constant folding, then call inlining, then the canonical order, then range rules, then the algebraic rules. Constant folding comes first because a folded constant makes other rules apply. Reordering rules changes which rewrite happens, never whether the result is correct, as long as every rule is correct. The final result of the engine can still depend on the order, and the next chapter shows how.

The function may decline. `new` is not `None` and `new` is not `u`. A rule returns `None` to say “my pattern matched but my condition did not”, and the matcher moves on to the next rule. It also ignores a rule that returns the node it was given: that would be a rewrite that changes nothing, and it would make the engine loop forever.

There is a counter, `fired`, that records how many times each rule produced a rewrite. It is used for tests (a rule that never fires in any test is a rule that is not tested) and to diagnose loops (chapter 8).

`rule` is the decorator that creates a `Rule` and appends it to a list. Matchers are made from lists of rules, and `PatternMatcher.__add__` joins two of them, which is how the fast math rules of chapter 10 are kept in a separate list and added only when asked for.

Writing a rule so that it is correct

A rule is correct if, for every graph that matches, the replacement has the same value as the node, under the contract of chapter 3, on every input. This is easy to say and easy to get wrong. A few habits prevent most mistakes.

Be as narrow as the truth. The rule `x*1` has `dtype=(I32, F32)`. It is not given for `bool`, where multiplication is AND and a separate rule handles it. The pattern says what the rule is known to be true for, no more. If you write the pattern too wide, the guard in the function has to protect you.

Test the guard for the condition you actually need. The rule `x+(-0.0)` for floats is correct. Its neighbour `x+0.0` is not, because `-0.0 + 0.0` is `+0.0`. The function checks `math.copysign(1.0, c.arg) < 0`, the sign bit, and not `c.arg == 0.0`, which is true for both zeros. Chapter 10 shows the line.

Never rely on the order of the checks. A rule can be called on any node that matches its pattern, at any point in the engine. It must be correct alone.

Write the must-not-fire cases. For each rule, there are inputs where it must do nothing. `x+0.0` on a float. `(x//2)//3` on a negative divisor. The unit tests in `tests/test_rules_unit.py` hold 52 hand written cases, and many of them are “this expression must not change”.

Ask a theorem prover. After the rule is written, chapter 15 shows how z3 can be asked to prove it, or to produce the input where it fails.

The same thing in tinygrad

tinygrad calls these the same names: `UPat`, `PatternMatcher`, and `UPat.var` and `UPat.cvar`. They are in `tinygrad/uop/ops.py` (`UPat` and `PatternMatcher`, near the end of the file). A rule there is a pattern paired with a lambda or a function, written as a list of tuples: `(UPat(...), lambda x, c: ...)`. Its `PatternMatcher` also indexes patterns by op (a dictionary called `pdict`), and it also uses the first match. It adds a cheap early test: each pattern records which ops its srcs must have, and a node whose srcs lack them is skipped before the full match runs. The other difference is speed: tinygrad compiles a pattern into Python source (`tinygrad/uop/upat.py`, on by default through the variable `UPAT_COMPILE`), so that matching does not interpret a tree of objects on every node. The `UPat` of this chapter interprets the tree each time. It is easier to read and slower to run.

Where to learn more

- *Engineering a compiler*, the chapters on introduction to optimization and on scalar optimizations. Peephole optimisation and algebraic simplification are the names for what a matcher does.
- The LLVM instcombine pass, in `llvm/lib/Transforms/InstCombine`. The source is large but the file `InstCombineAddSub.cpp` shows the same idea as this chapter at scale. The header `llvm/include/llvm/IR/PatternMatch.h` is the matcher it uses.
- tinygrad’s `UPat` and `PatternMatcher` in `tinygrad/uop/ops.py`, and `tinygrad/uop/upat.py` for the compiled form.
- Nuno lopes, david menendez, santosh nagarakatte and john regehr, *provably correct peephole optimizations with alive*, PLDI 2015. The paper that shows how to state a rewrite rule so that a solver can prove it.
- Nuno lopes and others, *Alive2: bounded translation validation for LLVM*, PLDI 2021.

- The Alive2 project at github.com/AliveToolkit/alive2, a tool that checks LLVM's rewrites for correctness. It handles the same concerns about undefined behaviour that chapter 12 raises.

Exercises

1. Write a pattern that matches `MUL(x, x)` for any dtype, and a function that always returns `None`. Then run the matcher on `a*a` and on `a*b`, and read the store.
2. Write a rule for `max(x, x)`. It is already in `symbolic.py`. Find it and read the pattern. Why does the pattern use the same name twice?
3. What would the pattern `UPat(Ops.ADD, src=(UPat.var("x"), UPat.var("y")))` match? And does it match `a + a`? What is stored for `x` and `y`?
4. Write the rule `x - x -> 0` for ints, as a pattern over the ops of this module (remember that there is no `SUB`). Is it already a rule? Find it in `symbolic.py`.
5. The rule `x*recip(x) -> 1.0` is in the fast math list. Give an `x` for which it is wrong in float32. Check your answer.

Chapter 8

The rewrite engine

A matcher (chapter 7) looks at one node. The engine's job is to apply it to every node in the graph, in an order that makes sense, and to keep going until nothing changes. This chapter builds that engine, shows it working step by step, and then looks at its two hard questions: does it stop, and does the answer depend on the order of the rules?

The goal: a fixed point

The engine takes a root, a matcher and returns a new root. Its contract is:

1. The new graph has the same value as the old one, provided every rule is correct.
2. No rule of the matcher applies to any node of the new graph.

The second condition is called a **fixed point**: rewriting the result again changes nothing. A rewrite engine that reaches a fixed point has finished, in the sense that it has nothing left it knows how to do.

Bottom up

To reach a fixed point it helps to rewrite **bottom up**: a node's srcs are rewritten before the node is. The reason is mechanical. A rule looks at a node and its srcs. If the srcs are in their simplest form when the rule looks, there are fewer cases to match and a rule is more likely to fire on the first look. For example, in $(x * (1 + 1)) * 3$ the rule $(x * c1) * c2 \rightarrow x * (c1 * c2)$ needs the second src of the inner `MUL` to be a constant, and at first it is an `ADD`. Rewriting bottom up, $1 + 1$ becomes 2 before the outer rule looks, so the outer node is $(x * 2) * 3$, which matches at once and becomes $x * 6$. Rewriting the srcs first does the inner work before the outer work needs it.

When a rule fires, its replacement is a new graph that may itself contain things to rewrite. So the engine then rewrites the replacement, bottom up, and what that gives is the final value for the node. This repeats as deep as needed.

The code

Here is the whole engine:

graph_rewrite in uopc/rewrite.py

```

def graph_rewrite(root: UOp, pm: PatternMatcher, ctx=None, limit: int = 1_000_000) -
> UOp:
    """rewrite bottom up: a node's srcs are finished before the node is matched.
    when a rule fires, the
        result is itself rewritten, until no rule applies anywhere. iterative, so deep
    graphs are safe."""
    done: dict[UOp, UOp] = {}
    stack = [(root, 0, None)]
    steps = 0
    while stack:
        node, state, extra = stack.pop()
        if state == 0:
            if node in done:
                continue
            stack.append((node, 1, None))
            for s in reversed(node.src):
                if s not in done:
                    stack.append((s, 0, None))
            elif state == 1:
# all srcs are done: look at the node
                new_src = tuple(done[s] for s in node.src)
                n = node if new_src == node.src else UOp(node.op, new_src, node.arg)
                r = pm.rewrite(n, ctx)
                steps += 1
                if steps > limit:
                    top = pm.fired.most_common(3)
                    raise RewriteLimit(f"more than {limit} rewrite steps; busiest
rules: {top}")
                if r is None:
                    done[node] = n
                    done.setdefault(n, n)
                else:
                    stack.append((node, 2, r)) # after r is finished,
node maps to it
                    stack.append((r, 0, None))
                else: # state 2: r is finished
                    done[node] = done[extra]
    return done[root]

```

It has a stack of work items, each a triple `(node, state, extra)`, and a dictionary `done` that maps each original node to its final rewritten form. The stack makes the traversal iterative, so a graph can be deeper than Python's recursion limit (chapter 4 tested a depth of 200,000). Each node goes through up to three states.

- **State 0, first visit.** If the node is already in `done`, skip it. Otherwise push it back with state 1, then push each src that is not done, with state 0. The srcs are processed first because the stack is last in, first out.
- **State 1, the srcs are finished.** Build a version of the node with the finished srcs, if any changed (`UOp(node.op, new_src, node.arg)`), which is a lookup through the hash cons

table). Ask the matcher. If no rule applies, the node is final: `done[node] = n`. If a rule fired and returned `r`, push `(node, 2, r)` and then `(r, 0)`. The node cannot be finished until `r` is.

- **State 2, the replacement is finished.** `done[node] = done[r]`.

The dictionary is the memo. A node reached from two places is rewritten once, because the second time it is in `done`. This is the property that keeps sharing: the output graph shares what the input graph shared, with no extra work. A graph that has 19 nodes because of sharing is rewritten in time proportional to 19, not 524,287.

There is a step counter. Every time the matcher is asked about a node, the counter goes up. If it passes `limit`, which defaults to a million, the engine raises `RewriteLimit` and says which rules were busiest. A reason is explained below.

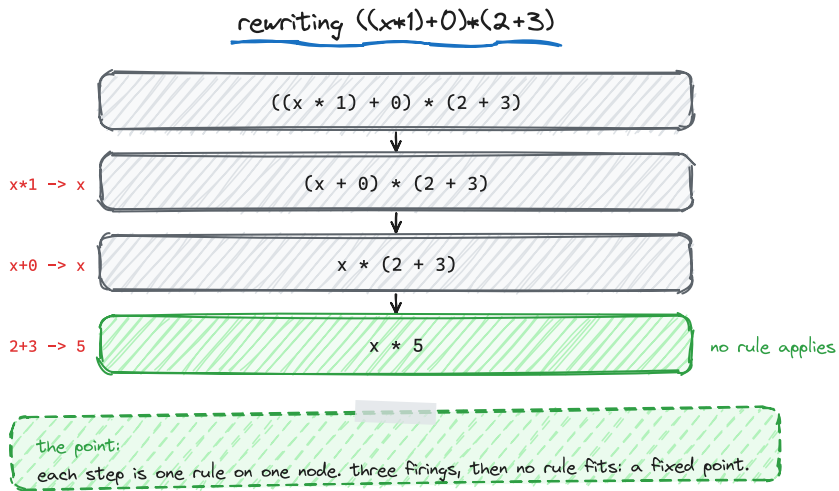
The engine at work

Here is a trace of the engine simplifying `((x*1)+0)*(2+3)` for an int `x`. The program printed each step by wrapping the matcher. This is the real output of `demos/walkthrough.py`:

a rewrite trace · output of `demos/walkthrough.py`

```
start: (((x * 1) + 0) * (2 + 3))
  step 1: (x * 1) -> x
  step 2: (x + 0) -> x
  step 3: (2 + 3) -> 5
result: (x * 5)   total firings by rule: {'x*1': 1, 'x+0 (int)': 1, 'const fold': 1}
```

There are three steps, each one rule. `x*1` becomes `x`. Then the `ADD` has `x` and `0`, so `x + 0` becomes `x`. Then `2 + 3` is a constant fold, which is a rule like any other (chapter 9). The result is `x * 5`. No rule applies to it. The engine is done. The counts at the end confirm it: each of the three rules fired once.



Does it stop?

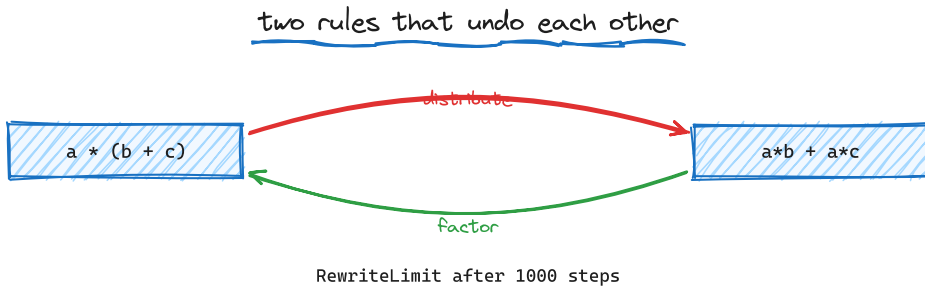
A fixed point exists only if the engine reaches it. An engine can fail to stop. If two rules undo each other, each one's output is the other's input, and the engine alternates forever.

Here is a minimal example, built and run in `demos/run_demos.py`. Rule one distributes: $a * (b + c) \rightarrow a*b + a*c$. Rule two factors: $a*b + a*c \rightarrow a * (b + c)$. Each is correct in true mathematics (for ints, in wrapped arithmetic too). Together they never stop:

two rules that undo each other · output of `demos/run_demos.py`

```
RewriteLimit raised: more than 1000 rewrite steps; busiest rules: [('distribute',
498), ('factor', 497)]
distribute alone terminates: %5 = add %2, %4
```

With the limit set to 1,000 the engine raises `RewriteLimit` and names the two rules, 498 and 497 firings. A bug like this shows up in a compiler as a hang, and the message names the culprit. With only the distribute rule, the engine stops after one step, as the last line shows.



the point:
the engine named the two rules: distribute 498 firings, factor 497, a limit turns a hang into an error.

So the first design rule for a rule set is: **each rule must make progress by some measure the others do not undo.** The measures that this module's rules use:

- Most rules reduce the number of nodes: $x*1 \rightarrow x$, $x+0 \rightarrow x$, constant folding, $\max(x, x) \rightarrow x$.
- The rules that do not shrink the graph move toward a normal form. `canonical_order` swaps the srcs of `ADD`, `MUL`, `COMPNE` and integer `MAX` when they are out of order by `(is_const, id)`. After a swap they are in order, and the rule does not swap back, so it fires at most once per node.
- The rules $(x+c1)*c2 \rightarrow x*c2 + c1*c2$ and the integer division split spread a constant into a sum, and no rule factors a sum back together. There is no `factor` rule in this module for that reason.

These are arguments, and arguments can be wrong. The evidence is the test: `simplify(simplify(x))` must be the same node as `simplify(x)`. A graph that is not a fixed point after one pass fails. The soundness test (`test_rewrites_sound`) checks this on each of its 700 random programs, and the exhaustive test checks the results on every input of a small domain. The limit then protects the user in case a rule combination is missed.

Does the answer depend on the order?

The engine finds a fixed point. Is there only one? Run the experiment: take 600 random index programs, simplify each with the rules in the default order, then simplify again with the order reversed. Both orders are correct, which the program checks on sampled inputs. Do they give the same graph?

the same programs with two rule orders · output of demos/order.py

```
600 random index programs, each simplified with the default rule order and with the
reverse order
  both results correct on 74 sampled inputs each: yes
  same result (same node): 594    different result: 6
  of the different ones, the default order gave the smaller graph 1 times, the
reverse gave the smaller graph 0 times, equal size 5
  total nodes: default 4210, reversed 4211
an example (seed 551): 26 nodes
  start:  where((neg((((j + (((k + i) // 4) + i) % 16)) % 5) * -3)) != ((((((j +
k) + (j + k)) + 2) + j) // 1) * 6)), neg((((j + (((k + i) // 4) + i) % 16)) % 5) *
-3)), ((((((j + k) + (j + k)) + 2) + j) // 1) * 6))
  default: where((((j + ((i + ((i + k) // 4)) % 16)) % 5) * 3) != ((j + (((j + k) +
(j + k)) + 2)) * 6)), (((j + ((i + ((i + k) // 4)) % 16)) % 5) * 3), ((j + (((j + k)
+ (j + k)) + 2)) * 6))    (23 nodes)
  reverse: where((((j + ((i + ((i + k) // 4)) % 16)) % 5) * 3) != (((k + (j + (k +
(j + j)))) * 6) + 12)), (((j + ((i + ((i + k) // 4)) % 16)) % 5) * 3), (((k + (j +
(k + (j + j)))) * 6) + 12))    (24 nodes)
```

594 of the 600 are the same node. Six are not, and in those six the sizes differ by one node at most. Look at the example, seed 551. Both results are correct. They differ in how a constant was merged: the default order gave $(j + \dots) * 6$ and the reverse order gave $(\dots * 6) + 12$. Exercise 8.4 counts how often each rule fired in the two runs. Two rules, the general split $(a*c1+b)//c2$ and the distribute rule $(x+c1)*c2$, fired only in the reverse order, and that is where the different grouping and the extra $+ 12$ come from.

So this rule set is not **confluent**: the end result can depend on the order in which rules fire. This is normal, and it is the usual state of an optimiser. There are two consequences.

- The output of the compiler is not guaranteed to be the best form. It is a correct form. A program is not made worse by a different order in this experiment (the default had the smaller graph once and the sizes were equal five times), but nothing proves that it cannot be made worse.
- The rule order is part of the compiler. It should be written down, fixed, and tested, as it is here: the order in `symbolic.py` is the order of the list.

The study of this problem has a name, the **phase ordering problem**: an optimisation can enable or disable another one, so the order of passes changes the result.

A way around it: equality saturation

There is a different design that does not have this problem. Instead of replacing a node by a rewritten one, you keep **both**. A data structure called an **e-graph** stores, for each value, the set of all the equal forms found so far. A rewrite adds a new form to the set and removes

nothing. You apply all rules everywhere until no rule produces anything new, or until a size limit. Only then do you choose the best form with a cost function. Every order of rules reaches the same set, so the order cannot matter.

The idea is from ross tate, michael stepp, zachary tatlock and sorin lerner, *equality saturation: a new approach to optimization* (popl 2009, and the journal version in logical methods in computer science 7(1), 2011, at arxiv.org/abs/1012.1802). A fast implementation of the data structure is **egg**, by max willsey and others, *egg: fast and extensible equality saturation* (popl 2021).

It is not the design of this module, and it is not tinygrad's design either. tinygrad uses the direct kind of engine of this chapter, a pattern matcher and a graph rewrite that replaces nodes. The reasons are practical. An e-graph can grow very large when rules like distribute and associate apply to each other, a compile must finish in bounded time, and choosing the best form needs a cost model the compiler does not yet have in this course. You do not need it for the first module. You should know it exists, because it is the principled answer to the question in this section, and it is the best next thing to read if this section made you curious.

The same in tinygrad

`graph_rewrite` in `tinygrad/uop/ops.py` does the same job with more options. Its signature is `graph_rewrite(sink, pm, ctx=None, bottom_up=False, name=None, bpm=None, walk=False, enter_calls=False)`. The matcher `pm` is applied after a node's `srcs` are rewritten, as in this chapter. A second matcher `bpm` can be given and is tried on the way down, before the `srcs`. There is a safety limit on the size of the rewrite stack, a variable called `REWRITE_STACK_LIMIT`, set to 250,000 in the version read for this book, which guards against runaway rewriting as `limit` does here. The traversal uses an explicit stack, as here.

Where to learn more

- The three papers above. Start with the egg paper: it explains e-graphs with pictures and examples. It is the easiest of the three to read.
- The egg website, at egraphs-good.github.io, and its tutorial.
- *Term rewriting and all that* by franz baader and tobias nipkow. The standard book on rewriting systems: termination, confluence and what each means. Chapter 2 is the place to start.
- Nachum dershowitz, *termination of rewriting*, journal of symbolic computation, 1987. The classic result on proving termination with an ordering.

Exercises

1. Trace the engine by hand on $(a + 0) * 1$ for an int a . List the steps the way the trace above does. Check with the code.
2. Build $x + x$ for an int x where x is $(p + 0)$ and p is a param. How many times is $p + 0$ rewritten? Why?
3. Write the two rules `distribute` and `factor` from the demo and run the engine with `limit=50`. What does the message say?
4. Find a program in the experiment where the two orders differ. You can use seed 551. Which rule fired in only one order?
5. Construct a rule set of two rules where order matters and both orders terminate with different, correct results. Hint: one rule can rewrite $x*2$ into $x+x$, and another can rewrite $x+x$ into $x*2$. What goes wrong when both are present?

Chapter 9

Running programs by folding

The syllabus asks for a rewrite engine “capable of running these programs via const folding”. It is a plain sentence with a large consequence: the compiler does not need a separate interpreter. Running a program is a special case of simplifying it. This chapter shows the two rules that make it work, and then checks the claim with a test.

Constant folding

An op whose inputs are all constants has a value that can be computed now. So replace the op with a constant of that value. This is **constant folding**, and it is one rule:

uopc/fold.py, the whole file: constant folding, argument substitution and call inlining

```
"""running programs by rewriting: constant folding, and inlining a CALL."""
from .dtypes import VALUE_DTYPES, DType
from .ops import ALU, Ops
from .rewrite import PatternMatcher, Rule, UPat, graph_rewrite, rule
from .semantics import eval_op
from .uop import ALU_SPACE, UOp

FOLD: list[Rule] = []

@rule("const fold", UPat(tuple(ALU), name="u"), FOLD)
def _const_fold(u):
    """an ALU op whose srcs are all constants is a constant. eval_op is the one
    place that knows
    what each op means."""
    if all(s.op is Ops.CONST for s in u.src):
        return UOp.const(eval_op(u.op, u.dtype, [s.arg for s in u.src]))

_substitute = PatternMatcher([])
SUBST: list[Rule] = []

@rule("param -> argument", UPat(Ops.PARAM, name="p"), SUBST)
def _param_to_arg(p, ctx):
    if isinstance(p.dtype, DType) and p.arg.slot < len(ctx):
        return ctx[p.arg.slot]

_substitute = PatternMatcher(SUBST)

INLINE: list[Rule] = []
```

```

@rule("inline call", UPat(Ops.CALL, name="c"), INLINE)
def _inline_call(c):

    """CALL(body, a0, a1, ...) where the body computes a value from scalars: replace
    every
        PARAM(i) in the body with a_i. the result is an ordinary expression."""
    args = c.src[1:]
    if c.dtype in VALUE_DTYPES and all(a.space == ALU_SPACE and isinstance(a.dtype,
DType) for a in args):
        return graph_rewrite(c.src[0], _substitute, ctx=args)

fold = PatternMatcher(FOLD + INLINE)

```

The first rule, `const fold`, has a pattern that matches any ALU op (the pattern lists every op in the `ALU` set). Its function checks that every `src` is a `CONST`. If so it asks `eval_op`, the contract of chapter 3, for the value, and builds a constant with it. The `dtype` comes from the node itself, so a `CMPLT` of two ints folds to a bool constant, and a cast folds to a constant of the target type.

This rule is where the contract and the engine meet. If `eval_op` is wrong, constant folding is wrong, and a wrong rule shows up as a program that gives a different answer when it happens to have a constant in it. Chapter 3 explained how `eval_op` was checked.

Folding applies bottom up (chapter 8). In $((2 + 3) * 4) + 1$ the inner sum folds first, then the product sees two constants and folds, then the outer sum does too. One pass folds the whole expression.

Running a call

A `CALL` (chapter 4) has a body and arguments. It means: the body, with each `PARAM` replaced by the matching argument. So to run a call, perform that replacement. This is the second rule.

```

@rule("inline call", UPat(Ops.CALL, name="c"), INLINE)
def _inline_call(c):
    args = c.src[1:]
    if c.dtype in VALUE_DTYPES and all(a.space == ALU_SPACE and isinstance(a.dtype,
DType) for a in args):
        return graph_rewrite(c.src[0], _substitute, ctx=args)

```

It uses a second, tiny rewrite: `_substitute` has one rule, `param -> argument`, which replaces a `PARAM` of slot `i` by `ctx[i]`, where `ctx` is the list of arguments. The call to `graph_rewrite` inside a rule is a fresh run of the engine on the body with that rule only.

The result is the body with its parameters replaced. It is an ordinary expression. The outer engine then rewrites it like any other, and if every argument was a constant, constant folding collapses it to a single constant.

The guard in the function limits the rule to calls that compute a **value** (a `bool`, `i32` or `f32`) from arguments that are values in `ALU` space. A call that writes memory is not inlined here. Chapter 12 renders it as a function, and chapter 14 builds such calls.

The operation of replacing the parameters of a function by its arguments is called **beta reduction** in the lambda calculus, and here it is a graph rewrite. The engine's memo keeps it cheap: an argument used in three places is substituted into three places, but all three are the same node.

Here is the program from chapter 1, run in this way, with four inputs. This is the output of `demofold.py`:

running relu by rewriting · output of demofold.py

```
relu(2.0, 3.0) -> CONST 7.0
relu(-2.0, 3.0) -> CONST 0.0
relu(nan, 3.0) -> CONST 0.0
relu(inf, 0.0) -> CONST 0.0
```

`relu(2, 3)` is `7.0` and `relu(-2, 3)` is `0.0`. The third line is the answer to exercise 3 of chapter 1: `max(nan, 0)` with the contract's `a > b ? a : b` is `0.0`, because `nan > 0` is false and the answer is the second argument. The fourth is a similar case: `inf * 0.0 + 1` is `nan`, so the result is again `0.0`.

So one engine, two rules, and the compiler can run programs. There is no interpreter in the compiler. There is a **reference interpreter** in `uopc/interp.py`, but it exists for tests. It shares only the contract (`eval_op`) with the rewrite rules, and no rendering code with the renderer (both use `toposort`), so it can be used to check both.

running a call is substitution, then folding



the point:
inlining a call is beta reduction. folding is the interpreter run by rewrite rules.

Partial evaluation

The same two rules do something more useful than running a complete program. If only some of the arguments are constants, the call is inlined with those constants in place, and folding does what it can. The rest of the program remains, specialised.

a call with one known argument · output of `demos/fold.py`

```
relu(z, 3.0) inlines to: max(((z * 3.0) + 1.0), 0.0)    nodes: 7
; uop v1
%0 = param : dtype=f32 slot=0
%1 = const : dtype=f32 value=3.0
%2 = mul %0, %1
%3 = const : dtype=f32 value=1.0
%4 = add %2, %3
%5 = const : dtype=f32 value=0.0
%6 = max %4, %5

relu(z, 0.0): the strict rules keep z*0.0 (it is nan for z = inf), so 6 nodes
remain, including a MUL
g(2, 1) with g = i*4+j: CONST 9
```

`relu(z, 3.0)` with `z` unknown inlines to `max((z * 3.0) + 1.0, 0.0)`. The 3.0 is now part of the program. This is **partial evaluation** or specialisation, and a compiler for machine learning uses it all the time: the shape of a tensor is known when the model is compiled, so every loop bound and every index that depends on a shape is a constant, and folding simplifies the index math (chapter 11).

The output also shows a limit. With the second argument 0.0, the product `z * 0.0` stays. The strict rules of chapter 10 do not remove it, because for `z = inf` or `nan` the product is `nan`, not `0.0`. Only the fast math rules would remove it, and they would change the answer for some inputs.

The check

The claim “a call with constant arguments folds to a constant, and that constant is what an interpreter computes” is tested in `test_fold_run`: for 400 random programs, with 20 sets of input values each, 40% of them drawn from the special values (NaN, infinities, zeros, extreme ints), the test builds a `CALL` with the inputs as constant arguments, runs the engine with `fold`, and checks two things. The result is a single `CONST` node, and its value is the same, bit for bit, as the interpreter’s. 16,000 checks pass.

The test has teeth. Two bugs were planted, one at a time, to see that it fails: constant folding that reverses the operands of every op, and inlining that picks the wrong argument. Both are caught by this test. Chapter 15 lists them with the others.

Constant folding is not free of danger

Two things can go wrong with folding even when `eval_op` is right.

Folding changes when an error happens. In a language where division by zero raises an error, folding `1 // 0` at compile time raises it at compile time, and not folding it raises it at run time, or never, if the code is never reached. In this contract there are no errors (chapter 3), so folding cannot move one. This is a benefit of totality that is easy to take for granted. `tinygrad` is less lucky: it treats a division by a constant zero as an error, in its rule for integer division (`tinygrad/uop/divandmod.py` raises `ZeroDivisionError`), so there the compile can fail on a graph that never executes the division.

Folding must use the machine's arithmetic, not the host's. Constant folding runs in Python, with Python numbers, and the generated code runs on the machine with machine numbers. If the two differ in any bit, a program that has a constant gives a different answer from one that has a variable with the same value. This is why `semantics.py` rounds floats to 32 bits after every operation, wraps ints, and why the tests compare it with real C. The one exception is `EXP2` and `LOG2`, which can differ by 1 ulp (chapter 3).

The same in `tinygrad`

`tinygrad` does the same with the same two ingredients. `exec_alu` in `tinygrad/uop/ops.py` computes an op on Python values using a table called `python_alu`, which holds one Python function per op. `fold_const_alu` in `tinygrad/uop/symbolic.py` is the rule that calls it. Substitution of arguments for parameters is handled in `ops.py` by a pattern matcher whose rule looks up `PARAM` nodes in a context, the same design as `_substitute` here.

Where to learn more

- *Engineering a compiler*, the sections on constant propagation and on algebraic simplification, in the chapter on scalar optimizations.
- *Types and programming languages* by benjamin pierce. Chapters 5 to 7 introduce the lambda calculus, substitution and beta reduction carefully. You do not need it for this module, but it explains the term.
- The *compiler explorer* at godbolt.org. Type a small C function with constants into it and look at the assembly at `-O0` and `-O2` to watch clang fold constants.
- *Partial evaluation and automatic program generation* by neil jones, carsten gomard and peter sestoft, free at itu.dk/people/sestoft/pebook. The standard book on the idea of specialising a program to part of its input.
- The `tinygrad` source: `exec_alu`, `python_alu` and `fold_const_alu`.

Exercises

1. Fold $(2 + 3) * (10 // 4)$ by hand and with the engine. Which node folds first?
2. Run `relu(x, 3.0)` with $x = 1.0$ by calling `graph_rewrite` on a `CALL` with constants. Verify with the C program of chapter 1 (the demo compiles it).
3. Build a body $x // y$ for ints and call it with $y = 0$ as a constant. What is the result? Why does it not raise an error?
4. Build $x * 0$ for an int x with a known range $[0, 100]$ and for a float x . Simplify both. Which one folds to 0 ? Explain with chapter 10.
5. The guard of the inline rule excludes calls that write memory. Build a call whose body is a store (read chapter 14 first) and check that the engine leaves it alone.

Chapter 10

Simplifying floats

The obvious algebra rules are `x*1 = x`, `x+0 = x`, `x*0 = 0`, `(x+y)+z = x+(y+z)`. In school mathematics all four are true. In float32 arithmetic, three of them are false. This chapter goes through each rule and says which are valid, with a proof or an input that breaks them, and then shows how the compiler keeps the risky ones apart.

The rule for the default rules

The default rule set, called the **strict** set, has one requirement: **it never changes the result of a program by even one bit, for any input**. That includes NaN, the infinities and the two zeros. A program that is compiled with the strict set gives the same answers as the same program with no optimisation.

This is the requirement under which the compiler can be trusted without testing every program. It is also a strong one, and the price is that some common simplifications are not allowed.

A second set, the **fast math** set, is allowed to change results. It lives in a separate list, `FAST`, and is added to the matcher only when the caller asks (`simplify(u, fast_math=True)`). It is for people who have decided that a different answer in a corner case is acceptable.

The rules, one by one

Each row below was checked by z3, a theorem prover, over **every** 32 bit float, using its model of IEEE arithmetic. “proved” means z3 showed there is no input where the two sides differ in their bit pattern (with all NaNs counted as the same value). “refuted” means z3 returned an input where they differ. The proofs are in `proofs.py` and the output is in `out/proofs.txt`.

Rewrite	Valid?	Why
$x * 1.0 \rightarrow x$	Proved	Multiplying by 1 is exact
$x + (-0.0) \rightarrow x$	Proved	-0.0 is the identity of float addition
$x + 0.0 \rightarrow x$	Refuted at $x = -0.0$	$-0.0 + 0.0$ is $+0.0$
$x * 0.0 \rightarrow 0.0$	Refuted at $x = -\text{inf}$ and NaN	$\text{inf} * 0$ is NaN , and $-3 * 0.0$ is -0.0
$(x+y)+z \rightarrow x+(y+z)$	Refuted	Rounding happens in a different place
$x * (1/x) \rightarrow 1.0$	Refuted at NaN	And for finite x , the product can round to a neighbour of 1
$x + (-x) \rightarrow 0.0$	Refuted at $+\text{inf}$	$\text{inf} - \text{inf}$ is NaN
$-(-x) \rightarrow x$	Proved	Negation flips the sign bit twice
$x * -1.0 \rightarrow -x$	Proved	Multiplying by -1 is exact, and flips the sign bit
$-(x*y) \rightarrow x*(-y)$	Proved	Rounding to nearest even is symmetric about zero
$\text{max}(x, x) \rightarrow x$	Proved	For NaN , $\text{nan} > \text{nan}$ is false, so the answer is the second argument, nan
$\text{max}(x, y) \rightarrow \text{max}(y, x)$	Refuted at $(+0.0, -0.0)$	The ternary returns the second argument when neither is greater
$x + y \rightarrow y + x$	Proved	Addition is commutative, even in floats
$x * y \rightarrow y * x$	Proved	Multiplication is commutative, even in floats

float rewrites, checked on every float32

proved: same bits for every input

$x * 1.0 \rightarrow x$
$x + (-0.0) \rightarrow x$
$-(-x) \rightarrow x$
$x * -1.0 \rightarrow -x$
$-(x*y) \rightarrow x*(-y)$
$\max(x, x) \rightarrow x$
$x + y \rightarrow y + x$
$x * y \rightarrow y * x$

refuted: z3 found an input

$x + 0.0 \rightarrow x$	$x = -0.0$
$x * 0.0 \rightarrow 0.0$	$x = -\text{inf}, \text{nan}$
$(x+y)+z \rightarrow x+(y+z)$	regroups
$x * (1/x) \rightarrow 1.0$	$x = \text{nan}$
$x + (-x) \rightarrow 0.0$	$x = +\text{inf}$
$\max(x, y) \rightarrow \max(y, x)$	$(+0.0, -0.0)$

the point:

the strict rule set keeps only the left column. the right column is wrong on some input.

The proved rows are the strict set. The refuted rows are what is left out of it, or put in the fast math set. Let us look at each refutation, because each teaches something.

$x + 0.0$. The two zeros are different values. The sum of -0.0 and $+0.0$ is $+0.0$ under round to nearest. So the sum $x + 0.0$ turns -0.0 into $+0.0$. Rewriting it to x leaves -0.0 there. The two print the same and compare equal, but if the next step is $1 / x$, the rewritten program gives $-\text{inf}$ where the original gives $+\text{inf}$. The right identity of float addition is -0.0 , and that rule is in the strict set. Output of `demos/run_demos.py` shows both facts, using the simplifier:

float rules that look free and are not · output of `demos/run_demos.py`

```

x + 0.0      simplify gives unchanged
x + (-0.0)   simplify gives x
x * 1.0      simplify gives x
x * 0.0      simplify gives unchanged
  at x = -0.0: x+0.0 -> 0.0   x*0.0 -> -0.0
  at x = inf:  x+0.0 -> inf    x*0.0 -> nan
  at x = nan:  x+0.0 -> nan    x*0.0 -> nan

```

$x + 0.0$ is left unchanged. $x + (-0.0)$ becomes x . $x * 1.0$ becomes x . $x * 0.0$ is left unchanged. The last lines show the inputs: at $x = -0.0$, $x + 0.0$ is 0.0 ; at $x = \text{inf}$, $x * 0.0$ is nan .

$(x+y)+z$. Chapter 2 gave the example: with $1e8$, $-1e8$ and 1 , one grouping gives 1.0 and the other gives 0.0 . z3 found a different example on its own, with huge values. Any rule that regroupes floats is unsafe, which rules out many familiar rewrites: collecting terms,

distributing a product over a sum, and merging constants across an addition. In the integer rules of chapter 11, $(x + c1) + c2 \rightarrow x + (c1 + c2)$ exists and is proved. There is no float version in the strict set.

`max`. The contract defines `max(a, b)` as `a > b ? a : b`. With `a = -0.0` and `b = +0.0`, neither is greater, so the answer is `b`, and swapping the arguments gives the other zero. With a NaN, the same thing happens. Therefore the rule `canonical order`, which swaps the srcs of `ADD`, `MUL`, `CMPNE` and `MAX` so that equal expressions have one form, must **not** swap a float `MAX`. Integer `MAX` is fine. This is a line in the code:

the float rules, and one fast math rule · `uopc/symbolic.py`

```
@rule("canonical order", UPat((Ops.ADD, Ops.MUL, Ops.CMPNE, Ops.MAX), name="u"),
SYM)
def _canonical_order(u):
    """put constants on the right, and otherwise the older node first, so a+b and
b+a become the same
    node. float MAX is left alone: with NaN or signed zeros, max(a,b) and max(b,a)
can differ."""
    a, b = u.src
    if u.op is Ops.MAX and u.dtype is F32:
        return None
    if (a.op is Ops.CONST, a.id) > (b.op is Ops.CONST, b.id):
        return UOp(u.op, (b, a))

@rule("x*1", UPat(Ops.MUL, dtype=(I32, F32), src=(var("x"), cvar("c"))), SYM)
def _mul_one(x, c):
    if c.arg == 1:
        return x

@rule("x*-1", UPat(Ops.MUL, dtype=(I32, F32), src=(var("x"), cvar("c"))), SYM)
def _mul_minus_one(x, c):
    if c.arg == -1:
        return -x

@rule("x+(-0.0) (float)", UPat(Ops.ADD, dtype=FLT, src=(var("x"), cvar("c"))), SYM)
def _add_neg_zero(x, c):
    """-0.0 is the identity of float addition. +0.0 is not: -0.0 + 0.0 is +0.0."""
    if c.arg == 0.0 and math.copysign(1.0, c.arg) < 0:
        return x

@rule("x+0.0 (float)", UPat(Ops.ADD, dtype=FLT, src=(var("x"), cvar("c"))), FAST)
def _fast_add_zero(x, c):
    if c.arg == 0.0:
        return x
```

Read `_canonical_order`: if `u.op` is `Ops.MAX` and `u.dtype` is `F32`: `return None`. The line was planted as a bug in the test of the tests, chapter 15: with the line removed, the suite fails at an input that holds `nan`.

`_add_neg_zero` is the rule `x + (-0.0)`. Its guard is `math.copysign(1.0, c.arg) < 0`, which reads the sign bit. An ordinary `c.arg == 0.0` would be true for both zeros, and the rule would be the unsafe `x + 0.0`. `_fast_add_zero` at the bottom is that unsafe rule, kept in the fast math list.

What is in the fast math set

Five rules, each refuted above:

Rule	What it gives up
<code>x * 0.0 -> 0.0</code>	NaN and infinity inputs, and the sign of zero
<code>x + 0.0 -> x</code>	The sign of zero
<code>(x*c1)*c2 -> x*(c1*c2)</code>	The last bit, from regrouping
<code>x * recip(x) -> 1.0</code>	NaN, zero, infinity, and the last bit
<code>x + (-x) -> 0.0</code>	NaN and infinity inputs

A test checks that the fast math set is really different from the strict one.

`test_fast_math_unsound` takes random programs, simplifies each in fast math mode, and looks for inputs where the result changes. **It is supposed to find some.** It does: 7 counterexamples, the first at the inputs `{0: 2147483646, 1: 1.0, 2: True}` where the original gives `-0.0` and the fast math version gives `0.0`. A test that expects a failure checks that the failing set is not accidentally the same as the safe one. If it found none, the test itself would be broken, and it checks for that: it asserts the count is greater than zero.

The cost of being strict

How much does strictness lose? For the programs in this module, very little. The strict set still folds every constant, removes `x*1`, `-(-x)` and `x+(-0.0)`, rewrites `x*-1` to `-x`, merges `max(x, x)`, reorders commutative ops, and prunes dead branches of `WHERE`. What it will not do is the algebra that makes numeric code faster by changing its answers. For a machine learning compiler that is a real trade, and the fast math set is where a user decides to make it.

The conclusion for a compiler is short. **A rule for floats has to be proved or tested against the special values, and the special values are not rare.** The test suite draws 40% of its float inputs from a list that holds both zeros, both infinities, NaN, the extreme normal numbers and the first subnormal, because a bug that appears only at `-0.0` will never be found by random numbers.

The same in tinygrad

tinygrad does not keep the two sets apart. On a float variable with no range, its simplifier turns `f + 0.0` into `f`, `f * 0.0` into `0.0`, `f - f` into `0.0` and `(f*3.0)*5.0` into `f*15.0`, which are the rules of this chapter's fast math set. Chapter 16 shows the output and the measured effect on a signed zero. The book keeps the strict set as the default because every rule in it can be proved, and the proof is what makes the tests of chapter 15 meaningful. tinygrad's choice is also reasonable, and the difference is worth knowing before reading its rules.

Where to learn more

- David goldberg, *what every computer scientist should know about floating-point arithmetic*. The section on “optimizers” explains why compilers cannot assume algebraic identities, and uses `x + 0` style examples.
- William kahan's notes at people.eecs.berkeley.edu/~wkahan, in particular *how futile are mindless assessments of roundoff in floating-point computation?* And *branch cuts for complex elementary functions*. Kahan is the principal designer of IEEE 754.
- The gcc manual's description of `-ffast-math` and the flags it turns on, and the clang equivalent, show what a production compiler gives up. Search “gcc optimize options ffast-math”.
- Jean-michel muller and others, *handbook of floating-point arithmetic*, for the formal results on exact operations (multiplying by a power of two, adding zero).
- The z3 documentation on the floating point theory, at microsoft.github.io/z3guide. Read the section on floating point to write your own proofs of rules.

Exercises

1. Use z3 (`proofs.py` has the pattern) to check `x * 2.0 -> x + x` in float32. Is it proved or refuted?
2. Find a float32 pair `x`, `y` for which `-(x - y) != (y - x)` in bits. If you cannot, prove that they are always equal.

3. Is $x * x \geq 0$ always true for floats? What does the answer mean for a rule that wants to remove an absolute value after a square?
4. The strict set has no rule for $x * 3.0 * 5.0 \rightarrow x * 15.0$. Give an x where the two disagree in float32. Then ask z3 the same question for $x * 2.0 * 3.0 \rightarrow x * 6.0$, and explain the difference.
5. Run `simplify(x + 0.0)` and `simplify(x + 0.0, fast_math=True)` on a float param. Read the difference.

Chapter 11

Simplifying integers

Integers are where the symbolic part of the compiler earns its keep. Every address a program touches is computed with integer arithmetic, and the integer expressions that a compiler builds for loops and arrays are full of redundancy. In a later module, a tensor of shape 4 by 4 that is read in a different order produces index expressions like `(i*4 + j) // 4` and `(i*4 + j) % 4`. The programmer means `i` and `j`. The compiler needs to see that.

Unlike floats, integer rules can be given freely: `+` and `*` on `int32` wrap, which makes them an exact algebraic structure (a ring, modulo 2^{32}), so `(x + c1) + c2 = x + (c1 + c2)` is simply true. The care that is needed is about two things: division, which is not a ring operation, and the facts the compiler knows about ranges, which are true for mathematical integers and so can fail when a value wraps.

The rules

Here are the integer rules in the order they appear in `symbolic.py`. Each is stated, with the evidence for it. “proved” means `z3` proved it for every 32 bit input and for the listed constants. The proofs are in `proofs.py`.

Ranges first. The two rules at the top of the list use the range property of chapter 6.

- *Range is a point.* If `vmin == vmax`, the node has one possible value, so replace it by a constant. `x % 1` has the range `[0, 0]`. `j < 4` with `j` in `[0, 3]` has the range `[1, 1]` and becomes `True`. `(i+k) * 0` becomes `0`.
- *Max by range.* If one side of an integer `max` is never smaller than the other, return it. `max(j, 4)` for `j` in `[0, 3]` is `4`.

The correctness of both rests on the range property being right, which chapter 6 tested directly.

Constants. These three are proved (for four pairs of constants each, in exact `int32` arithmetic):

- `(x + c1) + c2 -> x + (c1 + c2)`
- `(x * c1) * c2 -> x * (c1 * c2)`
- `(x + c1) * c2 -> x * c2 + c1 * c2`

And the constant on the right side is computed with `wrap32`, so that it is an `int32`. With plain Python arithmetic the sum could be out of range, and building the constant would raise an error. This is the case of the 200,000 chained additions in chapter 4: the merged constant was 20,000,100,000, wrapped to `-1,474,736,480`.

The same group has `x + 0 -> x`, `x * 1 -> x`, `x * -1 -> -x`, `-(-x) -> x`, `-(x * c) -> x * (-c)`, `x + (-x) -> 0`, `max(x, x) -> x` and `x < x -> False`. None of them needs a guard for ints.

Trivial divisors. `x // 1` is `x` and `x % 1` is `0`. `x // -1` is `-x` and `x % -1` is `0`, even for `INT_MIN`: the contract says `INT_MIN // -1` wraps to `INT_MIN`, and `-INT_MIN` wraps to `INT_MIN`. `x // 0` is `0` and `x % 0` is `x`. `z3` proved all of these in one step each, with the edge case at `INT_MIN`.

Division of a division. `(x // c1) // c2 -> x // (c1 * c2)` is true for positive constants. `z3` proved it for four constant pairs (2 and 3, 4 and 4, 3 and 5, 8 and 2). The rule in the code checks `c1 > 0`, `c2 > 0` and `c1 * c2 <= INT_MAX`, because a product that does not fit in an `int32` is not a valid constant. The rule is narrower than the truth, on purpose: a rule with fewer cases has fewer places to be wrong. With a negative `c1` the rule is not applied.

five integer rules · `uopc/symbolic.py`

```
@rule("range is a point", UPat(tuple(ALU), dtype=(I32, BOOL), name="u"), SYM)
def _range_point(u):
    if u.vmin == u.vmax:
        return const(bool(u.vmin) if u.dtype is BOOL else u.vmin)

@rule("max by range", UPat(Ops.MAX, dtype=(I32, BOOL), src=(var("a"), var("b"))), SYM)
def _max_range(a, b):
    if a.vmin >= b.vmax:
        return a
    if b.vmin >= a.vmax:
        return b

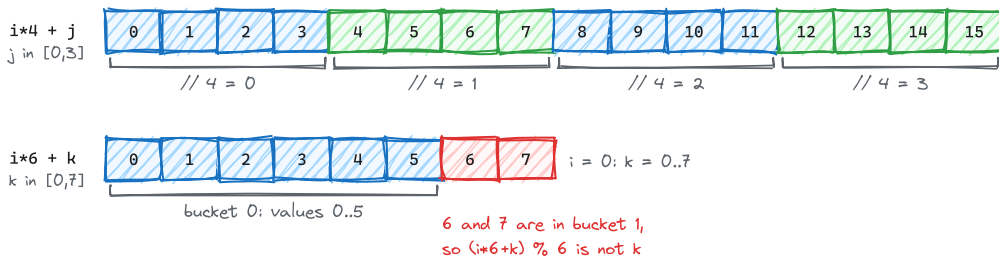
@rule("(x+c1)+c2", UPat(Ops.ADD, dtype=INT, src=(UPat(Ops.ADD, src=(var("x"), cvar("c1"))), cvar("c2"))), SYM)
def _add_add_const(x, c1, c2):
    return x + const(wrap32(c1.arg + c2.arg))

@rule("x%c inside one bucket", UPat(Ops.FLOORMOD, dtype=INT, src=(var("x"), cvar("c"))), SYM)
def _mod_bucket(x, c):
    if c.arg != 0 and x.vmin // c.arg == x.vmax // c.arg:
        k = x.vmin // c.arg
        return x if k == 0 else x + const(wrap32(-k * c.arg))
```

```
@rule("(x//c1)//c2", UPat(Ops.FLOORDIV, dtype=INT, src=(UPat(Ops.FLOORDIV,
src=(var("x"), cvar("c1"))),
                                cvar("c2"))), SYM)

def _div_div(x, c1, c2):
    if c1.arg > 0 and c2.arg > 0 and c1.arg * c2.arg <= INT_MAX:
        return x // const(c1.arg * c2.arg)
```

when does the split of $x // c$ work



the point:

$(i+4+j)/4$ is i only because j stays inside one bucket of 4, only $i = 0$ to 3 are drawn.
with k up to 7, a bucket of 6 is not enough.

A remainder inside one bucket. The rule `_mod_bucket` is the one that takes a range and makes a remainder disappear. $x \% c$ is $x - c*k$ where $k = x // c$. If the whole range of x is inside one block of c consecutive numbers, then k is the same for every value of x , and the remainder is just a subtraction. For x in $[8, 11]$ and $c = 4$, $k = 2$ always, so $x \% 4 = x - 8$. In particular when the range starts at zero, $k = 0$ and $x \% c$ is x . z3 proved the statement for three range and divisor pairs, including a range of negative numbers.

The divisible-terms split

This is the rule that simplifies index math. Here is the identity. Suppose a sum d has some terms that are multiples of c , and the rest. Write $d = c*q + r$, where $c*q$ collects the multiples and r the rest. Then

$$\begin{aligned} d // c &= q + r // c \\ d \% c &= r \% c \end{aligned}$$

This is true in the integers for any $c > 0$, because $c \cdot q$ is a multiple of c and does not change the remainder. So $(i \cdot 4 + j) // 4$ has $q = i$ and $r = j$, and becomes $i + j // 4$. If the ranges then show that j is in $[0, 3]$, the rule of ranges turns $j // 4$ into 0 , and the result is just i . Likewise $(i \cdot 4 + j) \% 4$ becomes $j \% 4$, which is j by the bucket rule.

The code finds the multiples by looking at the terms of the sum. A term is visibly a multiple of c if it is a constant divisible by c , or a product with a constant factor divisible by c . That is a limited test. It does not find $(i \cdot 2) \cdot 2$ as a multiple of 4. It is not wrong, only incomplete, and the incomplete case is handled by the constant merging rules first, which turn $(i \cdot 2) \cdot 2$ into $i \cdot 4$.

the divisible-terms split · uopc/symbolic.py

```
def _terms(u):
    """the non-ADD leaves of a tree of ADDs, left to right."""
    out, stack = [], [u]
    while stack:
        n = stack.pop()
        if n.op is Ops.ADD:
            stack.extend(reversed(n.src))
        else:
            out.append(n)
    return out

def _quotient(t, c):
    """t / c if t is visibly a multiple of c (a constant, or something times a
    constant), else None."""
    if t.op is Ops.CONST:
        return const(t.arg // c) if t.arg % c == 0 else None
    if t.op is Ops.MUL and t.src[1].op is Ops.CONST and t.src[1].arg % c == 0:
        return t.src[0] * const(t.src[1].arg // c)
    return None

def _sum(terms):
    out = const(0)
    for i, t in enumerate(terms):
        out = t if i == 0 else out + t
    return out

def _split(d, c):
    """split the sum d into (quotients of the multiples of c, the rest), or None if
    nothing splits
    or if wraparound could make the split wrong."""
    if c <= 0 or not d.safe:
        return None
    terms = _terms(d)
    parts = [(t, _quotient(t, c)) for t in terms]
```

```

quot = [q for _, q in parts if q is not None]
rest = [t for t, q in parts if q is None]
if not quot:
    return None
r = _sum(rest)
if rest and not r.safe:
    return None
return _sum(quot), (r if rest else None)

@rule("(a*c1+b)//c2", UPat(Ops.FLOOR DIV, dtype=INT, src=(var("d"), cvar("c"))), SYM)
def _div_split(d, c):
    s = _split(d, c.arg)
    if s is not None:
        q, r = s
        return q if r is None else q + r // c

@rule("(a*c1+b)%c2", UPat(Ops.FLOORMOD, dtype=INT, src=(var("d"), cvar("c"))), SYM)
def _mod_split(d, c):
    s = _split(d, c.arg)
    if s is not None:
        q, r = s
        return const(0) if r is None else r % c

```

The overflow guard

In true mathematics the identity holds without conditions. In int32 arithmetic it does not. Here is the failing case, found by z3: for $(a*4 + b) // 4$, take $a = 2^{30}$ and $b = -2^{31}$. The true sum $a*4 + b$ is $2^{32} - 2^{31} = 2^{31}$, which does not fit and wraps to -2^{31} , so the left side is $-2^{31} // 4 = -2^{29}$. The right side is $a + b // 4 = 2^{30} - 2^{29} = 2^{29}$. The answers have opposite signs. The rule changed the program.

So the split has a guard, at the top of `_split`: `if c <= 0 or not d.safe: return None`. The flag `safe` (chapter 6) says that no operation inside `d` can wrap. When `d` is safe, the wrapped value of `d` is the true value, the identity holds, and the rule can fire. The guard also checks the part left over, `rest`, because after the split the leftover sum has to be safe on its own. The guard was tested by removing it. With the guard deleted from a copy of the code, the exhaustive test of small domains below still passes, because in a small domain nothing overflows. The soundness test, the special value test and the range test pass too. The overflow test fails, and so do the hand written unit cases. So two of the test layers exist for this one line, and chapter 15 shows the whole experiment.

z3 shows what the guard does. Here is the list of results for the split with `c = 4` and `c = 8`, and for the other divisors:

the split: proved with the guard, refuted without it · output of proofs.py

```

--- the divisible-terms split: (a*k*c + b)//c -> a*k + b//c ---
ok (a*4+b)//4 -> a*1+b//4 when a*4 and the sum fit          proved for every
input [0.2s]
ok (a*4+b)%4 -> b%4 when a*4 and the sum fit                proved for every
input [0.0s]
ok (a*4+b)//4 -> a*1+b//4 WITHOUT the guard                  COUNTEREXAMPLE
a=1073741824, b=-2147483648 [0.0s]
ok (a*24+b)//8 -> a*3+b//8 when a*24 and the sum fit          proved for every
input [0.1s]
ok (a*24+b)%8 -> b%8 when a*24 and the sum fit                proved for every
input [0.0s]
ok (a*24+b)//8 -> a*3+b//8 WITHOUT the guard                  COUNTEREXAMPLE
a=457266860, b=1929388032 [0.0s]
--- the same split for divisors that are not powers of two, over mathematical
integers (z3 bit vectors time out on division by 3) ---
ok (a*6+b)//3 -> a*2+b//3 (integers, guarded)                proved for every
input [0.0s]
ok (a*6+b)%3 -> b%3 (integers, guarded)                      proved for every
input [0.0s]
ok (a*15+b)//5 -> a*3+b//5 (integers, guarded)                proved for every
input [0.0s]
ok (a*15+b)%5 -> b%5 (integers, guarded)                      proved for every
input [0.0s]
ok (a*35+b)//7 -> a*5+b//7 (integers, guarded)                proved for every
input [0.0s]
ok (a*35+b)%7 -> b%7 (integers, guarded)                      proved for every
input [0.0s]
ok (a*6+b)//6 -> a*1+b//6 (integers, guarded)                proved for every
input [0.0s]
ok (a*6+b)%6 -> b%6 (integers, guarded)                      proved for every
input [0.0s]

```

With the guard, $(a*k*c + b) // c = a*k + b // c$ and $(a*k*c + b) \% c = b \% c$ are proved for every a and b , in bit vectors for $c = 4$ and $c = 8$, and over the true integers for four other divisors. Without the guard, z3 produces an input where the division fails, for $c = 4$ and for $c = 8$. (The unguarded split was only tried for those two divisors, and the remainder was only checked with the guard.)

A table of results

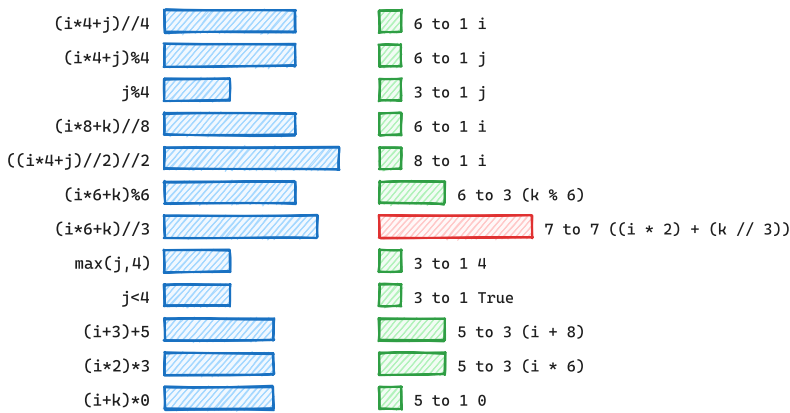
Here are twelve index expressions simplified, with i in $[0, 15]$, j in $[0, 3]$, k in $[0, 7]$. The program counts the nodes of the input and the output and checks the output against the input on all 512 inputs. This is the real output of `demons/run_demos.py`:

index arithmetic before and after · output of demos/run_demos.py

<code>(i*4+j)//4</code>	nodes 6 -> 1 i	checked on all 512
<code>inputs: True</code>		
<code>(i*4+j)%4</code>	nodes 6 -> 1 j	checked on all 512
<code>inputs: True</code>		
<code>j%4</code>	nodes 3 -> 1 j	checked on all 512
<code>inputs: True</code>		
<code>(i*8+k)//8</code>	nodes 6 -> 1 i	checked on all 512
<code>inputs: True</code>		
<code>((i*4+j)//2)//2</code>	nodes 8 -> 1 i	checked on all 512
<code>inputs: True</code>		
<code>(i*6+k)%6</code>	nodes 6 -> 3 (k % 6)	checked on all 512
<code>inputs: True</code>		
<code>(i*6+k)//3</code>	nodes 7 -> 7 ((i * 2) + (k // 3))	checked on all 512
<code>inputs: True</code>		
<code>max(j,4)</code>	nodes 3 -> 1 4	checked on all 512
<code>inputs: True</code>		
<code>j<4</code>	nodes 3 -> 1 True	checked on all 512
<code>inputs: True</code>		
<code>(i+3)+5</code>	nodes 5 -> 3 (i + 8)	checked on all 512
<code>inputs: True</code>		
<code>(i*2)*3</code>	nodes 5 -> 3 (i * 6)	checked on all 512
<code>inputs: True</code>		
<code>(i+k)*0</code>	nodes 5 -> 1 0	checked on all 512
<code>inputs: True</code>		

Eight of the twelve reduce to a single node: `i`, `j`, or a constant. `(i*6+k)%6` becomes `k % 6` and not `k`, because `k` reaches 7, past the first bucket. `(i*6+k)//3` becomes `(i*2) + (k//3)` and it is not smaller: the split has made the program no shorter. In this case the rule is not a gain, and the rule set does not know it. A cost model would reject it. There is none in this module.

index expressions, nodes before and after



the point:
blue is nodes before, green is after. the split on $(i*6+k)//3$ does not shrink the graph.

How it was tested

The integer rules are tested at four levels.

- Hand written cases.** 52 of them in `tests/test_rules_unit.py`, each an expression with a declared result, many of them “must not fire”. Plus 547 point checks of the hardest cases: the overflowing remainder, the wrapped product, $(x // 2) // 3$ for every x from -200 to 200 and the five edge values.
- Exhaustive small domains.** `test_exhaustive` builds 1,200 random index programs over three params with small ranges, simplifies each, and runs the original and the result on **every** point of the domain (320 points). 994 of the programs were changed by the simplifier, and every one was checked on all 320 inputs. 385,200 checks.
- Big ranges.** `test_overflow` repeats this with params that have ranges as large as the whole `int32` and checks 640,260 points that include the boundaries. It checks that rules that need “no overflow” are not applied when overflow is possible.
- Proofs.** The 45 integer claims in `proofs.py`. 2 of them are counterexamples that show the unguarded split is wrong.

A rule that never fires in the random tests is not tested by them. The test prints the list. In the last run, the only rule that never fired was `cast to own dtype`, which is covered by two hand written cases (`x.cast(I32)` for an `int` and `f.cast(F32)` for a `float`), because the random program generator never builds a cast to the same type.

The same in tinygrad

The file `tinygrad/uop/divandmod.py` holds tinygrad's version of this chapter. Its function `fold_divmod_general` begins with the same two ideas: if the quotient has a constant value by its range, return it, and if the divisor is a positive constant, split the sum into its terms and treat the ones that divide evenly. It has more rules than this module, among them one for `(x % (k*c)) // c -> (x // c) % k` and one that removes a nested modulo from a sum. The idea of the chapter is the real thing. What the real thing adds is more patterns of the same kind.

Where to learn more

- Henry warren, *hacker's delight*, chapter 10 on integer division by constants. The compiler technique of replacing a division by a multiply is a different topic from this chapter, but the same book teaches the arithmetic.
- Sven verdoolaege, *isl: an integer set library for the polyhedral model*, 2010, and the isl manual. The full theory of reasoning about affine index expressions with floor division and modulo.
- The Halide paper, jonathan ragan-kelley and others, *Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines*, PLDI 2013, and the Halide source file `Simplify_Div.cpp`. Halide has the most developed simplifier for exactly this kind of expression.
- z3's guide for integers and bit vectors, at microsoft.github.io/z3guide, for writing your own checks.
- The tinygrad source: `tinygrad/uop/divandmod.py`, and the tests under `test/` whose names contain `symbolic`.

Exercises

1. Simplify `(i*8 + j) // 8` for `i` in `[0, 15]` and `j` in `[0, 7]` by hand, then by `simplify`. Now widen `j` to `[0, 8]`. What happens, and why?
2. Check `(x // 2) // 3 == x // 6` for every `x` in `[-50, 50]`. Then check `(x // -2) // 3 == x // -6`. What do you find?
3. Build `(a*4 + b) // 4` with `a` in `[0, 2**29 - 1]` and `b` in `[0, 3]`. Is the split applied? Change the top of the range of `a` to `2**29` and try again. Which is the largest top for which the split still applies, and why?
4. Write the rule `x % c == 0 -> ...` for the case where `x` is `i * c`. Is it already covered by the split?

5. Use z3 to check $(x * 3) // 3 \rightarrow x$ over 32 bit integers. Proved or refuted? Find a counterexample if refuted, and compare with the guarded split.

Part 4

From graph to machine

Chapter 12

Rendering C

The renderer turns a `CALL` into C source text. It is the only part of the compiler that produces code, and it is short: 133 lines. Most of those lines are not about format. They are about C, which has rules about what programs mean that the contract of chapter 3 does not share. This chapter shows the renderer, then each place where C would have changed the meaning of a program if the renderer were careless, with a demonstration of each.

One statement per node

The idea is simple. Take the nodes of the body in topological order (chapter 4). For each node, write one line of C that computes it from the names of its srcs. Give it the name `v0`, `v1`, and so on. The parameters are `p0`, `p1`. Constants are written in place as literals. The last node is returned.

render in uopc/render.py

```
def render(call: UOp) -> str:
    assert call.op is Ops.CALL
    body, args = call.src[0], call.src[1:]
    name: dict[UOp, str] = {}
    decls, lines, nvar = [], [], 0
    for u in body.toposort():
        op = u.op
        if op is Ops.CONST:
            name[u] = literal(u)
        elif op is Ops.PARAM:
            name[u] = f"p{u.arg.slot}"
        elif op is Ops.ALLOC:
            name[u] = f"a{u.arg.slot}"
            decls.append(f" {u.dtype.base.c} a{u.arg.slot}[{u.dtype.size}];")
        elif op is Ops.AFTER:
            name[u] = name[u.src[0]] # the same pointer; the
srcs only fix the order
        elif op is Ops.SINK:
            pass
        elif op is Ops.STORE:
            lines.append(f" *{name[u.src[0]]} = {name[u.src[1]]};")
        else:
            v = name[u] = f"v{nvar}"
            nvar += 1
            if op is Ops.INDEX:
                lines.append(f" {u.dtype.base.c} *{v} = {name[u.src[0]]} + {name[u.
src[1]]};")
            elif op is Ops.LOAD:
```

```

        lines.append(f"    {u.dtype.c} {v} = *{name[u.src[0]]};")
    else:
        lines.append(f"    {u.dtype.c} {v} = {_alu(u, [name[s] for s in
u.src])};")
    params = ", ".join(f"{a.dtype.base.c} *p{i}" if isinstance(a.dtype, Ptr) else
f"{a.dtype.c} p{i}"
                    for i, a in enumerate(args)) or "void"
    ret = "void" if body.dtype is VOID else body.dtype.c
    out = [PRELUDE, f"{ret} {call.arg.name}({params}) {{{" + decls + lines
if body.dtype is not VOID:
    out.append(f"    return {name[body]};")
out.append("}")
    return "\n".join(out) + "\n"

```

For the relu program of chapter 1 this gives, as printed there:

```

float relu(float p0, float p1) {
    float v0 = p0 * p1;
    float v1 = v0 + 1.0f;
    float v2 = (v1 > 0.0f) ? v1 : 0.0f;
    return v2;
}

```

There are three details of the design.

Shared nodes are computed once. A node used in two places has one name and one line. This is where hash consing pays again: a repeated subexpression is one node, so it is one statement. The renderer does no search for repeats.

Every node is computed, in order. A `WHERE` node is the line `c ? a : b`, with `a` and `b` already computed on earlier lines. This is why chapter 3 insisted that every op is total: nothing in the generated code can skip a computation, so nothing may crash.

The code is not optimised. `v0`, `v1`, `v2` are separate variables, where an expert would write one expression. clang removes the difference: its register allocator and instruction selector do the work that the renderer leaves out, at `-O2`.

The operations themselves are written by two small functions:

`_cast` and `_alu` in `uopc/render.py`: one c expression per op

```

def _cast(src, dst, a):
    if src is dst:
        return a
    if dst is B00L:
        return f"({a}) != 0{'.'0f' if src is F32 else ''})"
    if dst is F32:
        return f"((float){a})"
    return f"f32_to_i32({a})" if src is F32 else f"((int32_t){a})"

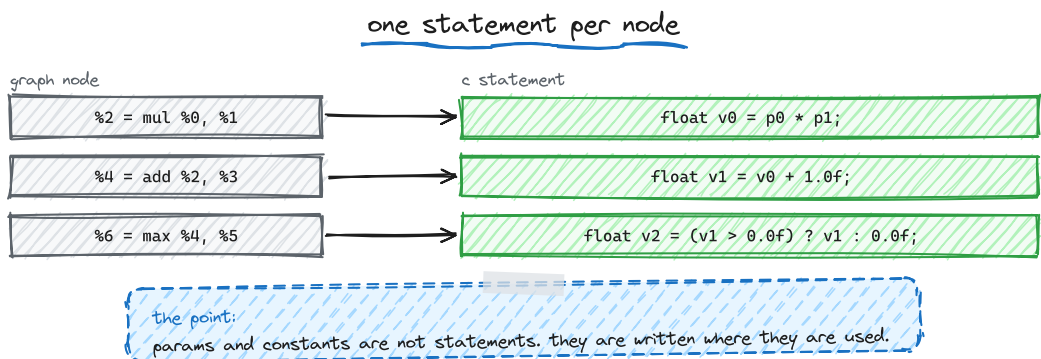
```

```

def _alu(u: UOp, a: list) -> str:
    op, k = u.op, u.dtype.kind
    s0 = u.src[0].dtype
    if op is Ops.CAST:
        return _cast(s0, u.dtype, a[0])
    if op is Ops.WHERE:
        return f"{a[0]} ? {a[1]} : {a[2]}"
    if op is Ops.NEG:
        return f"-{a[0]}"
    if op is Ops.RECIP:
        return f"1.0f / {a[0]}"
    if op is Ops.EXP2:
        return f"exp2f({a[0]})"
    if op is Ops.LOG2:
        return f"log2f({a[0]})"
    if op is Ops.ADD:
        return f"{a[0]} + {a[1]}"
    if op is Ops.MUL:
        return f"{a[0]} && {a[1]}" if k == "bool" else f"{a[0]} * {a[1]}"
    if op is Ops.MAX:
        return f"{a[0]} || {a[1]}" if k == "bool" else f"({a[0]} > {a[1]}) ? {a[0]} : {a[1]}"
    if op is Ops.CMPLT:
        return f"{a[0]} < {a[1]}"
    if op is Ops.CMPNE:
        return f"{a[0]} != {a[1]}"
    x, y = u.src
    if x.vmin >= 0 and y.vmin > 0:
        # no sign problem and no
        zero: C's own / and % are right
        return f"{a[0]} / {a[1]}" if op is Ops.FLOORDIV else f"{a[0]} % {a[1]}"
    return f"'floordiv_i32' if op is Ops.FLOORDIV else 'floormod_i32'({a[0]}, {a[1]})"

```

Most ops are one-line translations. Division and the casts are where C's rules need care.



Trap 1: signed overflow is undefined

In C, `int32_t a + b` has no defined meaning when the sum does not fit. The standard says the program has undefined behaviour, and a compiler may assume it never happens. The contract says the sum wraps. The demonstration compiles the function `x + 1 > x` three ways and calls it with the largest `int32`:

the same c, three compiler settings · output of `demos/run_demos.py`

```
clang -O0          f(INT_MAX) = 0
clang -O2          f(INT_MAX) = 1
clang -O2 -fwrapv  f(INT_MAX) = 0
with -ftrapv at -O0: exit code -4 (SIGILL is 4, SIGTRAP is 5, so -4 or -5 means the
overflow trapped)
```

At `-O0` the answer is 0, because the machine really does wrap: `INT_MAX + 1` is `INT_MIN`, which is not greater than `INT_MAX`. At `-O2` the answer is 1: clang assumes that overflow never happens, so `x + 1 > x` is always true, and it deletes the comparison. The same source gives two answers depending on how hard the compiler tries. With `-fwrapv`, which defines signed overflow to wrap, `-O2` gives 0 again. The last line shows a third way to see the problem: `-ftrapv` makes overflow stop the program with a signal.

So the runtime of chapter 13 compiles with `-fwrapv`. With it, `+`, `*` and unary `-` on `int32_t` in the generated C match the contract.

Trap 2: division

C truncates, so `-7 / 2` is `-3`, and the contract floors. And C has two inputs with no defined result: a zero divisor, and `INT_MIN / -1`, whose result `231` does not fit. On x86 the second kills the process:

the division that crashes · output of `demos/run_demos.py`

```
exit code (negative = killed by signal): -8 (SIGFPE is 8, so -8)
our floordiv on the same inputs: -2147483648
```

The exit code is `-8`, which means the process died of signal 8, `SIGFPE`. The contract's `INT_MIN // -1` is `INT_MIN`. So division is not written as `/` in general. The prelude of the generated file holds two helper functions:

the prelude of every generated file, and the literal writer · `uopc/render.py`

```
PRELUDE = """\
#include <stdint.h>
#include <stdbool.h>
```

```

#include <math.h>
#include <float.h>
_Static_assert(FLT_EVAL_METHOD == 0, "float ops must round to float32 after every
operation");

static inline int32_t floordiv_i32(int32_t a, int32_t b) {
    if (b == 0) return 0;
    if (b == -1) return (int32_t)(0u - (uint32_t)a);
    int32_t q = a / b, r = a % b;
    return (r != 0 && ((r < 0) != (b < 0))) ? q - 1 : q;
}
static inline int32_t floormod_i32(int32_t a, int32_t b) {
    if (b == 0) return a;
    if (b == -1) return 0;
    int32_t r = a % b;
    return (r != 0 && ((r < 0) != (b < 0))) ? r + b : r;
}
static inline int32_t f32_to_i32(float x) {
    if (x != x) return 0;
    if (x >= 2147483648.0f) return INT32_MAX;
    if (x <= -2147483648.0f) return INT32_MIN;
    return (int32_t)x;
}
"""

def literal(u: UOp) -> str:
    v = u.arg
    if u.dtype is BOOL:
        return "true" if v else "false"
    if u.dtype is I32:
        if v == INT_MIN:
            return "(-2147483647 - 1)"
        # -2147483648 is not an int literal in C
        return f"({v})" if v < 0 else str(v)
    if v != v:
        return "NaN"
    if math.isinf(v):
        return "INFINITY" if v > 0 else "(-INFINITY)"
    s = "%.9g" % v
    if not any(c in s for c in ".e"):
        s += ".0"
    s += "f"
    return f"({s})" if s.startswith("-") else s

```

`floordiv_i32` handles a zero divisor first, then `-1` (negating through unsigned arithmetic, which wraps by definition, to avoid `INT_MIN / -1`), then does C's truncating divide and corrects the quotient by one when the remainder is nonzero and its sign differs from the divisor's. `floormod_i32` is the matching remainder. Both are marked `static inline`, so clang inlines them.

There is one shortcut, in `_alu`. When the numerator has `vmin >= 0` and the divisor has `vmin > 0` (chapter 6), no sign problem and no zero are possible, C's own `/` and `%` are correct, and the renderer writes them. This is the common case in index math, where every quantity is non-negative. The shortcut has a precise condition, and the condition has a test: a planted bug that loosens `y.vmin > 0` to `y.vmin >= 0` is caught by `test_render_zero_divisor`, which runs 1,080 divisions with divisors that include 0 and negative numbers against the interpreter.

Trap 3: float to int

C says converting a float that is outside the int range, or NaN, to an int has no defined result. The contract says it saturates and NaN gives 0. The prelude's `f32_to_i32` has the two range checks and the NaN check, and then does the plain C cast, which is defined for the values that remain.

Trap 4: fused multiply add

Many CPUs have an instruction that computes `a*b + c` with one rounding instead of two. A compiler is allowed, in some modes, to use it. The result is more accurate and different. This is the demonstration, with `a = 1 + 2-12` and `c = -(1 + 2-11)`:

`a*b+c` with and without fused multiply add · output of `demos/run_demos.py`

```
clang -march=native -ffp-contract=fast      -> 5.96046448e-08
clang -march=native -ffp-contract=off      -> 0
python, rounding a*b first as float32 does: 0 (matches the unfused build)
python, rounding once after the add, as a fused instruction does: 5.96046448e-08
(matches the fused build)
fused instruction in the assembly: ['vfmadd213ss\t%xmm2, %xmm1, %xmm0      # xmm0 =
(xmm1 * xmm0) + xmm2']
```

`a * a` is `1 + 2-11 + 2-24`. Rounded to float32 on its own, the `2-24` is lost, the product is `1 + 2-11`, and adding `c` gives exactly 0. Computed with one rounding, the `2-24` survives, and the answer is `5.96e-08`. The machine has a `vfmadd` instruction, and with `-ffp-contract=fast` clang uses it, as the assembly line shows. The Python reference, rounded after the multiply, agrees with the unfused build; rounded once at the end, it agrees with the fused build. The contract says two roundings, so the compiler flag is `-ffp-contract=off`.

A second flag, `-fno-fast-math`, makes sure no option turns on algebra that changes floats, the same thing chapter 10 refused to do in the simplifier.

Trap 5: the width of float arithmetic

On a 32-bit x86 target that uses the old x87 unit, C may compute `float` arithmetic in a wider format and round later. Then `a + b` can differ from a true float32 sum. The macro `FLT_EVAL_METHOD` says which mode the compiler uses, and the generated file starts with

```
_Static_assert(FLT_EVAL_METHOD == 0, "float ops must round to float32 after every operation");
```

So on such a target the compile fails loudly and does not run with the wrong numbers. On the machine used for this book it is 0.

Trap 6: literals

Writing a constant as text has three small traps.

- **The smallest int.** `-2147483648` is not a literal in C. It is the negation of `2147483648`, which does not fit in an int, so its type is something else. The renderer writes `(-2147483647 - 1)`.
- **Negative numbers.** `--(-5)` printed without brackets could become `--5`, a decrement. So a negative literal is written in brackets: `(-5)`, `(-1.5f)`.
- **Floats.** A float constant is printed with `%.9g`, nine significant digits. Nine digits are enough to identify any float32 uniquely, so the literal reads back as the same value. An `f` suffix makes it a `float` and not a `double`. NaN is `NAN`, and infinities are `INFINITY` and `(-INFINITY)`, from `math.h`.

The contract, checked on real C

The renderer has to produce C that agrees with the interpreter. `test_render_vs_interp` generates 250 random programs, half of them with integer params that have bounds (so the fast `/` path is exercised), renders each original and each simplified, compiles both, and runs each on 60 random and special inputs, comparing every result with the interpreter, bit for bit. 30,000 checks, all pass. The programs here leave out `EXP2` and `LOG2`, whose last bit can differ (chapter 3).

There are three planted bugs that the render tests catch: a float to int cast written as a plain C cast, floor division written as `/`, and `max` for floats written with `fmaxf` (when one argument is NaN, `fmaxf` returns the other one, while the contract returns `b` whenever `a > b` is false, so they differ when `b` is NaN). Each of them fails at the first random program that hits the case.

What the renderer does not do

It does not use `restrict`. Pointers in C may alias, and the renderer treats every buffer pointer as possibly overlapping with the others. tinygrad's C renderer marks its buffer parameters `restrict`, a promise that they do not overlap, which lets clang reorder loads and stores more freely. This module leaves it out, because it has to be correct before it is fast. Chapter 14 returns to aliasing.

It does not generate loops, because there are no loops in the ops of this module. A `CALL` renders as straight-line code.

And it does not check that the output compiles. clang does. An error from clang means a bug in the renderer, and the runtime reports clang's message, which names the source file in the cache directory.

The same in tinygrad

tinygrad's C renderers are in `tinygrad/renderer/cstyle.py`. There is a base class, `CStyleLanguage`, and subclasses for clang (`ClangRenderer`), opencl, metal and CUDA, which differ in how they spell types and builtins. `ClangRenderer` writes infinity as `__builtin_inff()` and NaN as `__builtin_nanf("")`, marks buffer arguments `restrict`, and maps ops to C text with a table, `code_for_op`. In the compile line for CPU in `tinygrad/runtime/support/compiler_cpu.py`, the flags read for this book are `-O2 -fPIC -ffreestanding -fno-math-errno -nostdlib`. This book's flags, `-fwrapv`, `-ffp-contract=off` and `-fno-fast-math`, are the price of an exact contract. tinygrad does not need that exact a contract for its index math, and for its float math it accepts what clang does.

Where to learn more

- *The C standard*: n3096 (a free late draft of c23) for the rules on integer conversion, undefined behaviour, `FLT_EVAL_METHOD` and floating point contraction (the `FP_CONTRACT` pragma).
- The clang documentation on `-fwrapv`, `-ffp-contract` and `-ffast-math`, at clang.llvm.org/docs/UsersManual.html.
- John regehr's blog posts on undefined behaviour, and the paper *finding and understanding bugs in C compilers* (csmith) by xuejun yang, yang chen, eric eide and john regehr, PLDI 2011, which shows how many compiler bugs hide in exactly these corners.
- Chris lattner, *what every C programmer should know about undefined behavior*, three posts on the LLVM blog.

- *A tour of C's undefined behaviour*: the sanitizers `-fsanitize=undefined` and `-fsanitize=address`, in the clang docs, which find these bugs at run time (they need a runtime library that was not installed on the machine of this book, so this book did not use them).
- The tinygrad source: `tinygrad/renderer/cstyle.py`.

Exercises

1. Render `x // y` for ints `x` and `y` with no known ranges. Now give `x` the range `[0, 100]` and `y` the range `[1, 10]` and render again. What changed in the C, and why is it correct?
2. Render `max(0.0, x)` for a float `x`. What does the C ternary return for `nan`? What would `fmaxf(0.0f, x)` return? Which one matches the contract?
3. Write the C for `cast(x, I32)` for a float `x` by hand, using the helper. Check `1e10`, `-1e10`, `nan`, `-0.5`.
4. Compile the `x + 1 > x` function from the demo at `-O1` and at `-O3`. Which optimisation levels give 1?
5. A constant `-2147483648` is rendered as `(-2147483647 - 1)`. Write the C for the constant `-2147483647` and for `2147483647`, and say why neither needs the trick.

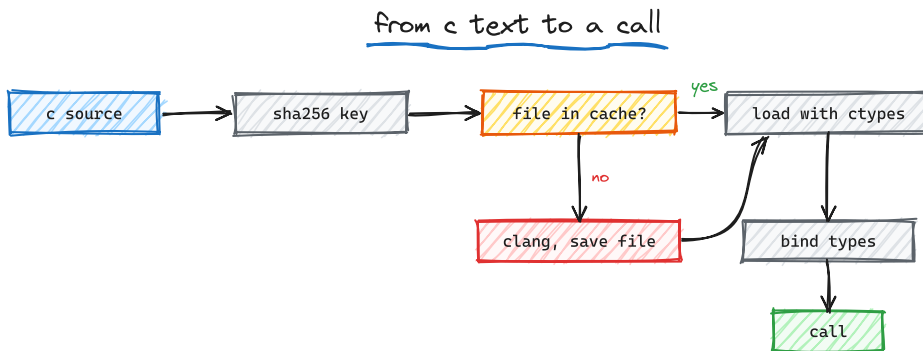
Chapter 13

Compiling and running

The renderer made C text. This chapter makes it run. The runtime is a function that takes a `CALL`, gets C from the renderer, has clang compile it, loads the result into the Python process, and returns something you can call like a Python function. It is 105 lines.

The steps

1. **Render** the call to C source (chapter 12).
2. **Hash** the source, together with the compiler's name and version and the flags. This is the cache key.
3. If a file with that name exists in the cache directory, use it. If not, write the source to a file, run `clang`, and save the output under the key.
4. **Load** the compiled shared library into the process, with Python's `ctypes`.
5. **Bind** the function: tell `ctypes` the type of each argument and of the result.
6. **Call** it.



the point:
the file name is a hash of the source, the compiler version and the flags, so a stale file cannot be loaded.

The compile step is this function:

compile_c and Program in `uopc/runtime.py`

```
def compile_c(src: str, cc: str = CC, flags=None) -> str:
    """compile C source to a shared library and return its path. the file name is a
    hash of the
```

```

    compiler (name and version), the flags and the source, so identical requests
    compile once."""
    flags = CFLAGS if flags is None else flags
    key = hashlib.sha256("\0".join([cc, _cc_version(cc), *flags,
src])).encode().hexdigest()[:24]
    os.makedirs(CACHE_DIR, exist_ok=True)
    so = os.path.join(CACHE_DIR, key + ".so")
    if os.path.exists(so):
        STATS["hits"] += 1
        return so
    cfile = os.path.join(CACHE_DIR, key + ".c")
    with open(cfile, "w") as f:
        f.write(src)
    t0 = time.perf_counter()
    tmp = so + f".{os.getpid()}.tmp"
    r = subprocess.run([cc, *flags, cfile, "-o", tmp, "-lm"], capture_output=True,
text=True)
    if r.returncode != 0:
        raise RuntimeError(f"{cc} failed:\n{r.stderr}")
    os.replace(tmp, so) # atomic: no one sees
half a file
    STATS["compiles"] += 1
    STATS["compile_seconds"] += time.perf_counter() - t0
    return so

class Program:
    """a CALL, rendered, compiled and loaded."""
    def __init__(self, call: UOp):
        assert call.op is Ops.CALL
        self.call = call
        self.source = render(call)
        self.lib = ctypes.CDLL(compile_c(self.source))
        self.fn = getattr(self.lib, call.arg.name)
        args = call.src[1:]
        self.fn.argtypes = [ctypes.POINTER(CTYPE[a.dtype.base]) if isinstance(a.dtyp
e, Ptr) else CTYPE[a.dtype]
                            for a in args]
        self.fn.restype = None if call.dtype is VOID else CTYPE[call.dtype]

    def __call__(self, *args):
        return self.fn(*[a.arr if isinstance(a, Buffer) else a for a in args])

```

The flags are the ones chapter 12 explained: `-O2 -fwrapv -ffp-contract=off -fno-fast-math -std=c11 -shared -fPIC`. `-shared -fPIC` make a shared library, a file of machine code that a running process can load. clang is run with `subprocess`. If it fails, the error carries clang's message, with the source file path, and the function raises `RuntimeError`.

The cache

Compiling takes time (measured below) and the same C is requested over and over: a test that runs a program on 60 inputs compiles once. The cache is a directory, by default under the system's temporary directory, or wherever `UOPC_CACHE` points. The file name is the first 24 hex digits of the sha256 hash of: the compiler name, the compiler's version line, every flag, and the whole source text.

So it is **content addressed**. Two requests with equal content use one file, and any change to the source, the flags, or the compiler gives a different name. The version is part of the key because a cache from an older clang is not valid for a newer one, and a key without it would load the old machine code silently. The cost is one run of `clang --version` per process.

A compile is made safe against two processes compiling the same thing at once: clang writes to a temporary name that includes the process id, and `os.replace` moves it into place in one step, so a reader never sees a half written file. `STATS` counts compiles, cache hits and the seconds spent compiling, which the tests print.

Calling it

A `Program` loads the library and gets the function by name. The types for `ctypes` come from the call's arguments:

Dtype	ctypes type
<code>bool</code>	<code>c_bool</code>
<code>i32</code>	<code>c_int32</code>
<code>f32</code>	<code>c_float</code>
A pointer to <code>T</code>	A pointer to the ctypes type of <code>T</code>

Scalars are passed by value. A pointer argument is passed a `Buffer`, which is a ctypes array of the element type, so its memory has the layout C expects. `Buffer(F32, 4).copyin([...])` makes one and fills it, and `.tolist()` reads it back. The result type is `None` for a body of type `void` and the ctypes type of the value otherwise.

`execute(call, buffers)` is a convenience for the usual case: it takes a call whose arguments are `BUFFER`s and constants, makes a zeroed buffer for each `BUFFER` slot it has not been given, compiles, calls, and returns the result.

And that is the whole path from a graph in Python to a number computed by machine code. In the example of chapter 14, `bump(in, out)` is compiled once, loaded, and run on two lists of floats.

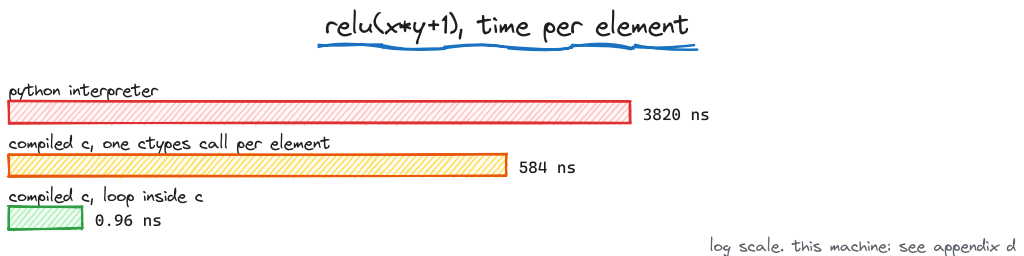
What it costs

Where does the time go? `bench.py` measured three ways of running the same function, `relu(x*y + 1)`, on this machine (appendix d names it):

one function, three ways of running it · output of `bench.py`

python interpreter (evaluate)	3820 ns per element	(measured on 20,000 elements, scaled to 200,000)
compiled C, one ctypes call each	584 ns per element	(200,000 elements)
compiled C, loop inside C	0.96 ns per element	(20,000,000 elements)
ratio interpreter / C loop: 3,966x ctypes call / C loop: 606x		

- The **Python interpreter** (`evaluate` , walking the graph for every element) takes about 3,800 ns per element.
- The **compiled C function, called once per element from Python** takes about 580 ns per element. This is a factor of 6.5 faster than the interpreter, and still slow, because almost all of that time is the call. A `ctypes` call converts its arguments, switches into C, and converts the result back. The actual arithmetic takes less than a nanosecond.
- The **compiled C with the loop inside C** takes about 1 ns per element, over 20 million elements. That is about 4,000 times faster than the interpreter and about 600 times faster than calling one element at a time.



the point:
calling c from python costs more than doing the arithmetic. put the loop in c.

There is a lesson in the third line that this course will return to in week 3. The function compiled here does one element of work, so a loop in Python has to cross the boundary once per element, and that boundary costs far more than the work. The fix is to move the loop into the generated code, so it crosses once per array. That is exactly why the course introduces loops next. A compiler for machine learning that cannot generate loops is a compiler for a few thousand elements.

The second measurement is the cost of compiling:

clang's time against the size of the function · output of bench.py

statements clang)	-00	-01	-02 (seconds, median of 3, includes starting
10	0.029	0.027	0.028
100	0.029	0.034	0.037
1000	0.044	0.085	0.086
5000	0.144	0.658	0.701

A function of 10 or 100 statements compiles in about 0.03 seconds at any optimisation level. That is the cost of starting clang, and it dominates. At 1,000 statements `-01` and `-02` take about 0.09 seconds, and at 5,000 statements about 0.7 seconds, while `-00` takes 0.14 seconds. So optimisation costs time that grows faster than the size of the function, and `-00` is about 5 times cheaper at 5,000 statements. For these tests the default, `-02`, is fine: the programs are small, and each compiles once.

Two lessons follow, and both are about the cache. A 30 ms compile costs tens of thousands of times as much as one call of the function, so a program that is run once is better interpreted than compiled, and a program that is run often must be compiled once and kept. The cache does the second.

And the cost of Python itself, from the graph side:

building and simplifying graphs · output of bench.py

```
300 random index expressions: 4,311 nodes if each is counted alone, 2,106 distinct
nodes in memory
memory while building them (python objects, measured by tracemalloc): 870 KiB, 423
bytes per distinct node
simplify on all 300: 50 ms, 85,490 nodes per second (nodes counted per expression,
before simplifying); 2,735 nodes after
p=p+p repeated 18 times: 19 nodes in the graph; as a tree it would have 524,287
```

Simplifying the index expressions of this benchmark runs at about 85,000 nodes per second in Python. The 400,001 node chain of chapter 4 took 14.2 seconds, which is about 28,000 nodes per second, so if the time grows in proportion, a chain of a million nodes takes about 36 seconds. A production compiler in a compiled language does the same work thousands of times faster, which is one reason the course lets a student choose the language.

What can go wrong

- **clang is not installed.** `subprocess` raises `FileNotFoundError`. The book assumes clang is on the path. You can set `CC` in `runtime.py` to another compiler that accepts the same flags, such as gcc, whose `-fwrapv` and `-ffp-contract=off` are the same.

- **clang rejects the source.** This means a renderer bug. The message includes the source file path in the cache directory, where the C can be read.
- **A stale cache.** Removed by the version in the key. You can always delete the directory.
- **Running on another machine.** The compiled library is for the CPU and operating system that built it. The flags do not include `-march=native`, so the code uses only the baseline instruction set of the target and does not use fused multiply add or wide vector instructions that exist on this machine. A later module that wants speed will add them, and with them the question of which results are allowed to change.

The same in tinygrad

tinygrad's CPU runtime does the same job in `tinygrad/runtime/ops_cpu.py` and `tinygrad/runtime/support/compiler_cpu.py`. The compiler class there runs clang with `-c` (compile only, no linking) for a target such as `x86_64-none-unknown-elf`, passes `-march=<cpu>` taken from a target string, and reads the object code from clang's output. There is no shared library and no loader from the operating system. `ops_cpu.py` loads the object itself: a function called `jit_loader` links the object's sections, and the result is copied into a block of executable memory obtained with `mmap`, and called through `ctypes`. The flags read for this book are `-O2 -fPIC -ffreestanding -fno-math-errno -nostdlib`. The design, render, compile, cache, call, is the same as here. What differs is that tinygrad owns the last step, which lets it run on targets where a normal loader is not available, and lets it pick the CPU per target.

Where to learn more

- *Computer systems: a programmer's perspective*, chapter 7 on linking, for what a shared library and position independent code (`-fPIC`) are.
- Python's `ctypes` documentation, section on foreign function interfaces and calling conventions.
- The clang user's manual, at clang.llvm.org/docs/UsersManual.html, section "command line options" for `-O0` to `-O3`.
- *The architecture of open source applications*, chapter on LLVM, free at aosabook.org, by chris lattner. It explains what clang does between your C and the machine code.
- The tinygrad source: `tinygrad/runtime/ops_cpu.py` and `tinygrad/runtime/support/compiler_cpu.py`.

Exercises

1. Run `Program` on the `relu` call twice in one process and print `STATS`. How many compiles and how many cache hits?
2. Change one flag in `CFLAGS` (add `-O3`) and run again. Is the cache hit or a new compile? Why?
3. Time a Python loop that calls the `relu` function 1,000,000 times with `ctypes`, then write C that loops 1,000,000 times inside. What ratio do you measure on your machine?
4. Force a clang failure by editing the renderer to emit a bad type name. Read the error message and find the source file it names.
5. Compile the `x + 1 > x` function from chapter 12 with the runtime's flags, and then with `-fwrapv` removed. At what optimisation levels does the answer for `INT_MAX` change?

Chapter 14

Memory

So far a program is a pure function: numbers in, one number out. Real kernels read arrays and write arrays. This chapter adds memory to the graph, shows the one real difficulty that memory brings (order), and shows how the graph says what must happen before what. It is week 2 of the syllabus.

The memory ops

Op	Sres	What it is
<code>BUFFER</code>	None	A global array that exists before and after the call, with an element type and a size. It is passed to a call as an argument
<code>PARAM</code> with a pointer dtype	None	Inside a body, the stand-in for a <code>BUFFER</code> argument, as a <code>PARAM</code> is for a number
<code>ALLOC</code>	None	Scratch memory that lives for one call, like <code>float a[10];</code> in C. Its contents start undefined
<code>INDEX</code>	Pointer, int	The address of element <code>i</code> of a buffer. No memory is touched
<code>LOAD</code>	An <code>INDEX</code>	Read the element at that address
<code>STORE</code>	An <code>INDEX</code> , a value	Write the value at that address. Gives no value
<code>AFTER</code>	Pointer, then stores and loads	The same pointer, but anything that uses it must happen after the listed stores and loads

`PARAM` is not new. With a pointer dtype it plays the part of a `BUFFER` inside a body. The other six ops are new.

The first three create storage. `INDEX` computes an address from storage, `LOAD` and `STORE` are the only two ops that touch memory (chapter 6 showed the type checker enforces this with address spaces), and `AFTER` handles order. There is also one op of this book's own, `SINK`, explained below.

A call that uses memory has a body of type `void`. It does not compute a value. It changes buffers. Here is the smallest useful example, from `demos/walkthrough.py`: read `in[0]`, add 1, store into `out[0]`, then read `out[0]` back, double it, and store it into `out[1]`:

a kernel that reads, writes, reads what it wrote, writes again · output of `demos/walkthrough.py`

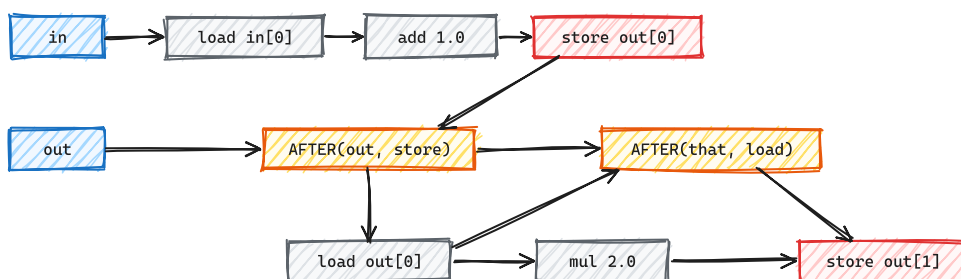
```
; uop v1
%0 = param : dtype=f32 slot=1 size=2
%1 = const : dtype=i32 value=0
%2 = index %0, %1
%3 = param : dtype=f32 slot=0 size=2
%4 = index %3, %1
%5 = load %4
%6 = const : dtype=f32 value=1.0
%7 = add %5, %6
%8 = store %2, %7
%9 = after %0, %8
%10 = index %9, %1
%11 = load %10
%12 = after %9, %11
%13 = const : dtype=i32 value=1
%14 = index %12, %13
%15 = const : dtype=f32 value=2.0
%16 = mul %11, %15
%17 = store %14, %16
%18 = sink %17
%19 = buffer : dtype=f32 slot=0 size=2
%20 = buffer : dtype=f32 slot=1 size=2
%21 = call %18, %19, %20 : name=bump
```

```
void bump(float *p0, float *p1) {
    float *v0 = p1 + 0;
    float *v1 = p0 + 0;
    float v2 = *v1;
    float v3 = v2 + 1.0f;
    *v0 = v3;
    float *v4 = p1 + 0;
    float v5 = *v4;
    float *v6 = p1 + 1;
    float v7 = v5 * 2.0f;
    *v6 = v7;
}
```

C result, out = [11.0, 22.0] interpreter: [11.0, 22.0]

Read the graph from the top. %0 is a pointer parameter for slot 1 (the output) with size 2, %3 is slot 0 (the input). %5 loads `in[0]`, %7 adds 1.0, and %8 stores. Then %9 = after %0, %8 is the output pointer again, but “after the store %8”. %10 indexes it, %11 loads. %12 = after %9, %11 is the pointer after that load, and the second store goes through it. %18 is the SINK, and %21 is the call, with two BUFFER arguments. The C is below it: ten statements, two of them stores. The second store, `*v6`, comes after the load `v5` of the first. The C result is `out = [11.0, 22.0]` and the interpreter gives the same.

the memory kernel as a graph



`out = [11.0, 22.0]` from the c, and the same from the interpreter

the point:

AFTER computes nothing. it says: use this pointer only once those accesses are done.

The problem: order

In a pure graph, order is free. The value of a node depends on its srcs and on nothing else, so a node can be computed any time after its srcs. With memory, a `LOAD` depends on what was stored before it, and that dependency is not in its srcs.

The kernel above has two accesses to `out[0]`: a store (%8) and a load (%11). The load must see the value the store wrote. If the graph only said `LOAD(INDEX(out, 0))`, nothing would say “after that store”.

Worse, hash consing makes the plain version actively wrong. The demonstration builds a load of address 0, a store to address 0, and a second load of address 0, all with no ordering:

the hash consing trap with memory · output of `demos/walkthrough.py`

```
ld2 is ld1 (the load after the store is the same node): True
a load through AFTER(p, store) is a different node: True
sequenced program, order=id: buf = [20.0] (1 -> store 2 -> load 2 -> store 20)
sequenced program, order=dfs: buf = [20.0] (1 -> store 2 -> load 2 -> store 20)
unsequenced program, order=id: buf = [5.0, 5.0, 0.0]
```

```
unsequenced program, order=dfs: buf = [5.0, 1.0, 0.0]
the two orders disagree: with no AFTER edge the program has no single meaning
```

The first line says `ld2 is ld1`. The load after the store has the same op, the same src (`INDEX(p, 0)`) and the same arg as the load before it. So hash consing returns the **same node**. The program text said “read, store, read again”, and the graph holds one read. A program that means “read the old value, then read the new value” has become “read once”.

The fix has to be in the graph, as a src: a node that is different after the store from before it. That is `AFTER`.

AFTER

`AFTER(p, s1, s2, ...)` evaluates to the pointer `p`, unchanged, but it has the stores or loads `s1, s2, ...` as extra srcs. Two things follow.

- **It is a different node.** `INDEX(AFTER(p, st), 0)` has a different src from `INDEX(p, 0)`, so a load through it is a different node. The second line of the demo output confirms: a load through `AFTER(p, store)` is a different node: `True`.
- **It orders.** The renderer writes statements in topological order. An `AFTER` is a src of everything that uses the pointer, and the store is a src of the `AFTER`, so the store comes first. The sequenced program in the demo gives 20.0 in both of the interpreter’s visiting orders: 1, store 2, load 2, store 20.

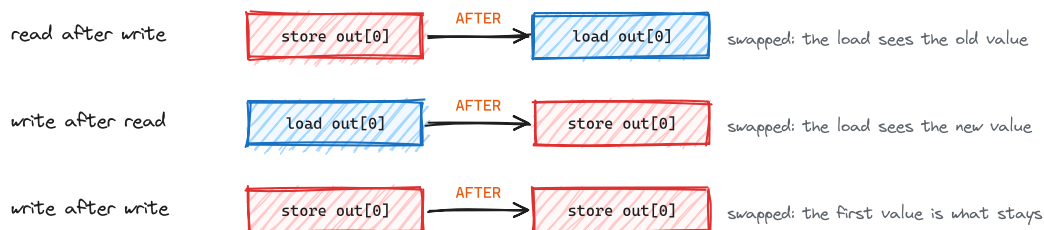
The renderer treats `AFTER` as the identity: in the C, the name of an `AFTER` node is the name of its first src. The extra srcs only fix the order, and cost no code.

The three hazards

Two memory accesses to the same place conflict if at least one of them is a store. There are three cases.

- **Read after write (RAW).** A load must see the store that came before it. The pointer for the load is `AFTER(p, store)`.
- **Write after read (WAR).** A store must not run before a load of the old value. The pointer for the store is `AFTER(p, load1, load2, ...)`: it depends on every load since the last store.
- **Write after write (WAW).** Two stores to the same buffer must run in order. The pointer for the second store is the `AFTER` made by the first.

the three hazards



the point:
each AFTER edge pins one order. without it, c or the interpreter may swap the two accesses.

A person writing graphs by hand would forget one of these. The module has a helper, `Builder`, that tracks them and writes the `AFTER` edges for you:

the builder · `uopc/builder.py`, the whole file

```
"""a tiny frontend for memory programs. you write loads and stores in order, and it
threads the AFTER
edges that keep them in that order:
    read after write: a load sees the buffer as it is after the last store to it
    write after read: a store waits for every load of the old contents
    write after write: a store waits for the previous store (its pointer is the
previous AFTER)"""
from .dtypes import I32
from .ops import Ops
from .uop import CallArg, ParamArg, UOp

class Mem:
    def __init__(self, ptr: UOp, builder: "Builder"):
        self.base, self.version, self.readers, self.last = ptr, ptr, [], None
        self.b = builder

    def load(self, i) -> UOp:
        i = i if isinstance(i, UOp) else UOp.const(i)
        ld = UOp(Ops.LOAD, (UOp(Ops.INDEX, (self.version, i)),))
        self.readers.append(ld)
        return ld

    def store(self, i, v) -> UOp:
        i = i if isinstance(i, UOp) else UOp.const(i)
        v = v if isinstance(v, UOp) else UOp.const(v)
        ptr = UOp(Ops.AFTER, (self.version, *self.readers)) if self.readers else self.version
        st = UOp(Ops.STORE, (UOp(Ops.INDEX, (ptr, i)), v))
        self.version, self.readers, self.last = UOp(Ops.AFTER, (ptr, st)), [], st
        self.b.stores.append(st)
```

```

        return st

class Builder:
    def __init__(self):
        self.stores, self.mems = [], []

    def buffer(self, slot: int, dtype, size: int) -> Mem:
        from .dtypes import Ptr
        m = Mem(UOp(Ops.PARAM, (), ParamArg(slot, Ptr(dtype, size))), self)
        self.mems.append(m)
        return m

    def alloc(self, slot: int, dtype, size: int) -> Mem:
        """call-scope scratch memory, like `float a[10];`. its contents start
        undefined."""
        from .dtypes import Ptr
        m = Mem(UOp(Ops.ALLOC, (), ParamArg(slot, Ptr(dtype, size))), self)
        self.mems.append(m)
        return m

    def body(self) -> UOp:
        """the SINK of the last store to every buffer that was written."""
        return UOp.sink(*[m.last for m in self.mems if m.last is not None])

def make_call(body: UOp, args: list, name: str = "kernel") -> UOp:
    return UOp(Ops.CALL, (body, *args), CallArg(name))

```

Each buffer has a `Mem` that remembers its current **version**, a pointer node. `load(i)` indexes the current version and records the load. `store(i, v)` makes a pointer that is the version **AFTER** all the loads since the last store, stores through it, and makes the new version **AFTER** the store. The three hazards are covered by those two methods, in about 20 lines.

`Builder.body()` collects the last store to every buffer written and makes a `SINK` of them.

This is what a front end that produces graphs would do: keep a notion of “the current state of this buffer” and thread it through. A compiler for tensors does it when it lowers a loop (week 3).

When merging loads is right

Hash consing merges loads, which looked like a trap, and in one case it is a gain. If a buffer is never written, equal loads are always equal, and one load is enough. `demons/walkthrough.py` computes a three point stencil, `out[k] = in[k-1] + in[k] + in[k+1]` for `k = 1, 2, 3`, which reads 9 times:

a stencil: nine loads written, five in the graph · output of demos/walkthrough.py

```
9 loads were written; the graph holds 5 LOAD nodes (the input is never written, so
equal loads merge safely)
void stencil(float *p0, float *p1) {
    float *v0 = p1 + 1;
    float *v1 = p0 + 0;
    float v2 = *v1;
    float *v3 = p0 + 1;
    float v4 = *v3;
    float v5 = v2 + v4;
    float *v6 = p0 + 2;
    float v7 = *v6;
    float v8 = v5 + v7;
    *v0 = v8;
    float *v9 = p1 + 2;
    float v10 = v4 + v7;
    float *v11 = p0 + 3;
    float v12 = *v11;
    float v13 = v10 + v12;
    *v9 = v13;
    float *v14 = p1 + 3;
    float v15 = v7 + v12;
    float *v16 = p0 + 4;
    float v17 = *v16;
    float v18 = v15 + v17;
    *v14 = v18;
}

out = [0.0, 7.0, 14.0, 28.0, 0.0]
```

The graph holds 5 `LOAD` nodes, one for each of `in[0]` to `in[4]`, and the C has five loads. The three neighbours of `out[2]` share two loads with the neighbours of `out[1]`, and so on. The input buffer is never written, so the builder never makes an `AFTER` for it and the loads stay merged. A buffer that is written gets an `AFTER` after each store, and its loads are distinct when they have to be.

So merging is a rule that the graph applies to both safe and unsafe cases alike, and `AFTER` is the thing that separates them.

SINK

A function body has one root. A function that stores into two buffers has two stores and nothing that depends on both of them. The second store would not be reachable from the first, and `toposort` of the root would never visit it, so the store would be silently dropped from the C.

`SINK` is a node with no value whose `srcs` are stores. It is a root that keeps several stores reachable. It is not in the syllabus, which lists `INDEX/LOAD/STORE/AFTER`, and it is the only op this module adds. The builder always ends a body with a `SINK`. `infer` checks that a `SINK` has at least one `src` and all `srcs` are `STORE`s. `tinygrad` has the same op, also called `SINK`.

What the interpreter checks and what C does not

The reference interpreter (`uopc/interp.py`) models memory as Python lists and adds two checks that C does not have.

two memory errors the interpreter reports · output of `demos/walkthrough.py`

```
read of scratch: read of memory that was never written
store at 2 into a buffer of 2: index 2 is outside a buffer of 2
```

- **Uninitialised reads.** A scratch array starts as a list of `None`. Loading a `None` raises an error. In C the same read gives whatever was on the stack.
- **Out of bounds.** An index outside `[0, size)` raises an error. In C it is undefined behaviour.

The tests only generate programs that are in bounds and initialise before they read, and the interpreter's checks are the proof of that: if a test program broke either rule, the interpreter would raise, and the test would fail.

The interpreter can also be run in two visiting orders. In `"id"` order it walks nodes by creation time, and in `"dfs"` order it walks them in the renderer's depth first order. A correctly sequenced program gives the same buffers in both. The last example of the earlier demo shows the opposite. It stores 5.0 to `b[0]`, loads `b[0]` into `b[1]`, and has no `AFTER`: the two orders give `[5.0, 5.0, 0.0]` and `[5.0, 1.0, 0.0]`. The unsequenced program has **no single meaning**, and this is what the interpreter's two orders are for: they expose a missing edge as a disagreement.

Aliasing

Two `BUFFER` arguments could be given the same memory. A call that writes `a[0]` and reads `b[0]` means something different if `a` and `b` are the same array. The graph says nothing about it, and neither does the interpreter, which gives each argument its own list. The rule for the module is the promise that `restrict` makes in C: **the caller must pass distinct, non-overlapping buffers**. Nothing checks this. A call that breaks it can give results that

differ from the interpreter's. A later module that makes kernels fast by reordering loads and stores will need exactly this promise, and tinygrad's C renderer already writes `restrict` on its buffer parameters.

How it was tested

`tests/test_memory.py` builds 200 random memory programs. Each has three global buffers (two float, one int) and one scratch array, runs 14 random steps of load, store and arithmetic with static and dynamic indices (always in range, by construction), and then runs:

- The interpreter in "id" order and in "dfs" order,
- The C compiled from the original graph,
- The C compiled from the simplified graph,

And compares every buffer, bit for bit, in all four. 1,805 checks pass. A planted bug that makes the builder skip the `AFTER` edge for stores is caught by this test: the two interpreter orders disagree on a buffer.

The same in tinygrad

The version of tinygrad read for this book has `BUFFER`, `ALLOC`, `PARAM`, `CALL`, `INDEX`, `LOAD`, `STORE`, `SINK` and `AFTER`, in the `Ops` enum in `tinygrad/uop/__init__.py`. A comment there defines `AFTER` the way this chapter does: it passes `src[0]` through and promises, in the topological sort, that anything using the `AFTER` runs after `src[1:]`. One naming difference: a comment in the tinygrad file says `ALLOC` declares unbound global storage and that value-producing calls write to `ALLOC` arguments, where the syllabus, and this book, use `ALLOC` for storage that lives for the length of one call. The idea of the op is the same: a name for memory that is not a caller's buffer.

Where to learn more

- *Engineering a compiler*, the sections on data dependences (true, anti and output dependences are what this chapter calls RAW, WAR and WAW), in the chapter on instruction scheduling.
- John hennessy and david patterson, *computer architecture: a quantitative approach*, the section on data hazards in the chapter on instruction level parallelism. The same three hazards, in a processor pipeline.
- The C standard on the `restrict` qualifier (6.7.3.1 in the c11 and c23 texts) and on the effective type and aliasing rules.

- *LLVM's memory SSA*: the LLVM documentation page `MemorySSA`, at LLVM.org/docs/MemorySSA.html, on the same problem solved with a different graph.
- The LLVM language reference, the sections on the `noalias` attribute and on the LLVM memory model.
- The tinygrad source: the comment on `AFTER` in `tinygrad/uop/__init__.py`.

Exercises

1. Use `Builder` to write a kernel that swaps `a[0]` and `a[1]` in one buffer. Print the text format. Which `AFTER` edges were made?
2. Write the same kernel by hand without the builder, with `INDEX`, `LOAD`, `STORE` and no `AFTER`. Run the interpreter in both orders. Do they agree?
3. Build a kernel that copies a buffer into itself shifted by one: `a[i+1] = a[i]` for `i = 0, 1, 2`. What does a correct sequenced version produce for `[1, 2, 3, 4]`? How many `LOAD` nodes does the graph have?
4. Why must `ALLOC` memory be written before it is read? What does the interpreter do if you break the rule? What would the C do?
5. Call the same compiled kernel twice with the same buffer for both arguments. When does the result differ from two separate buffers?

Part 5
Trust, and the real
thing

Chapter 15

How we know it works

A compiler that gives a wrong answer once in a million runs is worse than no compiler, because nobody can see the wrong answer. A test suite is the only thing that stands between the author and that outcome, so the first question to ask of a test suite is plain: for which inputs was each claim checked, and by what means?

This chapter answers that question for the compiler in this book. It lists every claim the compiler makes, names the check behind each one, shows how the checks were themselves tested, and ends with a list of what no check covers. Nothing in the earlier chapters depends on this one, but every number in them does.

The claims

The compiler makes eight claims. Each is stated as something that can fail.

1. **The contract is implemented as written.** `eval_op` in `semantics.py` gives the same answer as NumPy and as separate C code, for every op, on every kind of input (chapter 3).
2. **A graph is one object per meaning.** Building the same program twice gives the same node, and two different programs never share a node (chapter 5).
3. **simplify keeps the result.** For any program and any input, the original and the simplified program give identical values, bit for bit (chapters 8 to 11).
4. **Ranges are sound.** The `vmin` and `vmax` of a node bound every value the node can take, and a node marked `safe` never wraps (chapter 6).
5. **Folding is running.** Replacing a call by its body with the arguments substituted, and then folding constants, gives the same value as calling the compiled function (chapter 9).
6. **The C means what the graph means.** The rendered C, compiled with the flags of chapter 12, gives the interpreter's answer, including for division by zero and for memory programs (chapters 12 and 14).
7. **The text format round trips.** `loads(dumps(u))` is `u` for every graph, and a damaged file is refused with a message (chapter 5).
8. **The proofs say what they claim.** Every rule that was sent to `z3` is proved for every input, for the constants listed, and every fast math rule has an input where it fails (chapters 10 and 11).

The next four sections explain the checks. The numbers are in the table below, which is produced by `summary.py` from the saved output of every test file.

how many checks the final run made · output of summary.py

```

400,000 semantics against numpy
400,000 semantics against c
    599 rule unit cases and point checks
1,328,156 pipeline: soundness, special values, exhaustive, overflow, ranges,
folding, text, c renderer, zero divisors
    1,805 memory programs
2,130,560 total checks
    59 z3 claims, 59 as expected (8 are counterexamples to wrong rules)
    19 planted bugs, 19 caught

```

Three references

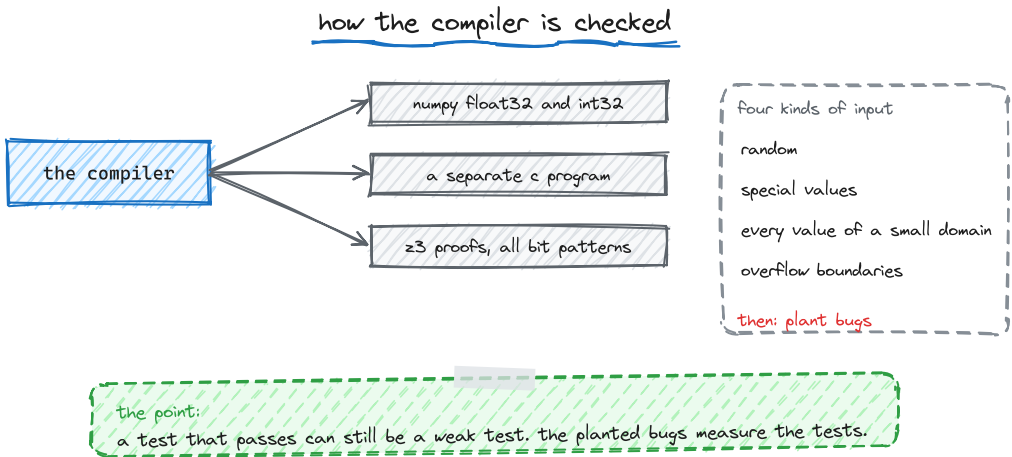
A test needs something to compare with. If the thing compared with is written by the same hand, in the same way, a misunderstanding is copied into both and the test passes. So the contract was checked against two programs that share nothing with it.

The first reference is **NumPy**. NumPy float32 arrays add, multiply, compare and cast with the machine's own instructions, and NumPy's integer arrays wrap. For the integer division ops, which NumPy does not define the way the contract does, the test widens to 64 bits, divides with Python's floor division, and narrows again.

The second reference is **C**, in the file `tests/ref_ops.c`. It was written separately from the renderer and does not use the renderer's helpers. It is compiled with `-fwrapv -ffp-
contract=off` and called through ctypes. Each op is run on 20,000 pairs of inputs, 400,000 checks for NumPy and 400,000 for C.

The two references agree with the contract bit for bit on every op except `EXP2` and `LOG2`. There the contract is Python's `math.exp2` and `math.log2` rounded to float32, and the references use their own implementations (NumPy's, and the C library's `exp2f` and `log2f`), which are allowed to differ in the last place. The largest difference measured over the test inputs was 1 unit in the last place, against both references. So these two ops are checked to within one unit in the last place, and no closer. That is stated again in the list at the end of the chapter.

The third reference is z3's own theory of floating point numbers and bit vectors. It knows nothing about this code. A rule is checked by asking it for an input where the two sides of the rule differ.



Four kinds of input

Random inputs alone do not find the bugs that matter. A random 32 bit integer is almost never `INT_MIN`, and a random float is almost never `nan` or `-0.0`, yet those are the inputs where a rule is most likely to be wrong. So the tests use four kinds of input, and each one exists because the others miss something.

Random programs. `gen_program` in `tests/common.py` builds a well typed expression over three parameters, 6 to 22 nodes, drawn from a pool so that subexpressions are shared. Its random integers come from a mix: a quarter are from a list of special values, about a third are small numbers, and the rest are spread over the whole range. The seeds are fixed, so a failure can be reproduced. The test `rewrites keep results` runs 700 programs on 40 inputs each and checks that `simplify` is idempotent as well (a second pass changes nothing).

Special values. Every parameter takes its value from the lists `SPECIAL_INT` (0, 1, -1, `INT_MIN`, `INT_MAX`, and others) and `SPECIAL_FLOAT` (both zeros, both infinities, `nan`, the smallest normal and subnormal numbers, 16777216, and others). It runs every program that `simplify` changed (329 of 400 programs) on 300 inputs each.

Exhaustive small domains. When each parameter has a small range, every input can be tried. The test builds 1,200 programs over three parameters with ranges `[0, 7]`, `[0, 3]` and `[-4, 5]`, and runs every program on all $8 * 4 * 10 = 320$ inputs. There is no sampling here. A wrong range, or a wrong rule that fires on a range, is found if any input shows it.

Overflow boundaries. The exhaustive test cannot see overflow, because in a small domain nothing overflows. The overflow test builds 300 programs of index arithmetic whose parameters have bounds such as `[INT_MIN, INT_MAX]`, `[0, 2**30]` and `[-2**30, 2**29]`,

and compares the original and the simplified program on every combination of boundary values (`INT_MIN`, `-2**30`, `-1`, `0`, `1`, `2**30`, `INT_MAX` and others) and a few random values. It is the largest layer in the table.

The chapter 11 experiment shows why the last one is needed. The guard `not d.safe` was deleted from a copy of the code. The exhaustive test still passed. So did the soundness test, the special value test, and the range test. Only the overflow test and the hand written unit cases failed. The guard is one line, and two of the test layers exist because of it.

Testing the tests

A test suite that passes proves little. It could pass because the code is right, or because the tests cannot fail. To tell the two cases apart, put a bug into the code on purpose and see whether the suite notices. This is **mutation testing**, and the planted bug is called a **mutant**.

`tests/mutants.py` holds 19 mutants. Each is a small edit: delete the `safe` guard, let `x + 0.0` simplify to `x`, make the C divide with C's own `/`, drop the `min` and `max` of a parameter from the text format, remove the ordering edge between a load and a store. Each edit is applied to a private copy of the code, a test is run, and the run must fail. `tests/mutant_matrix.py` goes further and runs all twelve test layers on every mutant, which is how the table below was made.

which test layer caught which planted bug (x means the layer failed, so it noticed) ·
output of `tests/mutant_matrix.py`

planted bug	A	B	C	D	E	F	G	H	I	J	K	L	n
division split ignores overflow	x	.	.	.	.	x	.	.	.	.	.	.	2
split remainder may overflow	x	.	.	.	.	.	.	.	.	.	.	.	1
<code>x+0.0</code> -> <code>x</code> on floats	x	.	x	x	.	.	.	.	.	.	.	.	3
float MAX is reordered	.	.	x	x	.	.	.	.	.	x	.	x	4
mod bucket off by one	x	.	.	x	x	.	.	.	.	x	.	.	4
ADD range one too small	x	.	.	.	x	.	x	.	.	.	.	.	3
FLOORMOD range one too small	.	.	x	.	x	.	x	.	.	.	.	.	3
<code>x*-1</code> -> <code>x</code>	x	.	.	.	x	x	.	.	.	x	.	x	5
<code>(x//c1)//c2</code> as <code>x//(c1+c2)</code>	x	.	.	.	.	.	.	.	.	.	.	.	1
MUL range ignores overflow	x	.	.	.	.	.	x	.	.	.	.	.	2
const fold reverses operands	.	.	x	x	x	x	.	x	x	x	.	x	8
inline picks the wrong argument	.	.	.	.	.	.	.	.	x	x	.	.	2
C: float->int cast not saturating	.	.	.	.	.	.	.	.	.	.	x	.	1
C: floor division uses C's <code>/</code>	.	.	.	.	.	.	.	.	.	.	x	x	3
C: plain <code>/</code> allowed when divisor may be 0	.	.	.	.	.	.	.	.	.	.	x	.	1
C: max of floats uses <code>fmaxf</code>	.	.	.	.	.	.	.	.	.	.	x	.	2
text format drops min/max bounds	.	.	.	.	.	.	.	.	.	x	.	.	1
hash cons key ignores sign of zero	x	x	x	x	.	.	.	.	x	.	x	.	6
AFTER dropped from the renderer's order	.	.	.	.	.	.	.	.	.	.	.	x	1

columns: A unit, B hash, C sound, D special, E exhaust, F overflow,
G ranges, H fold, I text, J render, K zerodiv, L memory

```

caught by layer: A 9, B 1, C 5, D 5, E 5, F 3, G 3, H 3, I 3, J 8, K 2, L 6
caught by exactly one layer: 6 of 19

```

All 19 mutants were caught by at least one layer. Three things in the table are worth reading.

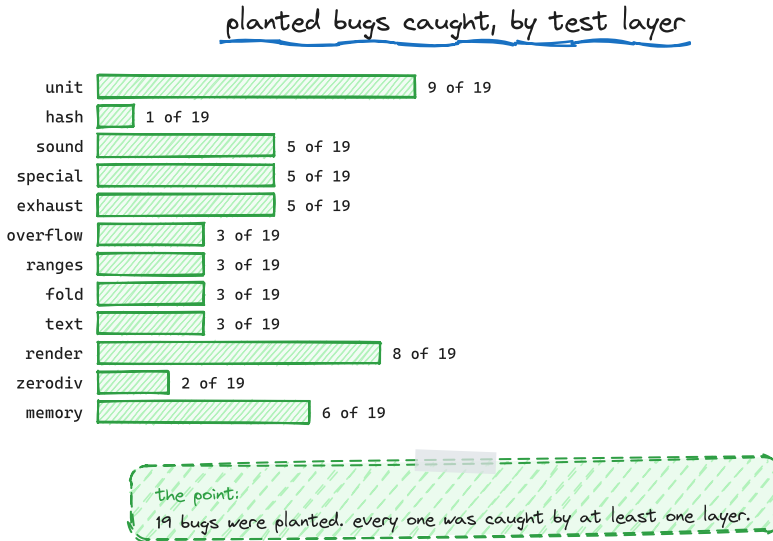
Six bugs are caught by exactly one layer. If that layer were removed, the bug would be invisible. `split remainder may overflow` is found only by the unit cases, which hold the one input that exposes it. `plain / allowed when divisor may be 0` is found only by the test that renders divisions whose divisor range includes 0 and runs the compiled code on every combination. A bug that only one test can see is a sign that the suite is thin there, and the right response is a second, independent test.

The layers do different work. The soundness, special value and exhaustive tests find wrong rules. The range test finds wrong ranges, and the overflow test finds rules that ignore wraparound. The render test finds wrong C. The memory test finds wrong ordering. `const fold reverses operands` is found by eight layers, because folding sits under almost everything. `AFTER dropped from the renderer's order` is found by one, because only memory programs have `AFTER`.

The first suite was much weaker. The first version of the tests caught 9 of the 16 bugs planted at that time. The early counts here (9, 14, 16, 17) come from the author's notes, not from a saved run, so they are the one unverified number in this chapter. One of the seven that got through was an entry whose text to change did not exist in the file, so it tested nothing and counted as a miss. The other six were real gaps. The suite passed on code that simplified `x + 0.0` to `x`, on code that reordered the arguments of a float `MAX`, and on a split rule that let a remainder overflow. Each survivor was followed by a test written to catch it. The count went 9, 14, 16, 17 and then 19 as survivors were fixed and as more mutants were added. Two survivors are worth naming. `x * -1 -> x` slipped past the test first chosen for it, and the hand written point checks now include a case for the constant -1. `plain / allowed when divisor may be 0` slipped past the renderer comparison, which did not exercise that case, and the zero divisor test now does.

The matrix also found a layer that caught nothing. The hash consing test originally built each random program twice and checked that both results were the same node. No mutant fails that check. The check could not detect a hash key that merges two different constants, such as `0.0` and `-0.0`, because merging them makes more things equal, not fewer. The test now builds a structural key for every node without using the table, and checks both directions: nodes with equal keys are one object, and one object never has two keys. It now catches the sign of zero mutant.

One more limit of this method. Mutation testing measures how well the suite finds the bugs on the list. It says nothing about bugs that nobody thought to plant. 19 mutants is a small sample. The list was written by the author of the code, and bugs that the author cannot imagine are the ones that stay.



Proofs

A test checks some inputs. A proof checks all of them. For a rewrite rule over 32 bit integers there are **232 values of `x`**, or 264 for a rule over two variables. No test can try them all. A solver can, because it does not try inputs one at a time. It reasons about the bits.

z3 is a solver from microsoft research. It can be given a claim about bit vectors or floating point numbers, and it answers one of three things: there is an input that breaks the claim (and here it is), there is none, or it gave up. To prove that a rule `L -> R` is valid, the script `proofs.py` asks z3 for an input where `L` and `R` differ. If the answer is “none”, the rule holds for every input. The claim is then settled by the solver’s search, not by examples.

The saved output has 59 claims. 51 are proved for every input. 8 are the opposite on purpose: they are wrong rules, and each comes with the input that breaks it.

The wrong rule	The input z3 produced
<code>x + 0.0 -> x</code>	<code>x = -0.0</code> (the sum is <code>+0.0</code>)
<code>x * 0.0 -> 0.0</code>	<code>x = -inf</code> (the product is <code>nan</code>)
<code>x * (1/x) -> 1.0</code>	<code>x = nan</code>
<code>x + (-x) -> 0.0</code>	<code>x = +inf</code> (the sum is <code>nan</code>)
<code>max(x, y) -> max(y, x)</code> on floats	<code>x = -0.0</code> , <code>y = +0.0</code>
<code>(x + y) + z -> x + (y + z)</code>	A triple of large values with different exponents
<code>(a*4 + b) // 4</code> split without the guard	<code>a = 1073741824</code> , <code>b = -2147483648</code>
<code>(a*24 + b) // 8</code> split without the guard	<code>a = 457266860</code> , <code>b = 1929388032</code>

The first six are the reason the strict float rule set (chapter 10) is as small as it is. The last two are the reason the split rule has a guard (chapter 11).

A proof has limits, and they matter here.

- The proof is about one rule with one constant, or a few constants. The book proves `(x*3)*5 -> x*15` and three other pairs, not every pair of constants. The rule in the code takes any constants. What connects the two is the argument in the chapter, plus the random and exhaustive tests of the rule with other constants.
- The split rule for divisors that are not powers of two could not be proved in bit vectors. z3 did not finish `(a*6 + b) // 3` in 120 seconds. So those cases are proved over mathematical integers, with the assumption that nothing wraps. That assumption is exactly what the `safe` flag is meant to guarantee. The link between the `safe` flag and real int32 behaviour is checked by tests, not proved: 61,840 values of nodes marked `safe` were computed with exact integers, and none had wrapped.
- The proofs use the contract's definition of each op, written again in z3 terms. A mistake in that rewrite would give a proof of the wrong statement. The z3 floating point theory follows the standard, and the float rules were checked with it, but the integer division written in z3 is the author's translation of `semantics.py`. The tests against C cover that gap from the other side.

What is not verified

This is the list the front of the book promised. Each item says what is missing and why.

- **EXP2 and LOG2 are checked to within one unit in the last place, not exactly.** The contract is Python's function rounded to float32. The C library's `exp2f` and `log2f` may differ from it in the last bit. A program that folds `exp2` of a constant and a program that

computes it at run time can therefore differ in the last bit. The random program tests leave these two ops out for that reason.

- **Aliasing is not checked.** `execute` and `Program` take buffers, and nothing stops a caller from passing the same buffer twice. Exercise 14.5 shows a kernel whose result changes when that happens.
- **Values are scalars.** Every node holds one number, and `shape` is a constant. The real compiler has shapes, and its rewrites depend on them. Nothing about shapes is tested because there is nothing to test.
- **The text format matches the course draft in shape only.** The draft was seen in a public post, not in a specification. Its op names and keys may differ from this book's in places, and it may change. Chapter 5 says what is known.
- **SINK is an addition.** The syllabus does not list it. It is a small op that groups several stores, and the memory tests cover it.
- **No undefined behaviour checker was run.** The C is compiled with `-fwrapv` so signed overflow wraps, and the division helpers avoid the one trapping case, `INT_MIN / -1`. But the C was not run under the undefined behaviour sanitizer, because its runtime library was missing from the machine used. The interpreter checks every index against the buffer size. The compiled C does not, so a wrong index writes outside memory.
- **The contract is not proved against the C standard.** It is checked on 400,000 inputs for each op, with the flags that are used. Another compiler, another processor, or other flags could behave differently. The generated file contains `_Static_assert(FLT_EVAL_METHOD == 0)`, which stops the build on a platform where float arithmetic is done at higher precision.
- **One machine.** Every test and every timing ran on one x86-64 machine with clang 18.1.3. Appendix d names it.
- **Termination and confluence are not proved.** The engine has a step limit, and `RewriteLimit` stops a loop. Chapter 8 measured that the result depends on the rule order for 6 of 600 programs. Both results were correct on every input tried, but the claim “the order does not matter” is false.
- **Double rounding.** Computing a float32 operation in double precision and rounding to float32 afterwards gives the correctly rounded float32 result for `+`, `-`, `*`, `/` and square root. This is a published theorem (roux, 2014, in the reading list of chapter 2) and was not re-proved here. It was tested on the 400,000 checks of claim 1.
- **One rule is covered only by hand written cases.** The rule `cast to own dtype` never fired in the random programs, because the generator never builds a cast to the same type. The unit cases cover it.
- **The tests share an author with the code.** The references are independent of each other and of the code, but the choice of what to test is one person's. The planted bugs are one person's too.

Where to learn more

- Xuejun yang, yang chen, eric eide and john regehr, *finding and understanding bugs in C compilers*, pldi 2011. Csmith, a random program generator, found hundreds of bugs in gcc and LLVM. The generator in this chapter is a toy version of the same idea.
- Nuno lopes, david menendez, santosh nagarakatte and john regehr, *provably correct peephole optimizations with alive*, pldi 2015, and nuno lopes, juneyoung lee, chung-kil hur, zhengyang liu and john regehr, *Alive2: bounded translation validation for LLVM*, pldi 2021. Both prove LLVM's rewrite rules with a solver, as `proofs.py` does at small scale.
- Leonardo de moura and nikolaj bjørner, *z3: an efficient smt solver*, tacas 2008. And the free online book *programming z3* by nikolaj bjørner, anh-dung phan and lars fleckenstein. The z3 guide at microsoft.github.io/z3guide has the floating point and bit vector theories used here.
- Yue jia and mark harman, *an analysis and survey of the development of mutation testing*, IEEE transactions on software engineering, 2011. The standard survey.
- David goldberg, *what every computer scientist should know about floating-point arithmetic*, acm computing surveys, 1991. It is the best explanation of why float rules need proofs.
- Xavier leroy, *formal verification of a realistic compiler*, communications of the acm, 2009. Compcert, a C compiler whose passes are proved correct in coq. The other end of the scale from this chapter.
- The hypothesis library for Python, at hypothesis.readthedocs.io, for random testing with shrinking, which this module's tests do by hand.
- tinygrad tests its symbolic rules in the same spirit. The file `tinygrad/uop/validate.py` turns a `UOp` graph into a z3 expression, `test/null/test_uop_symbolic.py` has a `check_equal_z3` helper that uses it, and `test/external/fuzz_symbolic_div.py` generates 700 random division and modulo expressions, simplifies each, and asks z3 whether the result can differ from the original (it skips an expression when z3 takes longer than a second). These files were present in the commit read for this book.

Exercises

1. Read the table of mutants and layers. Which layer caught the most bugs? Which caught the fewest? Is the layer that caught the fewest a layer you would remove?
2. Use z3 to check `max(x, y) == max(y, x)` for 32 bit integers and for float32 (with `max(a, b)` meaning `a > b ? a : b`). Which is proved and which is refuted? Find the refuting input.

3. Use `z3` to check $-(x - y) \rightarrow y - x$ for 32 bit integers, with $x - y$ meaning $x + (-y)$. Is it proved? Include the case `x = INT_MIN`. Then find an input that breaks it for `float32`.
4. In a copy of the code, change the rule `max by range` so that it fires only when `a.vmin > b.vmax` instead of `a.vmin >= b.vmax`. Run the unit, pipeline and memory tests. What do they say? Write one assertion that would notice the change.
5. In a copy of the code, change the rule `x*1` so that it also fires when the constant is 2. Which test layers fail? Use `tests/mutant_matrix.py` as a model.

Chapter 16

The same ideas in tinygrad

The compiler in this book is a small copy of an idea. The idea is the one that tinygrad is built on, and the course this module belongs to ends with students writing something close to tinygrad. So this chapter does three things. It maps each piece of the book's compiler to the file in tinygrad that does the same job. It lists the places where tinygrad does something different, with the evidence. And it says what the next weeks of the syllabus add, so that the reader knows where this module stops.

Everything said about tinygrad here was checked by reading or running the repository at commit `06e57e3`, the one cloned for this book. tinygrad changes fast: ops are renamed, files move, and rules are added every week. Look for the names, not the line numbers, and expect some of them to have moved by the time you read this.

The map

This book	tinygrad file	Name to look for
The op list	<code>tinygrad/uop/__init__.py</code>	The <code>Ops</code> enum
UOp , hash consing	<code>tinygrad/uop/ops.py</code>	<code>UOp</code> , <code>UOpMetaClass</code> , its <code>ucache</code>
<code>eval_op</code> , the contract	<code>tinygrad/uop/ops.py</code>	<code>python_alu</code> , <code>exec_alu</code>
Type rules of legal graphs	<code>tinygrad/uop/spec.py</code>	<code>spec_shared</code> , checked when <code>SPEC</code> is above 1
<code>vmin</code> , <code>vmax</code>	<code>tinygrad/uop/ops.py</code>	<code>vmin</code> , <code>vmax</code> , <code>_min_max</code>
<code>UPat</code> , <code>PatternMatcher</code>	<code>tinygrad/uop/ops.py</code> , <code>tinygrad/uop/upat.py</code>	The two classes, and the pattern compiler
<code>graph_rewrite</code>	<code>tinygrad/uop/ops.py</code>	<code>graph_rewrite</code>
Constant folding	<code>tinygrad/uop/symbolic.py</code>	<code>fold_const_alu</code>
Simplification rules	<code>tinygrad/uop/symbolic.py</code>	<code>symbolic_simple</code> , <code>symbolic</code> , <code>sym</code>
Division and modulo rules	<code>tinygrad/uop/divandmod.py</code>	<code>fold_divmod_general</code>
Proofs with z3	<code>tinygrad/uop/validate.py</code>	<code>uops_to_z3</code>
The C renderer	<code>tinygrad/renderer/</code> <code>cstyle.py</code>	<code>CStyleLanguage</code> , <code>ClangRenderer</code>
The compiler and cache	<code>tinygrad/device.py</code> , <code>tinygrad/runtime/support/</code> <code>compiler_cpu.py</code>	<code>Compiler.compile_cached</code> , <code>ClangCompiler</code>
Loading and running	<code>tinygrad/runtime/</code> <code>ops_cpu.py</code> , <code>tinygrad/</code> <code>runtime/support/elf.py</code>	<code>CPUProgram</code> , <code>jit_loader</code>
Memory ops	<code>tinygrad/uop/__init__.py</code>	<code>BUFFER</code> , <code>ALLOC</code> , <code>PARAM</code> , <code>INDEX</code> , <code>LOAD</code> , <code>STORE</code> , <code>AFTER</code>

There is one name that differs for no deep reason. This book calls the reciprocal op `RECIP` , as the syllabus does. In the commit read here the enum member is `RECIPROCAL` . tinygrad also has ops this module does not: `SUB` , `FDIV` , `SQRT` , `SIN` , `POW` , shifts and bitwise ops, and two integer division pairs, `FLOORDIV` and `FLOORMOD` as in this book, and `CDIV` and `CMOD` , which truncate toward zero as C does.

What the real thing writes

Here is the real compiler on the program of chapter 1, `relu(x*y + 1)`, written with tensors on four values. The command is `DEV=CPU DEBUG=4`, which makes tinygrad print the source it compiles. `DEBUG` is documented in `docs/env_vars.md` of the repository.

the c that tinygrad writes for `relu(x*y+1)` on four values, default optimisations · output of `demos/tinygrad_demo.py`

```
typedef float float4 __attribute__((aligned(16),ext_vector_type(4)));
void E_4(float* restrict data0_4, float* restrict data1_4, float* restrict data2_4)
{
    float4 val0 = (*((float4*)((data1_4+0))));
    float4 val1 = (*((float4*)((data2_4+0))));
    float alu0 = ((val0[0]*val1[0])+1.0f);
    float alu1 = ((val0[1]*val1[1])+1.0f);
    float alu2 = ((val0[2]*val1[2])+1.0f);
    float alu3 = ((val0[3]*val1[3])+1.0f);
    float alu4 = ((alu0<0.0f)?0.0f:alu0);
    float alu5 = ((alu1<0.0f)?0.0f:alu1);
    float alu6 = ((alu2<0.0f)?0.0f:alu2);
    float alu7 = ((alu3<0.0f)?0.0f:alu3);
    (*((float4*)((data0_4+0))) = (float4){alu4,alu5,alu6,alu7};
}
result: [11.0, 41.0, 91.0, 161.0]
```

And the same program with `NOOPT=1`, which turns the optimiser off:

the same program with optimisations off · output of `demos/tinygrad_demo.py`

```
void E_4(float* restrict data0_4, float* restrict data1_4, float* restrict data2_4)
{
    for (int Lidx0 = 0; Lidx0 < 4; Lidx0++) {
        float val0 = (*(data1_4+Lidx0));
        float val1 = (*(data2_4+Lidx0));
        float alu0 = ((val0*val1)+1.0f);
        float alu1 = ((alu0<0.0f)?0.0f:alu0);
        *(data0_4+Lidx0) = alu1;
    }
}
```

Compare each with the function of chapter 12, which renders the same graph. Three differences are visible in the listings.

The pointers are `restrict`. Every buffer argument is declared `float* restrict`, a promise to the C compiler that no two arguments overlap. Chapter 14 left this out and said that a caller must pass distinct buffers. tinygrad makes the promise in the source, which lets clang reorder and vectorise freely, and puts the burden on whatever launches the kernel.

There is a loop, or a vector. The book's compiler renders loop free functions, because loops arrive in week 3 of the syllabus. In the second listing the elementwise kernel is a loop over `Lidx0`. In the first, the optimiser split the loop by four and turned the four iterations into one vector of four floats, then wrote the arithmetic out lane by lane. That is the first optimisation of weeks 5 and 6, and it shows what it looks like in C.

The max is written the other way round. tinygrad's C is `(alu0<0.0f)?0.0f:alu0`, which is `(a<b)?b:a`. The book's contract is `a>b ? a : b`. For ordinary numbers the two agree. For `nan` they do not, and the next section has the measurement.

Where tinygrad does something else

Each item below is a place where a student who reads tinygrad after this module will see a different choice. None of them is a bug in either one. They are decisions, and the module made the opposite decision on purpose, for the reason given in the chapter shown.

Float simplification is not strict. Chapter 10 keeps the float rules that are valid for every input, and puts the rest in a separate fast math list. tinygrad does not separate them. Run on a float variable whose range is minus infinity to infinity, its simplifier gives:

simplifying float expressions in tinygrad · output of demos/tinygrad_demo.py

```
f + 0.0      -> f
f * 0.0      -> const 0.0
f - f        -> const 0.0
f * 1.0      -> f
(f * 3.0) * 5.0 -> (f*15.0)
```

Four of those five are in the book's fast math list. The fifth, `f * 1.0 -> f`, is a strict rule. `f + 0.0 -> f` is wrong for `f = -0.0`. `f * 0.0 -> 0.0` is wrong for infinity and `nan`. `f - f -> 0.0` is wrong for the same two. Reassociating `(f*3.0)*5.0` to `f*15.0` is wrong for some finite values, as exercise 10.4 shows. z3 refuted the first three forms in chapter 15, and tinygrad applies all four by default. The effect on a real tensor program is measurable, on the signed zero:

-0.0 plus 0.0 through tensors · output of demos/tinygrad_demo.py

```
tinygrad, tensor + tensor of zeros : [0.0, 1.0]   sign of first: 1.0
tinygrad, tensor + the scalar 0.0  : [-0.0, 1.0]  sign of first: -1.0
numpy, float32 array + 0.0         : [0.0, 1.0]   sign of first: 1.0
```

Adding a tensor of zeros gives `+0.0`, as IEEE requires and as NumPy does. Adding the scalar `0.0` gives `-0.0`, which is what happens when the addition has been removed. The difference is one bit. For machine learning it almost never matters, and a compiler that must

be fast on every benchmark has good reason to take the rules. The cost is that two programs that are the same on paper can give different bits. The book's choice, strict by default and fast by request, keeps the tests of chapter 15 simple: a rule either preserves every bit, and is tested for that, or it is in the fast list and is not.

The max of a nan. The book's `MAX(a, b)` is `a > b ? a : b`, so a `nan` in the first argument gives the second. tinygrad's C is `(a<b) ? b : a`, so a `nan` in the first argument comes through. The same relu graph then gives different answers:

`relu(x*y+1)` in tinygrad when `x` is `nan` · output of `demos/tinygrad_demo.py`

```
tinygrad, x = [nan, 1, -2, 0], y = 1: [nan, 2.0, 0.0, 1.0]
```

In the book, `relu(nan)` is `0.0`, as exercise 1.3 shows. In tinygrad it is `nan`. The contract could have been written either way, as long as it is written down, tested against C, and used by the folder and the renderer in the same way. What matters is that the folder and the renderer cannot disagree. In this module one function, `eval_op`, decides for both.

Constants keep their sign and their type. The chapter 5 trap is real in tinygrad, and it is handled in the data type. `tinygrad/dtype.py` has a class `ConstFloat`, a subclass of `float`, whose docstring says it “distinguishes -0.0 from 0.0 and where `nan == nan`”. The cache key of a node also contains `type(arg)`, so `True` and `1` are different nodes. The measurement:

constants as nodes in tinygrad · output of `demos/tinygrad_demo.py`

```
const 0.0 is const -0.0 : False
const nan is const nan  : True
const 1 is const True   : False
```

The book's `_key_arg` solves the same problem with a key made of the type and the bits. The two designs reach the same answer, which is a good sign that the trap is the natural place to look.

Division by zero. The book's `x // 0` is `0`, and the folder, the renderer and the interpreter agree. tinygrad's integer division rule raises `ZeroDivisionError` when the divisor is the constant zero (chapter 9), so a program that contains a division by zero it never runs can fail to compile. It also keeps the truncating ops `CDIV` and `CMOD`, which have the sign behaviour of C, next to the floor ops.

Checking the types. The book checks every node as it is built, which costs a little on every build and makes an ill typed graph impossible. tinygrad checks its spec only when the variable `SPEC` is above 1, so that production runs do not pay, and tests run with it on. `SPEC=3` also checks shapes. This is a speed decision that the book had room to avoid because its graphs are small.

The rewrite engine. The book's `graph_rewrite` matches a node after its sources are done and then rewrites the result again until nothing applies. `tinygrad`'s `graph_rewrite(sink, pm, ctx, bottom_up, name, bpm, walk, enter_calls)` does that by default, and with `bottom_up=True` it applies the matcher to a node before its sources instead. It stops a runaway rewrite with `RuntimeError("infinite loop in graph_rewrite (stack too big)")`, or, when a replacement depends on the node it replaces, with a message that says the rewrite stalled. The book's engine counts steps and raises `RewriteLimit`. `tinygrad` also does not rewrite the body of a `CALL` unless the caller passes `enter_calls=True`. The book inlines the call and rewrites the result, which is the simpler idea.

Patterns are compiled. `tinygrad` turns each `UPat` into Python source (`tinygrad/uop/upat.py`) so that matching does not walk a tree of objects at every node. The book's matcher walks the tree. The result is the same, and `tinygrad`'s is faster by a constant factor that matters when a model has millions of nodes.

The compile step. The book calls `clang` to build a shared library and loads it with `ctypes`. `tinygrad` asks `clang` for an object file for the target, with `-march` taken from a target string, and loads the object itself, with a loader in `tinygrad/runtime/support/elf.py` that maps the code into executable memory. The cache is a function of the compiler's `cachekey` and the source, in `Compiler.compile_cached`. The book's cache key also contains the compiler's version, so that a cache built by one compiler is never used by another.

Shapes. Every node in this module holds one number. In `tinygrad` a node has a shape, and a great deal of the compiler is about shapes: how to turn a reshape and an expand into index arithmetic. That is the week 3 material.

A path through the source

This is an order that works for a reader who has done the chapters of this book. It takes the files from the shortest to the largest.

1. Run the demo of this chapter yourself, with `DEV=CPU DEBUG=4`, on a program of your own. Read the source it prints.
2. Read the `Ops` enum in `tinygrad/uop/__init__.py` from the top to `CONST`. Its comments group the ops: loads and stores, unary, binary and ternary math, control flow.
3. Read `UOpMetaClass` and the top of `UOp` in `tinygrad/uop/ops.py`. You know all of it from chapters 4 and 5.
4. Read `python_alu` and `exec_alu`, then `fold_const_alu` in `tinygrad/uop/symbolic.py`. This is chapter 9 in about ten lines.

5. Read `symbolic_simple` in `tinygrad/uop/symbolic.py`. Every rule is a pattern and a result. Find the ones that this book has, and the ones it does not. Each is a candidate for an exercise: is it valid for every float, or only for non-nan, or only for a range?
6. Read `ClangRenderer` and the parts of `CStyleLanguage` that it uses in `tinygrad/renderer/cstyle.py`. Compare with `render.py` of the book.
7. Read `tinygrad/uop/validate.py` and the test `test/null/test_uop_symbolic.py`. This is where tinygrad proves its own rules.
8. Read `tinygrad/uop/divandmod.py`. It is chapter 11 with more cases. The file is long, and it is the best place to see which of the book's ideas survive contact with real index expressions.

What the next weeks add

The syllabus continues as follows, and the files named are where tinygrad does the same job. The correspondence was read from the repository, not tested.

- **Week 3, loops and movement ops.** `RESHAPE`, `EXPAND`, `PERMUTE`, `PAD` and `FLIP` are movement ops that only exist in tensor level graphs (`tinygrad/uop/movement.py`). `RANGE` and `END` are loops. `tinygrad/schedule/rangeify.py` and `tinygrad/schedule/indexing.py` turn movement ops into ranges and index arithmetic. The integer rules of chapter 11 are what simplify the indices it builds.
- **Week 4, splitting into kernels.** The `schedule` directory decides where one kernel ends and the next starts. The syllabus adds a temporary buffer op, `STAGE`, which is in the `Ops` enum.
- **Weeks 5 and 6, memory hierarchies and fast kernels.** `tinygrad/codegen/opt` has `heuristic.py`, `postrange.py` and `search.py`. The vector in the first listing of this chapter came from an upcast option, `OptOps.SPLIT` with an `UPCAST` axis.
- **Weeks 7 and 8, threads, GPUs, tensor cores.** `tinygrad/codegen/gpudims.py` maps loops to GPU dimensions, `tinygrad/renderer/tc.py` describes tensor cores, and `tinygrad/renderer/cstyle.py` also holds the CUDA, metal, opencl and hip renderers, which are subclasses of the same `CStyleLanguage` the C renderer uses.
- **Weeks 9 and 10, real models and autodiff.** `Tensor.backward` in `tinygrad/tensor.py` calls `gradient`, defined in `tinygrad/mixin/op.py`, and `tinygrad/mixin/gradient.py` has `compute_gradient`, which builds the backward graph from the forward one. `tinygrad/llm` holds a language model.

The syllabus warns that the course “is an exercise in slop management”. This module is the first place where slop can enter, in the contract, in the hash consing, and in the rules, and chapter 15 is how this book tried to keep it out.

where to learn more

- The tinygrad repository, github.com/tinygrad/tinygrad, and its `docs` directory. `docs/env_vars.md` lists `DEBUG` and `DEV`. `SPEC` and `N00PT` are read in `tinygrad/uop/ops.py` and `tinygrad/helpers.py`.
- The course syllabus that this module follows, quoted in chapter 1.
- The tinygrad tests named in chapter 15, which are the best examples of how the authors check a rule.
- *Engineering a compiler*, chapter 5 on intermediate representations, for the same graph ideas in a conventional compiler.

Exercises

1. Run `DEV=CPU DEBUG=4` on a tinygrad program that computes `(a + b) * (a + b)` for two tensors of four values, once as it is and once with `N00PT=1`. How many additions of `a` and `b` does each C function contain? Why is each sum multiplied by itself instead of being computed twice, given what chapter 5 says about hash consing?
2. Write the graph for the `relu` of this module that returns `nan` for a `nan` input, using only the ops of the module. Evaluate it on `nan`, `-2.0`, `0.0` and `-0.0`. How does it differ from the original on signed zero?
3. Find the clang flags tinygrad uses on the CPU, in `tinygrad/runtime/support/compiler_cpu.py`, and list them next to the book's. Which of the book's flags have no counterpart there, and what does each one protect against?
4. In tinygrad, build `f + 0.0` for an unbounded float variable and simplify it. Then run the tensor version with `-0.0` as in the measurement above. What does the answer say about the order in which the rules fire, and what would the book's compiler return for the same program?
5. Open `tinygrad/uop/divandmod.py` and find, in the first lines of `fold_divmod_general`, the check for a division by zero and the check that the quotient is constant. Which chapters of this book explain each?

The back
Answers, references and
measurements

Appendix A

Answers to the exercises

Every answer below was produced by running code. The program is `exercises/solutions.py`, and its saved output is `out/solutions.txt`. Each answer is an `assert` in that program, so a wrong answer stops the run. The final run made 385 such checks and all passed. A few answers that depend on the clock or on the machine, and so are not the same on every run, include their output.

A label like **4.2** means chapter 4, exercise 2.

Chapter 1

1.1 the graph has 5 nodes: `p`, `q`, `p+q`, `(p+q)*(p+q)` and the final `add`. The tree has 11, because `p+q` (3 nodes) appears three times.

1.2 the function is `relu`. Its body has 3 statements and then a `return`.

1.3 `0.0`. `max(a, b)` is `a > b ? a : b`, and `nan > 0.0` is false, so the second argument, `0.0`, is returned. The compiled C function gives the same.

1.4 two lines, one `param` and one `add`. The `add` names `%0` twice, and `loads` of the text returns the very same node as `x + x`.

Chapter 2

2.1 `wrap32(2**31 + 7)` is `-2147483641`, and `wrap32(-2**31 - 7)` is `2147483641`.

2.2 `-1.0` is `1 01111111 000000000000000000000000`: sign 1, exponent 127, fraction 0. The 32 bits are `0xbf800000`.

2.3 `n = 16777216`, which is `2**24`. Every integer below it is representable, and above it the gap between floats is 2. So `float32(n + 1)` rounds back to `n`.

2.4 with floor division, `-7 // 2 = -4` and `-7 % 2 = 1`, and `2 * -4 + 1 = -7`. With truncating division, the quotient is `-3` and the remainder is `-1`, and `2 * -3 + -1 = -7`. The identity holds for both, with different pairs.

2.5 `(a+b)+c` is `1.0` and `a+(b+c)` is `0.0`, because `-1e8 + 1.0` rounds back to `-1e8`. Other triples that differ: `1e20`, `-1e20`, `1.0` gives `1.0` and `0.0`. `2**24`, `1.0`, `1.0` gives `16777216.0` and `16777218.0`. And three random values in `[0, 1)`, `0.956034243106842`, `0.9478274583816528` and `0.05655136704444885`, give `1.9604130983352661` and `1.9604129791259766`.

Chapter 3

3.1 `-3` and `1`. And `3 * -3 + 1 = -8`.

3.2 `max(0.0, -0.0)` is `-0.0`, and `max(-0.0, 0.0)` is `0.0`. Neither zero is greater than the other, so each call returns its second argument. Swapping the arguments changes the sign of the result, so the rewrite engine must not reorder the arguments of a float `MAX`. The rule `canonical order` skips it for that reason.

3.3 the values are `0`, `14` and `-34`. The division is only reached when `x != 0`, but `WHERE` computes both branches, which is why the contract makes `x // 0` defined.

3.4 `a / b` is `a * recip(b)`. For `7.0 / 3.0` this gives `2.3333334922790527`, and NumPy's float32 division gives `2.3333332538604736`. They are different. The pair `a = 74.24996185302734`, `b = 92.31017303466797` also differs: `0.8043529391288757` against `0.8043529987335205`. The multiplication rounds `1/b` first and then rounds the product, so it rounds twice.

3.5 `a <= b` is false when `a` is `nan`, but `not (b < a)` is true, because `b < nan` is false. For `a = nan`, `b = 1.0` the two differ.

Chapter 4

4.1 the graph has 4 nodes: `x`, `1`, `x + 1` and the product. The add is used twice.

4.2 the message is `ADD: needs two srcs of the same dtype`. `x.cast(F32) + y` builds.

4.3 the hand written text is five lines after the header: `a param`, `a const 2`, `a mul`, `a const 3` and an `add`. `loads` of it returns the same node as `P(0) * 2 + 3`. `dumps` writes the same five lines in the order of the topological sort.

4.4 a `UOpError` when the `CALL` is built: `CALL: PARAM slot 2 has no argument`.

4.5 `UOp.const(1) is UOp.const(1) is True`. `UOp.const(1) is UOp.const(True) is False`, because the key is the type and the value, and `bool` and `int` are different types.

Chapter 5

5.1 before `simplify` the two products are different nodes, because `p+q` and `q+p` are different nodes. After `simplify` the rule `canonical order` rewrites `q+p` to `p+q`, and the two products are the same node.

5.2 the graph has 31 nodes. As a tree it would have `2**31 - 1 = 2147483647`.

5.3 the id is different. The table holds weak references, so when the last reference to a node goes, the node and its table entry go. Building the graph again makes new nodes with new, larger ids.

5.4 a float constant written as `value=1` is accepted, and the node is the same as before, because `float("1")` is `1.0`. An int constant written as `value=1.5` is refused: `line 3: bad or missing key invalid literal for int() with base 10: '1.5'`.

5.5 with `float(arg)` as the key, the first failure in `test_rules_unit.py` is `1/0 (float): got const -inf, wanted const inf`. The constants `0.0` and `-0.0` have become one node, since Python says they are equal and gives them the same hash, so the test that builds `1/0.0` and `1/-0.0` finds the wrong infinity.

Chapter 6

6.1 `i*4 + j` is in `[0, 63]` and `(i*4 + j) % 8` is in `[0, 7]`. Both ranges are exact: running all 64 inputs gives exactly those sets.

6.2 `[-3, 15]`.

6.3 `x // 5` is in `[-2, 1]` and `x % 5` is in `[0, 4]`. Both are exact over the 17 inputs.

6.4 for `x` in `[-50000, 50000]` the product is not safe, because `50000 * 50000 = 2500000000`, which is above `INT_MAX`. Its range is reported as the whole `int32` range. For `x` in `[-46340, 46340]` it is safe, because `46340**2 = 2147395600` fits and `46341**2` does not. Its range is then `[-2147395600, 2147395600]`: the interval rule multiplies the end points and does not know that a square cannot be negative. That is sound, and not tight.

6.5 `x % 1` has the range `[0, 0]`, the point `0`. The rule `range is a point` turns it into the constant `0`.

Chapter 7

7.1 the rule function runs once, for `a * a`, with `x` bound to the node `a`. It does not run for `a * b`, because the second name `x` must be the same node as the first. Both calls return `None`, and the `fired` counter stays empty because only a returned new node counts.

7.2 the pattern for `max(x, x)` uses the name `x` twice, so both sources must be the same node, compared with `is`. `a.max(a)` matches and `a.max(b)` does not.

7.3 the pattern matches any `ADD`. On `a + b` it stores `x = a` and `y = b`. On `a + a` it stores `x = a` and `y = a`, and the pattern with two different names does not require them to differ.

7.4 $x - x$ is $x + \text{neg}(x)$, and the rule $x+(-x)$ (int) in `symbolic.py` already rewrites it to `0`. `simplify(a - a)` is the constant `0`.

7.5 $x = 0.0$ breaks it: `0.0 * recip(0.0) = 0.0 * inf = nan`. So does $x = \text{inf}$. A finite counterexample is $x = 2000.9886474609375$ (that is `1.95409047603607177734375 * 2**10`): `x * recip(x) = 0.9999999403953552`, not `1.0`.

Chapter 8

8.1 the trace has two steps. Step 1: the `ADD` node `a + 0` becomes `a`, by the rule `x+0 (int)`. Step 2: the `MUL` node, now `a * 1`, becomes `a`, by `x*1`.

8.2 `p + 0` is one node, so it is rewritten once. The memo then answers for both uses. The counter shows one firing of `x+0 (int)`.

8.3 `RewriteLimit`: more than 50 rewrite steps; busiest rules: `[('distribute', 23), ('factor', 22)]`. Each rule undoes the other.

8.4 for seed 551 the two orders give different results, 23 nodes and 24 nodes. The rules `(a*c1+b)//c2` and `(x+c1)*c2` fired only in the reversed order.

8.5 two rules that both match `x * 4`, one rewriting it to `(x+x)+(x+x)` and the other to `(x*2)*2`, give an `ADD` when the first is listed first and a `MUL` when the second is. Both are correct on the inputs tried, and the nodes differ. With the pair from the hint, `x*2 -> x+x` and `x+x -> x*2`, each undoes the other: `RewriteLimit`: more than 100 rewrite steps; busiest rules: `[('double as add', 50), ('add as double', 49)]`.

Chapter 9

9.1 the result is `10`. The nodes fold in the order `ADD`, then `FLOORDIV`, then `MUL`: the left operand of the product first, because the engine finishes a node's sources from left to right.

9.2 the call with the arguments `1.0` and `3.0` simplifies to the constant `4.0`. The compiled function from chapter 1 returns `4.0` for the same arguments.

9.3 the result is the constant `0`. The contract defines `x // 0` as `0`, so folding does not raise.

9.4 the int case with range `[0, 100]` folds to `0`: the range of $x * 0$ is the point `0`. The float stays a `MUL`, because `inf * 0.0` and `nan * 0.0` are `nan`, and `-3.0 * 0.0` is `-0.0`, none of them `0.0`.

9.5 after `simplify` the op is still `CALL`. The inline rule is guarded to calls whose body is a value, and a body that stores to memory is not one.

Chapter 10

10.1 `z3` says `unsat`: there is no input where `x * 2.0` and `x + x` differ in `float32`, so the rule is proved. Multiplying by two is exact (it only changes the exponent), and so is adding `x` to itself, including for infinities, `nan`, both zeros and overflow.

10.2 with `x = y = 1.5`, `-(x - y)` is `-0.0` (bits `80000000`) and `y - x` is `+0.0` (bits `00000000`). So the rewrite is wrong for floats. Exercise 15.3 shows that it is right for integers.

10.3 no. `nan * nan >= 0` is false. `z3` finds `nan` as the counterexample for all `x`, and finds none when `nan` is excluded. Any rule that depends on `x * x >= 0` has to exclude `nan` or accept that `nan` behaves differently. A rule that removes an absolute value after a square relies on the same fact, so the `nan` case has to be argued on its own. This module has no absolute value op, so there is no such rule to check.

10.4 `x = 1.138539433479309` gives `(x*3)*5 = 17.078092575073242` and `x*15 = 17.07809066772461`. For the constants `2.0` and `3.0`, `z3` says `unsat`:

`x * 2.0 * 3.0 -> x * 6.0` is valid. The first multiplication is exact, so the second is the only rounding, the same as in `x * 6.0`.

10.5 `simplify(x + 0.0)` returns the `ADD` unchanged. With `fast_math=True` it returns `x`.

Chapter 11

11.1 for `j` in `[0, 7]`, `(i*8 + j) // 8` simplifies to `i`. For `j` in `[0, 8]` it simplifies to `i + j // 8`, because at `j = 8` the quotient is `i + 1`. Dropping `j` would be wrong there. Both results were checked on every input.

11.2 `(x // 2) // 3 == x // 6` holds for every `x` in `[-50, 50]`, and so does `(x // -2) // 3 == x // -6`. The identity holds whenever the outer divisor is positive, even if the inner one is negative, so the guard of the rule (both positive) is stricter than it needs to be. It fails when the outer divisor is negative: `(x // 2) // -3` differs from `x // -6` at `x = -47, -41, -35, -29, -23, -17` and more.

11.3 for `a` in `[0, 2**29 - 1]` and `b` in `[0, 3]` the split is applied and the result is `a`. For `a` in `[0, 2**29]` it is not applied, because `a * 4` can then be `2**31`, which wraps. The largest top is `2**29 - 1`: it is the largest `a` for which `a * 4 + 3` is at most `2147483647`. For tops `2**30` and `2**31 - 1` the split is not applied either.

11.4 with a known range, `i` in `[0, 1000]`, `(i * 3) % 3` simplifies to `0`: it is covered by the split. With no range it is left alone, and that is correct: at `i = 1431655766`, `i * 3` wraps to `2`, and `2 % 3` is `2`, not `0`.

11.5 z3 refutes it over 32 bit integers. A counterexample is $x = -1431655766$: $x * 3$ wraps to -2 , and $-2 // 3$ is -1 , not x . The guarded split is proved because its guard says that $a * k * c$ and the sum cannot wrap.

Chapter 12

12.1 with no ranges the line is `int32_t v0 = floordiv_i32(p0, p1);`. With x in $[0, 100]$ and y in $[1, 10]$ it is `int32_t v0 = p0 / p1;`. For a non negative dividend and a positive divisor, truncating and floor division agree, and the divisor is never 0 or -1 .

12.2 the C is `float v0 = (0.0f > p0) ? 0.0f : p0;`. For `nan` it returns `nan`, and the contract says `nan`. `fmaxf(0.0f, nan)` returns `0.0`, so it does not match the contract. The ternary does.

12.3 the C is `int32_t v0 = f32_to_i32(p0);`. The results are `1e10 -> 2147483647`, `-1e10 -> -2147483648`, `nan -> 0` and `-0.5 -> 0`, equal to the contract on all four.

12.4 without `-fwrapv`, $x + 1 > x$ at `INT_MAX` gives 0 at `-00` and 1 at `-01`, `-02` and `-03`. With `-fwrapv` it gives 0 at every level.

`x + 1 > x` at `INT_MAX`, by optimisation level · output of `exercises/solutions.py`

```
-00: without -fwrapv f(INT_MAX) = 0      with -fwrapv = 0
-01: without -fwrapv f(INT_MAX) = 1      with -fwrapv = 0
-02: without -fwrapv f(INT_MAX) = 1      with -fwrapv = 0
-03: without -fwrapv f(INT_MAX) = 1      with -fwrapv = 0
```

12.5 the constants are written `(-2147483647)` and `2147483647`. `2147483648` does not fit in an int, so a C compiler gives the literal a wider type. Both `2147483647` and `(-2147483647)` fit. Only `INT_MIN` needs the form `(-2147483647 - 1)`.

Chapter 13

13.1 one compile and one cache hit. The second `Program` finds the file the first one made.

13.2 adding `-O3` is a new compile: the compiles go from 1 to 2 and the hits stay at 1. The flags are part of the key.

13.3 a Python loop of a million `ctypes` calls costs hundreds of nanoseconds per call. A C loop of a million iterations inside one call costs well under a nanosecond per iteration. The ratio is in the hundreds to thousands, depending on the machine and the run. This run:

a million calls from python against a loop in c · output of exercises/solutions.py

```
python loop: 711 ns per call; one ctypes call running a c loop of 1,000,000: 0.64 ns
per iteration; ratio about 1,118x (this machine, this run)
```

13.4 the error reads `clang failed:` followed by the path of the cached `.c` file, then `:1:1: error: unknown type name 'float'; did you mean 'float'?`. The file named is the generated source in the cache directory.

13.5 with the runtime's flags, which include `-fwrapv`, every level gives `0`. With `-fwrapv` removed, `-O0` gives `0` and `-O1`, `-O2` and `-O3` give `1`.

the same function with the runtime's flags, with and without `-fwrapv` · output of exercises/solutions.py

```
-O0: with -fwrapv 0    without 0
-O1: with -fwrapv 0    without 1
-O2: with -fwrapv 0    without 1
-O3: with -fwrapv 0    without 1
```

Chapter 14

14.1 the text format shows two `AFTER` nodes in the body. `%7 = after %0, %3, %6` makes the first store wait for both loads, and `%10 = after %7, %9` makes the second store wait for the first. The interpreter gives `[20, 10]` and so does the compiled C.

14.2 the two visiting orders disagree. In creation order the result is `[20, 10]`. In the renderer's order it is `[20, 20]`, because the second store's load runs after the first store. Nothing in the graph says otherwise, which is what `AFTER` is for.

14.3 the sequenced version produces `[1, 1, 1, 1]`, and so does the compiled C. The graph has 3 `LOAD` nodes, one for each `i`: they read different indexes, so they are different nodes and are not merged.

14.4 a read of `ALLOC` memory that was never written has no defined value. The interpreter marks fresh scratch memory as unwritten and raises `read of memory that was never written`. The C would read whatever was on the stack, a different value on different runs, and would give no error.

14.5 with two buffers the kernel gives `[0, 1, 2]`. With the same buffer passed twice it gives `[1, 1, 1]`. The second load reads what the first store wrote, because nothing orders a load of one buffer against a store to the other. The result differs whenever a store through one argument writes an element that a load through the other argument reads later.

Chapter 15

15.1 the layer with the most catches is `unit`, with 9 of the 19 bugs. The layer with the fewest is `hash`. A low count is not a reason to remove a layer. The `hash` consing test is the only direct test of the property that every other test assumes, and `zerodiv` is the only layer that catches the bug `plain / allowed` when `divisor` may be `0`. The table in chapter 15 has the counts of the final run.

15.2 for 32 bit integers `z3` says `unsat`, so `max(x, y) == max(y, x)` is proved. For `float32` it finds a counterexample, for example `x = nan` and `y = -0.99998...e-126`: `max(nan, y)` is `y` and `max(y, nan)` is `nan`.

15.3 for 32 bit integers `z3` says `unsat` for all `x` and `y`, and also with `x = INT_MIN`, so `-(x - y) -> y - x` is proved even though it wraps. For `float32` it is wrong at `x = y`: `-(x - y)` is `-0.0` and `y - x` is `+0.0`.

15.4 the `unit`, `pipeline` and `memory` tests all pass on the weaker optimiser, because a rule that fires less often never gives a wrong answer. An assertion that notices: for `x` in `[5, 9]`, `simplify(x.max(5))` is `x`. It is `True` for the real code and `False` for the weaker one.

15.5 with `x*1` also firing for the constant 2, the layers `unit`, `exhaust`, `overflow`, `render` and `memory` fail. The layers `sound`, `special` and `fold` do not, because their random programs seldom have a multiplication by the constant 2 where the rule can fire. The bug is caught, and not by every layer.

Chapter 16

16.1 with the default optimisations the C has 4 additions, one for each lane of the vector, and each sum is multiplied by itself (`alu0*alu0`). With `NOOPT=1` it has 1. The two `a + b` in the Python expression are one node, because `UOp` is hash consed, so the sum is computed once and used twice.

16.2 `max(0.0, x)` returns `nan` for `nan`, and for `-0.0` it returns `-0.0`, while `max(x, 0.0)` returns `0.0` for both. For `-2.0` and `0.0` the two give `0.0`. So the first form, `max(0.0, x)`, is the one that matches `tinygrad` on `nan`, and the two forms differ on the sign of zero.

16.3 `tinygrad`'s flags are `-c -O2 -fPIC -ffreestanding -fno-math-errno -nostdlib -fno-ident` and a `--target` built from the target string, with the source on standard input. The book's are `-O2 -fwrapv -ffp-contract=off -fno-fast-math -std=c11 -shared -fPIC`. Both have `-O2` and `-fPIC`. The book alone has `-fwrapv` (signed overflow wraps instead of being undefined), `-ffp-contract=off` (no fused multiply add, which changes rounding), `-fno-fast-math` (no float optimisation that changes values; the default, written down), `-std=c11` and `-shared` (it makes a shared library, where `tinygrad` makes an object file).

16.4 in the book's compiler `simplify(f + 0.0)` is not `f`, and at `f = -0.0` the sum is `+0.0`. In tinygrad `simplify(f + 0.0)` is `f`. The measurement of chapter 16 agrees: adding a tensor of zeros keeps the IEEE answer, and adding the scalar `0.0` loses the sign. In the first case the zeros are data in a buffer, not a constant in the graph, so the rule has nothing to match.

16.5 the first lines of `fold_divmod_general` raise `ZeroDivisionError` when the divisor is the constant zero, which chapter 9 discusses, and test whether `x // y` is constant for the ranges of `x` and `y`, which is chapter 6 (ranges) used by the rules `range is a point` and `x%c inside one bucket` of chapter 11.

Appendix B

Op reference

Every op of the compiler on one page, then the meaning of each op on each dtype. The code is `uopc/ops.py` for the list, `uopc/uop.py (infer)` for the type rules and `uopc/semantics.py` for the meaning. A node is `UOp(op, src, arg)`. The dtypes are `bool`, `i32` and `f32`, and pointers to them.

The ops and their type rules

Op	Srcs	Arg	Result
PARAM	None	ParamArg(slot, dtype, vmin, vmax)	The dtype in <code>arg</code> . A value dtype is in ALU space, a pointer dtype in MEM space. <code>vmin</code> and <code>vmax</code> bound an int
CALL	body, arg0, arg1, ...	CallArg(name)	The dtype of the body. A body may not contain a <code>CALL</code> or a <code>BUFFER</code> . Every <code>PARAM</code> slot used in the body needs an argument of the same dtype
CONST	None	A Python <code>bool</code> , <code>int</code> or <code>float</code>	The dtype of the Python type. An int must fit in 32 bits, and a float is rounded to float32
CAST	One value	The target dtype	The target dtype
NEG	One int or float	None	The dtype of the src
RECIP, EXP2, LOG2	One float	None	Float
ADD	Two values of one dtype, int or float	None	The dtype of the srcs
MUL, MAX	Two values of one dtype	None	The dtype of the srcs
CMPLT, CMPNE	Two values of one dtype	None	Bool
FLOORDIV, FLOORMOD	Two ints	None	Int
WHERE	A bool, then two values of one dtype	None	The dtype of the two values
BUFFER	None	ParamArg(slot, Ptr(dtype, size))	A pointer in MEM space. A global buffer, passed to a call as an argument, never inside a body

Op	Srcs	Arg	Result
ALLOC	None	ParamArg(slot, Ptr(dtype, size))	A pointer in MEM space. Scratch memory of the call, its contents undefined at the start
INDEX	A pointer, a scalar i32	None	A pointer to one element, in MEM space
LOAD	One INDEX	None	The element dtype, in ALU space
STORE	An INDEX , a value of the element dtype	None	Void
AFTER	A pointer, then loads or stores	None	The same pointer. Orders the access after the listed ones
SINK	One or more STORE	None	Void. Keeps several stores alive. An addition to the course spec

ADD is not defined on bool. On bool, MUL is and, MAX is or, and CMPNE is xor.

The recursive properties

Property	Where it comes from	Used for
dtype	infer , from the srcs and the arg	Refusing ill typed graphs when a node is built
shape	infer . () for a value, (size,) for a pointer	The second week of the syllabus. Trivial in this module
space	ALU for values and MEM for pointers	Refusing a pointer where a value is needed
vmin , vmax	Computed for int and bool nodes from the srcs	Range rules (chapter 11)
safe	True when no int op in the node's graph can wrap	The guard of the split rule

What each op means

Every rule here is total. No input raises or leaves a result undefined. A float result is rounded to float32, ties to even. `nan` is one value, with one bit pattern. The same table is code in `semantics.py`.

Op	On i32	On f32	On bool
ADD	The sum, wrapped to 32 bits	The sum, rounded	Not defined
MUL	The product, wrapped	The product, rounded	And
MAX(a, b)	<code>a > b ? a : b</code>	<code>a > b ? a : b</code> . For <code>nan</code> and for <code>0.0</code> against <code>-0.0</code> , this returns <code>b</code>	Or
CMPLT	<code>a < b</code>	<code>a < b</code> , false if either is <code>nan</code>	<code>a < b</code> with false below true
CMPNE	<code>a != b</code>	<code>a != b</code> , true if either is <code>nan</code>	Xor
NEG	<code>-a</code> , wrapped, so <code>-INT_MIN</code> is <code>INT_MIN</code>	Flips the sign bit, so <code>-0.0</code> and <code>0.0</code> swap	Not defined
FLOORDIV	Rounds toward minus infinity. <code>x // 0</code> is <code>0 . INT_MIN // -1</code> is <code>INT_MIN</code>	Not defined	Not defined
FLOORMOD	Has the sign of the divisor. <code>x % 0</code> is <code>x</code>	Not defined	Not defined
RECIP	Not defined	<code>1 / a</code> , with <code>1 / 0.0</code> equal to <code>+inf</code> and <code>1 / -0.0</code> to <code>-inf</code> . <code>nan</code> stays <code>nan</code>	Not defined
EXP2 , LOG2	Not defined	Python's <code>math.exp2</code> and <code>math.log2</code> rounded to float32. <code>exp2</code> of a large value is <code>inf</code> . <code>log2(0)</code> is <code>-inf</code> , <code>log2</code> of a negative is <code>nan</code> . Within 1 ulp of C's <code>exp2f</code> and <code>log2f</code>	Not defined
WHERE(c, a, b)	<code>a</code> if <code>c</code> else <code>b</code> , and both <code>a</code> and <code>b</code> are computed	The same	The same

And the casts, from the dtype of the src to the target:

From	to	bool	i32	f32
bool		The same	0 or 1	0.0 or 1.0
i32		<code>x != 0</code>	The same	Rounded to the nearest float32
f32		<code>x != 0.0</code> , so nan gives true	Truncates toward zero, saturates at <code>INT_MIN</code> and <code>INT_MAX</code> , and nan gives 0	The same

Derived ops are not ops: `SUB(a, b)` is `ADD(a, NEG(b))`, and `DIV(a, b)` is `MUL(a, RECIPI(b))`, which can differ in the last bit from a correctly rounded division (exercise 3.4).

Appendix C

Commands, counts and proofs

Everything in the book can be rerun. This appendix lists the commands, the exact number of checks that the final run made, and every claim sent to z3.

What you need

Python 3.10 or newer, NumPy, the z3 solver (`pip install z3-solver`), and clang. The book was built with the versions in appendix D. Chapter 16 and its demo also need a clone of tinygrad. Set `TINYGRAD_DIR` to its folder, or put it where `demos/tinygrad_demo.py` looks. Nothing else in the book needs it.

The folders

Folder or file	What is in it
<code>uopc/</code>	The compiler: 1,517 lines of Python in 13 files. <code>ops.py</code> and <code>dtypes.py</code> are the vocabulary, <code>semantics.py</code> is the contract, <code>uop.py</code> is the node, <code>rewrite.py</code> and <code>symbolic.py</code> and <code>fold.py</code> are the rewriting, <code>textfmt.py</code> is the text format, <code>render.py</code> is the C writer, <code>runtime.py</code> runs it, <code>interp.py</code> is the reference interpreter and <code>builder.py</code> threads memory order
<code>tests/</code>	Every test file, <code>ref_ops.c</code> (the separate C reference), <code>mutants.py</code> and <code>mutant_matrix.py</code>
<code>short/</code>	The 13 programs of the short path, <code>s01_see.py</code> to <code>s13_prove.py</code> . Step 14 uses the tinygrad output of <code>demos/tinygrad_demo.py</code> . Run each one from inside this folder. Their output is saved in <code>out/short/</code>
<code>demos/</code>	The programs whose output the chapters quote
<code>exercises/solutions.py</code>	The answers of appendix A, each one an <code>assert</code>
<code>proofs.py</code>	The z3 claims listed below
<code>bench.py</code>	The benchmarks of chapter 13 and appendix D
<code>out/</code>	The saved output of every command, and <code>out/snips/</code> which holds one file per section of an output
<code>excerpts/</code>	Listings cut out of the source by <code>make_excerpts.py</code> , so no listing in the book is typed by hand

The commands

Run them from the `code` folder.

<code>./run_all.sh</code>	everything below, in order, saving each output into out/
<code>python3 tests/test_semantics.py</code>	the contract against numpy and against c
<code>python3 tests/test_rules_unit.py</code>	hand written cases and point checks of every rule
<code>python3 tests/test_pipeline.py</code>	the large layers: soundness, special values, exhaustive, ranges, ...
<code>python3 tests/test_memory.py</code>	random memory programs, c against the interpreter
<code>python3 tests/mutants.py</code>	plant each of the 19 bugs and check that a test fails
<code>python3 tests/mutant_matrix.py</code>	run every test layer on every planted bug
<code>python3 proofs.py</code>	the z3 claims
<code>(cd short && python3 s01_see.py)</code>	the short path programs, s01 to s13, one at a time
<code>python3 demos/run_demos.py</code>	demos 1 to 8
<code>python3 demos/walkthrough.py</code>	the worked examples of chapters 6, 8 and 14
<code>python3 demos/number_facts.py</code>	the number facts of chapter 2
<code>python3 demos/errors.py</code>	the error messages of chapters 4 and 5
<code>python3 demos/matcher.py</code>	the matcher traces of chapter 7
<code>python3 demos/fold.py</code>	the folding demo of chapter 9
<code>python3 demos/order.py</code>	the rule order experiment of chapter 8
<code>python3 demos/tinygrad_demo.py</code>	the tinygrad output of chapter 16
<code>python3 exercises/solutions.py</code>	the answers of appendix A
<code>python3 bench.py</code>	the benchmarks (run it on an idle machine)
<code>python3 summary.py</code>	add up the checks

`test_pipeline.py` takes `--quick` for smaller sizes and `--only=NAME` for one layer (hash, sound, special, exhaustive, fast, overflow, ranges, fold, text, render, zerodiv).

The counts

This is the output of `summary.py` on the saved results of the final run. A check is one comparison of two values, or one assertion. Every number comes from the test output, not from this book.

the checks of the final run · output of `summary.py`

```

400,000 semantics against numpy
400,000 semantics against c
  599 rule unit cases and point checks
1,328,156 pipeline: soundness, special values, exhaustive, overflow, ranges,
folding, text, c renderer, zero divisors
  1,805 memory programs
2,130,560 total checks
   59 z3 claims, 59 as expected (8 are counterexamples to wrong rules)
   19 planted bugs, 19 caught

```

The z3 claims

Each line is one claim. `proved` means z3 found no input where the two sides differ, so the rule holds for every input. `refuted` means z3 found one, and it is shown. The claims on `float32` use z3's floating point theory. The claims on integers use 32 bit vectors, except for the rows marked `(integers, guarded)`, which were proved over mathematical integers with the no-wrap guard assumed, as chapter 11 explains.

every claim sent to z3 · output of `proofs.py`

```
integer rules
(x*3)*5 -> x*15                proved
(x+3)+5 -> x+8                  proved
(x+3)*5 -> x*5+15               proved
(x*-2)*7 -> x*-14               proved
(x+-2)+7 -> x+5                 proved
(x+-2)*7 -> x*7+-14             proved
(x*4)*4 -> x*16                 proved
(x+4)+4 -> x+8                  proved
(x+4)*4 -> x*4+16               proved
(x*1)*-1 -> x*-1                proved
(x+1)+-1 -> x+0                 proved
(x+1)*-1 -> x*-1+-1             proved
x%2 == x-(2*(x//2))              proved
x%3 == x-(3*(x//3))              proved
x%4 == x-(4*(x//4))              proved
x%7 == x-(7*(x//7))              proved
x%16 == x-(16*(x//16))           proved
(x//2)//3 -> x//6 (c1,c2 > 0)    proved
(x//4)//4 -> x//16 (c1,c2 > 0)   proved
(x//3)//5 -> x//15 (c1,c2 > 0)   proved
(x//8)//2 -> x//16 (c1,c2 > 0)   proved
x//1 and x%1                     proved
x//-1 and x%-1                   proved
x//0 == 0 and x%0 == x (the total contract) proved
INT_MIN // -1 == INT_MIN (wraps)  proved

the divisible-terms split:
(a*4+b)//4 -> a*1+b//4 (guarded) proved
(a*4+b)%4 -> b%4 (guarded)        proved
(a*4+b)//4 -> a*1+b//4 WITHOUT the guard refuted: a=1073741824, b=-2147483648
(a*24+b)//8 -> a*3+b//8 (guarded) proved
(a*24+b)%8 -> b%8 (guarded)        proved
(a*24+b)//8 -> a*3+b//8 WITHOUT the guard refuted: a=457266860, b=1929388032

the same split for divisors that are not powers of two
(a*6+b)//3 -> a*2+b//3 (integers, guarded) proved
(a*6+b)%3 -> b%3 (integers, guarded)        proved
(a*15+b)//5 -> a*3+b//5 (integers, guarded) proved
(a*15+b)%5 -> b%5 (integers, guarded)        proved
(a*35+b)//7 -> a*5+b//7 (integers, guarded) proved
(a*35+b)%7 -> b%7 (integers, guarded)        proved
```

```

(a*6+b)//6 -> a*1+b//6 (integers, guarded) proved
(a*6+b)%6 -> b%6 (integers, guarded) proved

range rule: x inside one bucket of c
x in [8,11] -> x%4 == x-8 proved
x in [8,11] -> x//4 == 2 proved
x in [12,17] -> x%6 == x-12 proved
x in [12,17] -> x//6 == 2 proved
x in [-10,-6] -> x%5 == x-(-10) proved
x in [-10,-6] -> x//5 == -2 proved

float32 rules
x*1.0 -> x proved
x+(-0.0) -> x proved
x+0.0 -> x (the unsafe one) refuted: f=-0.0
x*0.0 -> 0.0 (the unsafe one) refuted: f=-oo
(x+y)+z -> x+(y+z) (the unsafe one) refuted: h=-1.5078133344650268554...
x*(1/x) -> 1.0 (the unsafe one) refuted: f=NaN
-(-x) -> x proved
x*-1.0 -> -x proved
-(x*y) -> x*(-y) proved
max(x,y) -> max(y,x) on floats (the unsafe one) refuted: g=+0.0, f=-0.0
x+(-x) -> 0.0 on floats (the unsafe one) refuted: f=+oo
max(x,x) -> x on floats proved
x+y == y+x proved
x*y == y*x proved

```

Appendix D

Measurements

Every performance number in the book was measured with `bench.py`, on the machine below, with the compiler and library versions below. The machine is a small virtual machine with two cores. Your numbers will differ. What should not change much is the ratios between them, and the shape of each curve.

the machine and tool versions · output of `machine_info.py`

```
cpu:      Intel(R) Xeon(R) Processor @ 2.10GHz
cores:    2 (as seen by this machine)
memory:   8031 MiB
os:       Linux-6.18.44-fc-v64-x86_64-with-glibc2.39
python:   3.13.16 | numpy 2.5.3 | z3 5.1.0
clang:    Ubuntu clang version 18.1.3 (1ubuntu1)
tinygrad: commit 06e57e3
```

One function, three ways of running it

The first benchmark is chapter 13's. It runs `relu(x*y + 1)` on floats in three ways: with the Python interpreter, with the compiled function called once for each element through `ctypes`, and with the loop inside C. The per element cost of the `ctypes` call is the cost of crossing from Python to C, not the cost of the arithmetic.

one function, three ways of running it · output of `bench.py`

python interpreter (evaluate elements, scaled to 200,000)	3820 ns per element	(measured on 20,000)
compiled C, one ctypes call each	584 ns per element	(200,000 elements)
compiled C, loop inside C elements)	0.96 ns per element	(20,000,000)
ratio interpreter / C loop: 3,966x ctypes call / C loop: 606x		

clang's time against the size of the function

Compile time grows with the number of statements, and it grows faster at `-O1` and `-O2` than at `-O0`. The table runs a chain of adds and multiplies on one float, at three optimisation levels, and takes the median of three runs, including the time to start clang.

clang's time against the size of the function · output of bench.py

statements clang)	-00	-01	-02 (seconds, median of 3, includes starting
10	0.029	0.027	0.028
100	0.029	0.034	0.037
1000	0.044	0.085	0.086
5000	0.144	0.658	0.701

Building and simplifying graphs

The third benchmark is about the data structure. 300 random index expressions are built and simplified. The memory is measured with `tracemalloc` before anything else keeps the graphs alive, and the last line is the hash consing example of chapter 5.

building and simplifying index graphs · output of bench.py

```
300 random index expressions: 4,311 nodes if each is counted alone, 2,106 distinct
nodes in memory
memory while building them (python objects, measured by tracemalloc): 870 KiB, 423
bytes per distinct node
simplify on all 300: 50 ms, 85,490 nodes per second (nodes counted per expression,
before simplifying); 2,735 nodes after
p=p+p repeated 18 times: 19 nodes in the graph; as a tree it would have 524,287
```

How each benchmark was run

- The Python interpreter number is measured on 20,000 elements and scaled to 200,000, because running 200,000 takes long enough to be a nuisance.
- The loop in C is run on 20,000,000 elements so that the time is large against the clock's resolution.
- The compile times include starting the compiler. Each is the median of three runs of `clang -c` writing to `/dev/null`, so the runtime's cache plays no part in them.
- The memory numbers are for Python objects only, from `tracemalloc`, and do not include the interpreter's own memory.
- All benchmarks run in one process, one after the other, on an idle machine. A second job on a two core machine changes the timings, so `run_all.sh` runs `bench.py` last.
- The million call loop of exercise 13.3 is a separate, smaller measurement and varies from run to run. Appendix A shows one run of it.

Appendix E

Reading list and study plan

Every chapter ends with the reading that goes with it. This appendix puts the same references in one place, grouped by what they teach, with a note on how to use each one. Then it gives an order in which to work through the book.

How to use this list

Read a book chapter, do its exercises, and only then open the reference. The references are deeper than the book, and they are easier to follow once a question is already in your head. None is required to read the next chapter of this book.

Compilers in general

- *Engineering a compiler* by keith cooper and linda torczon. The textbook that these chapters keep returning to. Chapter 5 is on intermediate representations, chapter 9 on data flow analysis, and the chapters on optimisation cover peephole and algebraic simplification, constant propagation and data dependences.
- *Crafting interpreters* by robert nystrom, free at craftinginterpreters.com. The clearest start for anyone to whom “parse”, “tree” or “bytecode” are new words.
- Cornell’s CS 6120 self guided course, at cs.cornell.edu/courses/cs6120/2020fa/self-guided. A course with video lessons on intermediate representations and optimisation.
- *Ssa-based compiler design*, free as a draft at pffalcon.github.io/ssabook/latest. The introduction explains static single assignment form, which the text format of this module is.
- The LLVM language reference at LLVM.org/docs/LangRef.html. The best real example of a typed SSA IR.
- Chris lattner and vikram adve, *LLVM: a compilation framework for lifelong program analysis and transformation*, cgo 2004, and chris lattner’s chapter on LLVM in *the architecture of open source applications*, free at aosabook.org.

Numbers on a machine

- David goldberg, *what every computer scientist should know about floating-point arithmetic*, at docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html. Read sections 1 and 2 first, and the section on optimizers after chapter 10.

- Randal Bryant and david o'hallaron, *computer systems: a programmer's perspective*. Chapter 2 for integers and floats, chapter 7 for linking.
- Henry warren, *hacker's delight*. Integer tricks, including division by constants.
- Jean-michel muller and others, *handbook of floating-point arithmetic*. The long reference. You do not need it for this module.
- Pierre roux, *innocuous double rounding of basic arithmetic operations*, journal of formalized reasoning, 2014. The proof that chapter 2 relies on.
- William kahan's notes at people.eecs.berkeley.edu/~wkahan. The principal author of IEEE 754 on why floating point behaves as it does.
- The IEEE 754-2019 standard. It is behind a paywall, and goldberg's paper is the free substitute.

Undefined behaviour and C

- John regehr, *a guide to undefined behavior in C and C++*, at blog.regehr.org/archives/213. Three parts. Read part 1 first.
- Chris lattner, *what every C programmer should know about undefined behavior*, three posts on the LLVM blog.
- The C standard. N3096, a late draft of c23, is free. Use it for integer conversions, `restrict`, `FLT_EVAL_METHOD` and floating point contraction.
- The clang user's manual at clang.llvm.org/docs/UsersManual.html, for `-fwrapv`, `-ffp-contract`, `-ffast-math` and the optimisation levels. The sanitizer pages describe `-fsanitize=undefined`.
- The *compiler explorer* at godbolt.org. Type a small C function and read the assembly at `-O0` and `-O2`.

Graphs, hash consing and analysis

- Jean-christophe filliâtre and sylvain conchon, *type-safe modular hash-consing*, ML workshop 2006. A short paper on doing it correctly.
- Donald knuth, *the art of computer programming*, volume 4A, on binary decision diagrams, which use the same table of unique nodes.
- Patrick cousot and radhia cousot, *abstract interpretation*, popl 1977, the paper behind the ranges of chapter 6, and anders møller and michael schwartzbach, *static program analysis*, free at cs.au.dk/~amoeller/spa, which is the readable treatment.
- The LLVM source file `llvm/lib/IR/ConstantRange.cpp`, the same idea as chapter 6 with wrapped ranges.

Rewriting and simplification

- The LLVM instcombine pass in `llvm/lib/Transforms/InstCombine`, and the header `llvm/include/llvm/IR/PatternMatch.h`. The same idea as chapter 7, at scale.
- *Term rewriting and all that* by franz baader and tobias nipkow, for termination and confluence. Nachum dershowitz, *termination of rewriting*, journal of symbolic computation, 1987.
- The egg paper and website at egraphs-good.github.io, for equality saturation, which chapter 8 mentions as the alternative to ordered rules.
- *Types and programming languages* by benjamin pierce, chapters 5 to 7, for substitution and beta reduction. And *partial evaluation and automatic program generation* by jones, gomard and sestoft, free at itu.dk/people/sestoft/pebook, for specialising a program to part of its input.
- Sven verdoolaege, *isl: an integer set library for the polyhedral model*, 2010, and the Halide paper by jonathan ragan-kelley and others, pldi 2013, for the full theory of index arithmetic with floor division and modulo.

Memory

- *Engineering a compiler*, on data dependences. John hennessy and david patterson, *computer architecture: a quantitative approach*, on the same three hazards in a processor pipeline.
- LLVM's `MemorySSA` page at LLVM.org/docs/MemorySSA.html, and the sections of the LLVM language reference on `noalias` and the memory model.

Trust

- Xuejun yang, yang chen, eric eide and john regehr, *finding and understanding bugs in C compilers*, pldi 2011.
- Nuno lopes and others, *provably correct peephole optimizations with alive*, pldi 2015, and *Alive2: bounded translation validation for LLVM*, pldi 2021.
- Leonardo de moura and nikolaj bjørner, *z3: an efficient smt solver*, tacas 2008. The z3 guide at microsoft.github.io/z3guide. The free book *programming z3* by bjørner, phan and fleckenstein.
- Yue jia and mark harman, *an analysis and survey of the development of mutation testing*, IEEE transactions on software engineering, 2011.
- Xavier leroy, *formal verification of a realistic compiler*, communications of the acm, 2009.
- The hypothesis library, at hypothesis.readthedocs.io.

tinygrad

- The repository at github.com/tinygrad/tinygrad and its `docs` folder. Chapter 16 names the files to read and the order to read them in.

A plan for working through the book

The book has 16 chapters. This plan splits them into six sessions, each of which ends with something that runs. How long a session takes depends on how much of the code you type yourself.

1. **The problem and the numbers (chapters 1 to 3).** Run `demos/run_demos.py` and read the output of each demo before the chapter that explains it. Do every exercise of chapters 2 and 3 by hand first, then check with `solutions.py`. You are ready to move on when you can say, without looking, what `x // 0`, `max(nan, 0.0)` and `INT_MIN // -1` are in this module and why.
2. **The graph (chapters 4 to 6).** Write your own `UOp` with hash consing from the description of chapter 5, before reading the listing. Then compare. Build the text format. You are ready to move on when `loads(dumps(u)) is u` holds for your graphs.
3. **Rewriting (chapters 7 to 9).** Write the matcher and the engine, then the folder. Run your engine on `(a + 0) * 1` and on a program of your own, and trace it by hand. You are ready to move on when `relu(2.0, 3.0)` folds to a constant in your engine.
4. **The rules (chapters 10 and 11).** Pick three rules from the chapters, prove each with `z3`, and then write a test that finds a bug in a wrong version of each. Then break the guard of the split rule and see which of your tests fails.
5. **C and memory (chapters 12 to 14).** Write the renderer, compile and run a function, then add `INDEX`, `LOAD` and `STORE`. Test with random programs against your interpreter. You are ready to move on when a swap of two array elements gives the same answer in both visiting orders.
6. **Trust and tinygrad (chapters 15 and 16).** Plant five bugs in your own code and see which tests catch them. Then follow the path through the tinygrad source of chapter 16.

After that, the syllabus continues with loops and movement ops, which is the week 3 material that chapter 16 previews.

Appendix F

Glossary

Each term is defined in the chapter where it first appears. This list repeats the definitions in one place, in alphabetical order.

AddrSpace the property of a node that says whether it is a value (ALU space) or a pointer to memory (MEM space).

AFTER an op that passes a pointer through and says that its access happens after the listed loads and stores. It is how memory order is written in the graph.

ALLOC an op for scratch memory that belongs to one call. Its contents are undefined until written.

ALU space the address space of values: bools, ints and floats that are computed, as opposed to memory.

Arg the third field of a `UOp`. It holds what the op needs besides its sources: the value of a constant, the slot of a parameter, the target dtype of a cast.

Associative an operation is associative if `(a op b) op c` equals `a op (b op c)`. Integer addition is, within wraparound. Float addition is not.

Bit pattern the 32 bits that store a value. Two floats are the same value to this book when their bit patterns are the same, so `0.0` and `-0.0` differ and all `nan`s are equal.

BUFFER an op for a global buffer that lives outside a call. It is passed to a call as an argument.

CALL an op whose first source is a function body and whose other sources are the arguments for the body's parameters.

Canonical form one chosen representation among several that mean the same. The rule `canonical order` puts constants on the right so that `a + b` and `b + a` become one node.

Confluent a rewrite system is confluent if every order of applying rules ends in the same result. The rule set of this book is not.

Const folding replacing an op whose sources are all constants by the constant it computes.

Contract the written meaning of every op, in `semantics.py`. The folder, the renderer and the tests all follow it.

DAG directed acyclic graph. A graph with arrows and no cycles. A `UOp` graph is one, where the arrows point from a node to its sources.

Dtype the type of a value: `bool`, `i32` or `f32`, or a pointer to one of them.

Floor division division that rounds toward minus infinity. `-7 // 2` is `-4`. The remainder then has the sign of the divisor.

FMA fused multiply add. One instruction that computes `a*b + c` with a single rounding. A compiler may use it unless told not to, and the result can differ from a multiply followed by an add.

Graph rewrite the process of replacing nodes of a graph by other nodes according to rules, until no rule applies.

Hash consing building a node by first looking for an existing node with the same op, sources and arg, and returning it if it exists.

Idempotent applying the operation twice gives the same result as applying it once. `simplify(simplify(u))` is `simplify(u)`.

Immutable never changed after it is built. A `UOp` is immutable.

INDEX an op that turns a pointer and an integer into a pointer to one element.

Interpreter here, a program that runs a graph directly on Python values, slowly, so that the compiled code can be checked against it.

IR intermediate representation. A form of the program that a compiler works on, between the source and the machine code. Here, the `UOp` graph.

LOAD an op that reads the element a pointer points to.

Mutant a copy of the code with one bug planted on purpose, used to test whether the tests can fail.

Nan not a number. The float result of operations such as `0.0 / 0.0` or `inf - inf`. Every comparison with a nan is false, except `!=`.

Node one `UOp`, a vertex of the graph.

Op the operation a node performs: `ADD`, `LOAD`, and so on.

Param short for parameter. A `PARAM` node stands for the argument of a call, found by its slot number.

Pattern a description of the shape of a node and its sources, with names for the parts. A `UPat` is a pattern.

PatternMatcher a list of rules, each a pattern and a function, with an index so that only the rules for a node's op are tried.

Peephole optimisation a rewrite that looks at a small piece of a program, such as a node and its sources, and replaces it by something cheaper.

Range here, the interval `[vmin, vmax]` that bounds the values of an integer node.

RAW, WAR, WAW read after write, write after read, write after write: the three ways two accesses to the same memory can depend on each other. **AFTER** edges express them.

Renderer the part of the compiler that turns a graph into source code, here C.

Rewrite rule a pattern and a function that returns a replacement node, or nothing.

Safe a node is safe if no integer operation in it can wrap around. The split rules only fire on safe nodes.

Shape the tuple of dimensions of a value. Here always `()` for a value and `(size,)` for a pointer.

SINK an op that holds several stores so that all of them stay in the graph.

Slot the number that connects a `PARAM` in a body to an argument of a `CALL`.

SSA static single assignment. A form in which every value is defined once. The text format of this book is in SSA form, with one numbered line for each node.

STORE an op that writes a value to the element a pointer points to.

Subnormal a float so close to zero that it has less precision than normal floats. The smallest positive float32 is a subnormal.

Topological sort an ordering of the nodes in which every node comes after its sources.

Total function a function that gives an answer for every input. Every op of the contract is total, including division by zero.

UB undefined behaviour. A program action for which the C standard places no requirements, such as signed integer overflow. The compiler may assume it never happens.

Ulp unit in the last place. The gap between a float and the next float. The error of a function is often given in ulps.

UOp the node type of this module: an op, a tuple of source nodes and an arg.

Wrap to reduce an integer modulo `2**32` into the range of `i32`. int32 addition and multiplication wrap.

z3 a solver from microsoft research that decides claims about bit vectors, integers and floating point numbers. A claim is checked by asking it for an input that breaks the claim.